VIETNAM NATIONAL UNIVERSITY, HO CHI MINH CITY

UNIVERSITY OF INFORMATION TECHN OLOGY

FACULTY OF INFORMATION SYSTEMS

REPORT

Data, Process and Object Modeling

TOPIC:

Stock Portfolio Management Platform

Instructor:

Instructor: Cao Thi Nhan

Instructor: Duong Phi Long

Group : 1

1	Tran Thi Kim Ngan	22520935
2	Ho Tan Loc	22520786
3	Le Trong Hoang Dung	22520281
4	Phan Huy Kien	22520709

INSTRUCTOR'S FEEDBACK

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

Ho Chi Minh City

TABLE OF CONTENTS

LIST OF FIGURES

Usecase Diagram

Acivity Diagram

Sequence Diagram

Class Diagram

User Interface

LIST OF TABLES

PROJECT SUMMARY

This project presents the design and analysis of a **Stock Portfolio Management Platform** that supports real-time market data access, portfolio tracking, and stock research for individual investors. The system is developed with a strong focus on data, process, and object modeling to ensure clarity, scalability, and maintainability. Key functionalities include user authentication, portfolio and watchlist management, real-time dashboard visualization, stock price history analysis, financial statement review, and community discussion features.

To address the limitations of traditional monolithic systems, the platform adopts a microservices-oriented architecture that enables independent service deployment, high availability, and low-latency data processing. Real-time market data is ingested via WebSocket connections, processed efficiently, cached for fast access, and persisted for historical analysis. UML diagrams such as use case, activity, sequence, class, and state diagrams are employed to model system behavior and structure comprehensively.

Overall, the project demonstrates how modern system modeling techniques and architectural design can be applied to build a robust, scalable, and user-centric stock analytics platform that supports informed investment decision-making

ACKNOWLEDGEMENTS

We extend our deepest gratitude to our instructors, **Cao Thi Nhan** and **Duong Phi Long**, for their invaluable guidance, unwavering support, and constructive feedback throughout the entire duration of the Data, Process and Object Modeling course and the subsequent development of this project. Their expertise and dedication were instrumental in shaping our understanding of system analysis principles and in successfully navigating the complexities of software development. Furthermore, we are thankful for all the resources, tools, and libraries that provided essential support for our work, enabling us to implement our designs effectively. Lastly, we appreciate the collaborative spirit, hard work, and commitment demonstrated by every member of our team, which were crucial in bringing this Apartment Management System project to a successful completion.

CHAPTER 1. Introduction and Project Overview

This introductory chapter outlines the project's overarching title and core objective: building a scalable platform for ingesting, processing, and serving real-time and historical financial data using a microservices architecture. Furthermore, the chapter details the critical purpose of the system, the expected tangible outcomes upon completion, and identifies the primary user groups the platform is designed to serve

1.1. Introduction

The Stock Analytics Platform is designed to provide real-time data processing and analytics for stock market data. Its primary objective is to offer a comprehensive system capable of ingesting large volumes of market data, processing it efficiently, and providing up-to-date information to end-users via intuitive web interfaces. The platform is built to support functionalities like real-time stock price updates, historical market data retrieval, and analysis of financial metrics, ensuring that users have access to accurate and timely information for making informed decisions in stock trading.

The project focuses on delivering high-performance, real-time data processing capabilities, leveraging modern technologies and architecture patterns to support scalability, fault tolerance, and resilience. By providing an integrated solution that combines both real-time data streams and batch processing, the system aims to offer seamless market insights across different stock assets. Additionally, the system prioritizes

the flexibility to scale with the increasing demand for market data and the expansion of supported assets and services.

The significance of the platform lies in its ability to integrate various data sources, including Alpaca WebSocket for real-time market data, Kafka for stream management, Postgres for persistent storage, and Redis Streams for high-speed data distribution. By doing so, the platform ensures low-latency data delivery while supporting robust analytics and visualization features.

1.2. Research Problems

The Stock Analytics Platform addresses several key challenges faced by traditional monolithic systems, particularly those related to inefficiencies, scalability, and maintenance. These challenges have significant implications for the system's performance, reliability, and ability to evolve over time.

1. **Inefficiencies in Traditional Monolithic Systems:**

In a monolithic architecture, all components of the system are tightly integrated into a single, large codebase. While this can be manageable in smaller systems, it becomes inefficient as the system scales. For a real-time stock analytics platform, where data must be processed and analyzed in real-time, these inefficiencies become particularly evident. In a monolithic system, different parts of the platform may struggle to handle the high volume and speed of data streams required for real-time stock updates, leading to slower response times and potential bottlenecks. For example, delays in

processing market data could directly impact the user experience, as stock prices and trading volumes need to be updated almost instantaneously.

2. **Scalability Issues and Handling Increased Traffic and Data**

As stock market data grows in volume and complexity, traditional monolithic systems face significant scalability challenges. A monolithic system typically requires scaling the entire application when certain parts need more resources, such as when processing a large influx of real-time stock data from multiple sources. For example, during high volatility in the market, real-time stock updates need to be processed and delivered with minimal delay. In such cases, scaling a monolithic system would require additional resources across the entire application, which can be both inefficient and costly. In contrast, a platform focused on real-time stock analytics needs to scale specific components independently to manage the increasing volume of market data, which is not always possible with a monolithic architecture.

3. **Issues with Maintaining and Upgrading Monolithic Systems**:

Monolithic systems can be difficult to maintain and update, especially as new features or services are introduced. In a real-time stock analytics platform, frequent updates and new features (such as new data sources or real-time analytics capabilities) are essential to keep up with market demands. However, the tightly coupled nature of monolithic systems means that updates to one part of the application often require modifications in other parts, creating complexity and increasing the risk of introducing errors. Moreover, the lack of flexibility in deploying individual components makes it

challenging to release new features or patches without significant downtime. In a rapidly changing stock market environment, where real-time responsiveness is critical, such downtime or delays can have negative consequences for the platform's users.

4. **Simplifying Maintenance through Smaller, Independent Services**:

Real-time stock analytics platforms require frequent updates to integrate new data sources, improve data processing algorithms, or add new features. Microservices simplify maintenance by breaking down the platform into smaller, more manageable services. Each service can be updated or maintained independently, minimizing downtime and reducing the risk of introducing bugs in unrelated parts of the system. For instance, when adding support for a new stock exchange or financial data provider, the relevant microservice can be updated without affecting the core functionalities of other services, such as market data visualization or real-time trading analysis. This modular approach to maintenance ensures that the platform can evolve rapidly to meet new business needs without disrupting the user experience.

5. **Enhanced Flexibility to Support Real-Time Market Requirements**:

A real-time stock analytics platform needs to deliver up-to-the-minute data with low latency. Microservices architecture provides the flexibility to design the system with optimal performance for real-time requirements. Services like real-time data ingestion, stream processing, and API services can be independently optimized for low-latency operations, ensuring that stock prices and other financial metrics are updated instantly. Additionally, the decoupling of services enables the platform to be more adaptive to

changes in market conditions, such as when adding new data sources or incorporating new analytical models, without the need to overhaul the entire system.

In conclusion, microservices architecture addresses the key challenges faced by traditional monolithic systems in the context of real-time stock analytics. By decoupling services, the platform can scale effectively, improve fault tolerance, and simplify maintenance, all of which are essential for meeting the real-time demands of stock market data processing and providing an optimal user experience. Through modularity and independent service management, microservices enable the Stock Analytics Platform to handle increasing data volumes, evolving business needs, and provide rapid, reliable insights to users.

1.3. Research Objectives

The Stock Analytics Platform is designed with a set of clear and measurable objectives that focus on solving key challenges related to providing timely, accurate, and actionable stock market insights. These objectives are centered on addressing user needs, particularly those around real-time data processing, system responsiveness, and ease of access to market information.

1. **Improving System Scalability and Real-Time Performance**:

A primary objective of this project is to ensure that the platform can handle a large volume of real-time stock market data while maintaining high performance. Users rely on the system to deliver up-to-the-minute stock prices, trading volumes, and other critical

financial metrics. As the volume of data increases, whether from additional stock tickers or more frequent market events, the platform must scale efficiently to meet these demands. The goal is to provide users with a seamless experience, ensuring that they have access to the most up-to-date information without delays or performance degradation, even during periods of market volatility.

2. **Ensuring Low Latency and High Availability for Real-Time Data**:

Real-time stock market analytics is a core feature of the platform, where every second counts. Users need immediate access to financial data to make informed decisions, especially in fast-moving markets. The platform's architecture aims to minimize latency, ensuring that data is processed and delivered to users with minimal delay. This objective directly addresses user needs by ensuring that they receive market updates in real time, which is essential for making timely trading decisions. Furthermore, the system is designed to maintain high availability, ensuring that users can access the platform without interruptions, even in high-demand situations.

3. **Providing Reliable and Accurate Market Insights**:

The platform's reliability and accuracy are crucial to meeting user expectations. Traders and investors depend on the system to provide trustworthy data to support their decision-making. Therefore, another key objective is to ensure that the platform delivers accurate market insights consistently. This involves not only providing real-time data but also integrating historical data, financial reports, and predictive analytics to support

informed decision-making. The goal is to help users make well-informed investment choices based on comprehensive, up-to-date market information.

4. **Meeting User/Organizational Needs**

The **Stock Analytics Platform** addresses several specific needs of both users and the organization, ensuring that it provides both value and responsiveness in a competitive market environment.

5. **Faster Time-to-Market for New Features**:

In the fast-paced financial market, the ability to rapidly introduce new features is a key advantage. The platform is designed to quickly adapt to new data sources, market trends, or user demands. For example, when adding support for a new stock exchange or financial instrument, the platform can integrate new data sources and deliver them to users without significant delays. This flexibility in development and deployment ensures that users have access to the latest market information and tools, enhancing their trading strategies.

6. **Improving System Resilience and Fault Tolerance**:

In a volatile market, system resilience is crucial. The platform must remain operational even in the face of technical issues or unexpected data influxes. By ensuring that different parts of the system operate independently and that failures can be isolated, the platform provides users with uninterrupted access to market data. This resilience

ensures that users can continue using the platform even if certain services experience issues, making the system more dependable and trustworthy.

7. **Providing a Seamless User Experience**:

A core objective of the platform is to provide a user-friendly interface that allows traders and investors to easily access and analyze real-time market data. The platform's design is focused on providing an intuitive user experience, with clear visualizations, real-time updates, and easy navigation between different data points. Users should be able to quickly view the stock prices, historical performance, and other financial metrics that are most relevant to them, without encountering performance lags or complex workflows.

8. **Supporting Informed Decision-Making**:

The platform’s ability to deliver accurate, up-to-date information plays a crucial role in helping users make informed decisions. By integrating real-time data with historical trends and other financial metrics, the platform empowers users to analyze market conditions and identify opportunities. This supports more strategic decision-making, helping traders and investors stay ahead of the market.

In conclusion, the **Stock Analytics Platform** is designed to meet the specific needs of its users by ensuring real-time performance, scalability, and accuracy. These objectives are centered around delivering timely, reliable, and accurate market data, ensuring that users can make informed decisions in a fast-paced, dynamic environment.

By addressing these user-centric objectives, the platform aims to provide a seamless, efficient, and resilient experience for stock market participants, enabling them to respond to market conditions and capitalize on trading opportunities.

1.4. Scope of the Project

The Stock Analytics Platform is designed to provide real-time data processing, analytics, and insights into stock market performance. The scope of this project encompasses several key features, modules, and services that are critical to enabling users to access up-to-the-minute stock data and financial metrics.

Covered Features:

1. **Real-Time Market Data Processing**:

The platform's core functionality revolves around the ingestion and processing of real-time stock market data. This includes live stock price updates, trading volumes, and other financial metrics sourced from Alpaca WebSocket and other APIs. The data is processed, normalized, and made available to users in real-time, ensuring timely and accurate market insights.

2. **Historical Data and Analytics**:

In addition to real-time updates, the platform integrates historical market data, allowing users to analyze past trends, price movements, and other financial metrics. This data helps traders and investors make informed decisions based on both current and historical market conditions.

3. **Portfolio Management**:

The platform offers basic portfolio management functionalities, where users can track their investments and monitor stock performance. Users can add stocks to their portfolio and view detailed performance metrics over different time periods, helping them make strategic decisions based on their holdings.

4. **Financial Metrics and Reporting**:

Users can access detailed financial reports, such as earnings reports, dividends, and historical stock performance. These reports are integrated with real-time market data, providing a comprehensive view of a stock's performance and financial health.

5. **Market Data Visualization**:

The platform includes visual tools such as heatmaps, trend charts, and price analysis graphs, which provide users with a clear and intuitive view of market trends, making it easier to analyze complex financial data.

Limitations

While the Stock Analytics Platform offers comprehensive functionality, there are several limitations that should be considered in terms of the technologies used, time constraints, and features that are excluded from the current version.

Technologies Used:

1. **Docker and Kubernetes**:

The platform relies on Docker for containerization of microservices and Kubernetes for orchestration. While these technologies enable scalability and flexibility in deployment, their complexity may pose challenges for initial setup and management, especially when scaling the system to support more users or additional services.

2. **Node.js and FastAPI**:

The backend services are implemented using Node.js for the API gateway and FastAPI for handling high-performance API requests. While both are suitable for real-time applications, there are certain use cases where other frameworks or languages may provide more optimized solutions. For example, Python-based services (like FastAPI) are used for data processing, while Node.js is better suited for handling I/O operations.

3. **Redis and Kafka**:

While Redis Streams and Kafka provide efficient real-time data handling, they also introduce complexity in managing distributed systems and ensuring data consistency across services. Configuring and maintaining these technologies requires careful tuning and monitoring to prevent bottlenecks or data loss in high-volume environments.

Time Constraints or Features Excluded:

1. **Real-Time Trading Execution**:

The current version of the platform does not include functionality for real-time trade execution, meaning users cannot place live orders or execute trades through the

platform. While this feature could be added in future versions, it is excluded from the current scope to focus on data processing and analytics.

2. **Advanced Portfolio Analytics**:

The platform offers basic portfolio management features, but advanced analytics such as risk analysis, portfolio diversification, and performance optimization are not included in this version. These features are considered for future enhancements and could be integrated in subsequent iterations.

3. **External Market Data Sources**:

While the platform integrates with Alpaca for real-time market data, other external data sources, such as Bloomberg, Reuters, or Yahoo Finance, are not included in the current scope. Integrating additional data sources could provide broader market coverage but is excluded due to time and resource constraints.

4. **Mobile Application**:

The platform is designed for web-based use, and currently, there is no mobile application to access the platform on smartphones or tablets. Future iterations of the platform could consider developing a mobile version to increase accessibility.

5. **Security Features**:

Although the platform includes basic security measures for data access, more advanced features such as two-factor authentication (2FA), role-based access control

(RBAC), and data encryption at rest are not fully implemented in this version. These will be added in future releases to further enhance data protection and user security.

In summary, while the Stock Analytics Platform provides a robust set of features focused on real-time market data processing and analytics, there are certain limitations related to the technologies used, features excluded in the current scope, and future enhancements. These limitations are considered to be part of the project's growth roadmap, with future versions intended to expand the platform's capabilities and address additional user needs.

CHAPTER 2. REQUIREMENT SPECIFICATION

This chapter presents the requirement specification of the Stock Portfolio Management Platform. It analyzes the current situation through surveys and interviews, identifies key user needs and system pain points, and translates them into functional and non-functional requirements. The chapter provides a clear foundation for subsequent system analysis and design by ensuring that all requirements are well-defined, realistic, and aligned with user expectations.

2.1. Situation Survey / Survey of the Current Situation

Understanding Existing Systems

In the context of stock market analytics, existing systems typically rely on monolithic architectures or traditional relational database management systems (RDBMS) to process, store, and manage market data. These systems are often designed to handle basic market information, such as stock prices, trading volumes, and other financial metrics. While they serve their purpose, the existing systems face several limitations when it comes to handling real-time, high-frequency market data, providing scalability, and ensuring high availability.

Monolithic architectures, which bundle multiple functionalities into a single application, are common in many legacy systems. These systems tend to have tightly coupled components, where even a small change in one module can affect the rest of the application. This approach can create bottlenecks, particularly when dealing with large

volumes of real-time data from various sources, such as stock exchanges or third-party data providers. Additionally, legacy systems often struggle with scaling, especially under high-demand situations where real-time data updates are critical. As market conditions change rapidly, the inability to scale specific components independently can severely limit the system's performance and responsiveness.

Current systems also typically rely on traditional batch processing, where data is collected and processed at scheduled intervals. While this approach can work for certain financial metrics, it is inadequate for real-time trading or decision-making, where minute-to-minute or even second-to-second updates are crucial. These systems do not provide users with the real-time insights required for immediate trading decisions, which is a significant drawback in today's fast-paced financial markets.

Furthermore, maintenance and upgrades of these legacy systems are often cumbersome and prone to downtime. Monolithic systems require significant effort to modify or update, as any change in one module often necessitates changes across the entire application. This lack of modularity increases both the complexity and risk of introducing errors during system updates or deployments. Given that market data systems are continuously evolving, the inability to make quick adjustments or integrate new features in a timely manner places these legacy systems at a competitive disadvantage.

Identifying Pain Points

The **Stock Analytics Platform** aims to address several pain points inherent in the existing systems. These issues primarily revolve around performance bottlenecks, lack of

scalability, difficulty in real-time data processing, and challenges with system maintenance and upgradeability.

1. **Scalability and Performance**:

One of the primary pain points of existing systems is their inability to scale efficiently to handle increased volumes of data. As the stock market grows and the number of trading events increases, legacy systems often struggle to keep up with the demand for real-time processing. In a stock analytics platform, where high-frequency data is generated constantly, the inability to scale specific components of the system (e.g., data ingestion, processing, or analytics) can lead to delays in data delivery and processing, which directly impacts users' ability to make timely decisions.

2. **Real-Time Data Processing**:

Legacy systems that rely on batch processing are ill-equipped to handle the dynamic, fast-moving nature of stock market data. This creates a major pain point for users who require up-to-the-minute data to make informed trading decisions. The lack of real-time capabilities means that users may not receive the necessary data in time, resulting in missed opportunities or suboptimal trading decisions. Furthermore, the ability to process vast amounts of market data in real time is essential for providing accurate analytics and insights, something that current systems are unable to achieve efficiently.

3. **System Maintenance and Upgrades**:

As existing systems grow and evolve, the process of maintaining and upgrading monolithic applications becomes increasingly difficult. The tightly coupled nature of these systems means that small updates or feature additions often require significant changes across the entire platform, leading to increased development time and a higher risk of introducing errors. Moreover, the deployment of updates in such systems often involves downtime, which can affect users' access to critical market data during important trading hours. This lack of flexibility in maintaining and evolving the system prevents organizations from quickly adapting to new market conditions or implementing new functionalities that could benefit users.

4. **Inability to Integrate New Features or Data Sources**:

The current systems also struggle with integrating new market data sources or external services. For example, if a new data provider is introduced, the process of integrating this source into a monolithic system can be time-consuming and complex, as it often requires changes across multiple components of the system. The lack of modularity in these systems prevents them from being flexible enough to integrate new features or services quickly, thereby limiting their ability to keep up with industry advancements or user demands for new capabilities.

2.1.1. Interviews

To better understand the specific challenges faced by users and stakeholders, a series of interviews were conducted with relevant parties, including platform users, domain experts, and business stakeholders. These interviews provided valuable insights

into the requirements, expectations, and pain points that the **Stock Analytics Platform** must address. The following table summarizes the key questions posed during these interviews, along with the answers gathered.

General Survey **System:** Stock Analytics Platform **Creator:** Ho Tan Loc **Created Date:** 2/10/2025				
Ord	**Topic**	**Requirement**	**Start**	**End**
1	Real-Time Data Accuracy	To gather insights on how users expect accurate and up-to-date stock data for their decision-making.	[Date]	[Date]
2	User Experience & Interface	To understand user expectations for a user-friendly interface and easy data visualization.	[Date]	[Date]
3	Customizable Alerts	To explore the need for customizable alerts and notifications based on stock price movements.	[Date]	[Date]
4	Portfolio Management	To gather feedback on how users wish to manage and track their investment portfolios on the platform.	[Date]	[Date]

5	Historical Data & Reports	To understand how users prefer to access historical market data and generate detailed reports.	[Date]	[Date]

Survey Form **Interviewee:** Nguyen Van Nam **Date:** 3/10/2025		
Topic	Questions	Records
1. Real-Time Data Accuracy	1. How important is it for you to receive stock price updates in real time?	Users emphasized the need for immediate stock price updates to make fast, informed decisions, especially during market volatility.
	2. What kind of data accuracy is crucial for your trading or investment decisions?	Users require precise data on stock prices, volumes, and market trends, as even minor discrepancies can lead to significant financial implications.
	3. How do you ensure the accuracy of the data you receive from the platform?	Users want verification methods such as reliable data sources and real-time syncing to avoid delays or inaccuracies in price reporting.
2. User Experience & Interface	1. What features would you want to see in the user interface to make it easier for you to navigate through market data?	Users prefer a clean, intuitive layout with easy access to key information like stock charts, price history, and news.

	2. How should stock data be presented to you for easy understanding?	Users expect clear visualizations, such as charts, heatmaps, and trend lines, to help them quickly grasp the stock's movement and performance.
	3. What information should be displayed at the top of the dashboard for a quick overview of market performance?	Users suggested showing high-level metrics like current prices, market trends, and recent performance, with quick links to more detailed data.
3. Customizable Alerts	1. Would you prefer to set alerts based on stock price thresholds or news events?	Users prefer the ability to set customizable alerts based on price movements and important news events that might affect their investments.
	2. How would you like to receive these alerts (e.g., email, mobile notifications, desktop alerts)?	Users want real-time push notifications for stock price changes, with customizable channels like mobile notifications or email alerts.
	3. How frequent do you want updates or alerts for stock price changes or market news?	Users prefer frequent updates during market hours, with a priority for significant price changes or major

		news related to stocks in their portfolio.
4. Portfolio Management	1. What features would help you track and manage your investment portfolio effectively on the platform?	Users want portfolio tracking, performance analysis, and visual indicators of gains and losses to help them assess their investment strategy.
	2. How important is it for you to see a breakdown of your portfolio by sector, asset type, or individual stocks?	Users prefer to have portfolio breakdowns that show the composition of their investments, such as sector allocations, stock percentages, and historical performance.
	3. Do you need tools for adjusting your portfolio based on market conditions? If so, what kind of tools?	Users would like to have tools that allow them to adjust their portfolio dynamically, such as rebalancing suggestions or the ability to add/remove stocks quickly.
5. Historical Data & Reports	1. How often do you access historical stock data for making investment decisions?	Users access historical data regularly to analyze trends and assess the performance of stocks over time,

		especially for long-term investment strategies.
	2. What specific types of financial reports or metrics do you need to access for your investment strategy?	Users are interested in reports such as earnings reports, price histories, and performance comparisons across different timeframes (e.g., daily, monthly, yearly).
	3. How would you like historical data and reports to be presented (e.g., in graphs, tables, downloadable files)?	Users prefer downloadable reports and visual graphs that clearly highlight trends, performance, and volatility over specific periods.

Purpose of This Survey:

The primary aim of this survey is to collect feedback from users regarding their expectations and preferences for key features in the Stock Analytics Platform. This information will guide the platform's design to ensure it meets user needs for real-time market data, portfolio management, customizable alerts, and historical data reporting. The insights gathered will ensure that the platform provides a highly effective, user-friendly experience, allowing users to make informed investment decisions based on real-time and historical data.

2.1.2. Survey of the Organization / Current Organizational Situation

Challenges within the Organization

In any organization, the efficiency and scalability of its technology infrastructure are crucial to supporting business goals and delivering value to customers. The **Stock Analytics Platform** addresses several challenges that are commonly faced by organizations in managing and processing large volumes of real-time data, ensuring data consistency across multiple systems, and facilitating seamless integration between various internal and external components. These challenges often arise from the limitations of traditional, monolithic systems, siloed data sources, and inefficient workflows, all of which can impede the organization's ability to respond to market changes effectively.

1. **Siloed Systems and Lack of Integration:**

One of the primary challenges within the organization is the use of siloed systems that are not integrated, making it difficult to maintain consistency across different platforms and data sources. For example, historical data may reside in one system, real-time market data in another, and customer-related data in yet another. This fragmentation complicates data access, analysis, and reporting, leading to inefficiencies and delays in decision-making. It also makes it challenging to provide a unified view of market performance, user activity, and financial metrics. The **Stock Analytics Platform**, by adopting a **microservices architecture**, will decouple these silos and ensure that each service focuses on a specific functionality (e.g., data ingestion, real-time processing, portfolio management). This decoupling allows the platform to integrate different data

sources seamlessly, providing a unified and up-to-date view of all market data, enabling faster and more informed decisions.

2. **Inefficiencies in Data Management**:

In the current system, data management processes are often inefficient, with limited automation and manual interventions required for data ingestion, processing, and analysis. The process of collecting and normalizing data from various external sources is time-consuming, leading to delays in providing real-time updates to end-users. Additionally, maintaining data accuracy and consistency across different databases and services can be difficult and error-prone, especially when handling large volumes of data. The **Stock Analytics Platform** aims to address these inefficiencies by automating data ingestion, streamlining data processing, and integrating data pipelines for real-time and historical analysis. By leveraging microservices to handle each aspect of data management independently, the platform ensures that data flows seamlessly through the system without unnecessary delays, ultimately providing real-time insights to users.

3. **Scalability and Performance Issues**:

Another challenge faced by the organization is the limited scalability of its current system. As the volume of market data grows and more users access the platform simultaneously, the system struggles to scale effectively. Scaling a monolithic system often requires scaling the entire application, which is not efficient and can lead to over-provisioning resources. The platform's real-time processing needs, such as tracking stock prices, trading volumes, and market trends, require a scalable infrastructure that can

dynamically adjust to varying workloads. The **Stock Analytics Platform** resolves this issue by leveraging microservices, which can be scaled independently to meet specific demands. For example, the service handling real-time data ingestion can be scaled independently from the services managing historical data, ensuring that the platform can handle large volumes of concurrent data and users without performance degradation.

4. **Difficulty in Maintaining and Upgrading the System**:

Maintaining and upgrading the existing system is another significant challenge. The monolithic structure of the current system makes it difficult to make incremental changes or introduce new features without affecting the entire application. This lack of flexibility leads to extended downtime during maintenance periods and increased risk of introducing errors when deploying updates. Additionally, as the platform needs to evolve to accommodate new stock exchanges, financial data sources, and market analysis features, the current system is not designed for rapid iteration or adaptation. The **Stock Analytics Platform**, by adopting a microservices approach, enables independent updates and upgrades to each service. This modularity allows for faster deployment of new features, improved testing, and reduced downtime, all of which contribute to a more agile and flexible system that can evolve in response to user needs and market changes.

5. **Limited Real-Time Insights**:

The existing system is not designed to provide the level of real-time insights required for modern stock market analysis. Traders and investors need up-to-the-minute data on stock prices, trading volumes, and other financial metrics to make timely

decisions. However, the current system's reliance on batch processing or periodic updates makes it challenging to deliver real-time data consistently. The **Stock Analytics Platform** addresses this limitation by incorporating real-time data ingestion and processing capabilities, ensuring that users receive accurate and current market data without delays. By integrating technologies like WebSockets for live data streaming and using Kafka for event-driven communication, the platform enables real-time insights that are critical for effective market analysis and trading decisions.

How the System Will Resolve These Challenges

The **Stock Analytics Platform** leverages microservices architecture to address the key challenges of siloed systems, inefficient data management, scalability, and real-time data processing. By decoupling services, the platform provides flexibility, scalability, and fault tolerance, ensuring that each service can evolve independently to meet specific requirements. Furthermore, the platform's focus on automation and real-time data processing enables it to handle large volumes of stock market data efficiently, providing users with accurate and timely insights.

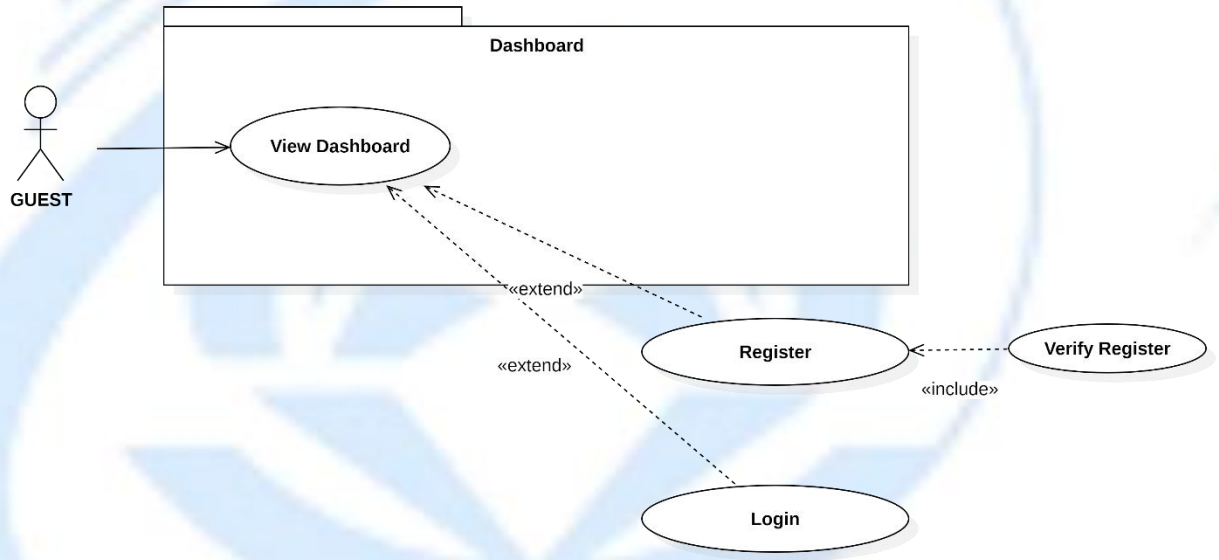
The adoption of microservices will also simplify system maintenance and upgrades, as each service can be updated or deployed independently, reducing downtime and minimizing the risk of errors. With a focus on integration, the platform will eliminate data silos and ensure that all components of the system work together seamlessly, providing a unified view of market data for users.

By addressing these organizational challenges, the **Stock Analytics Platform** will not only enhance operational efficiency but also provide a more reliable, scalable, and user-friendly system for real-time stock market analysis and decision-making.

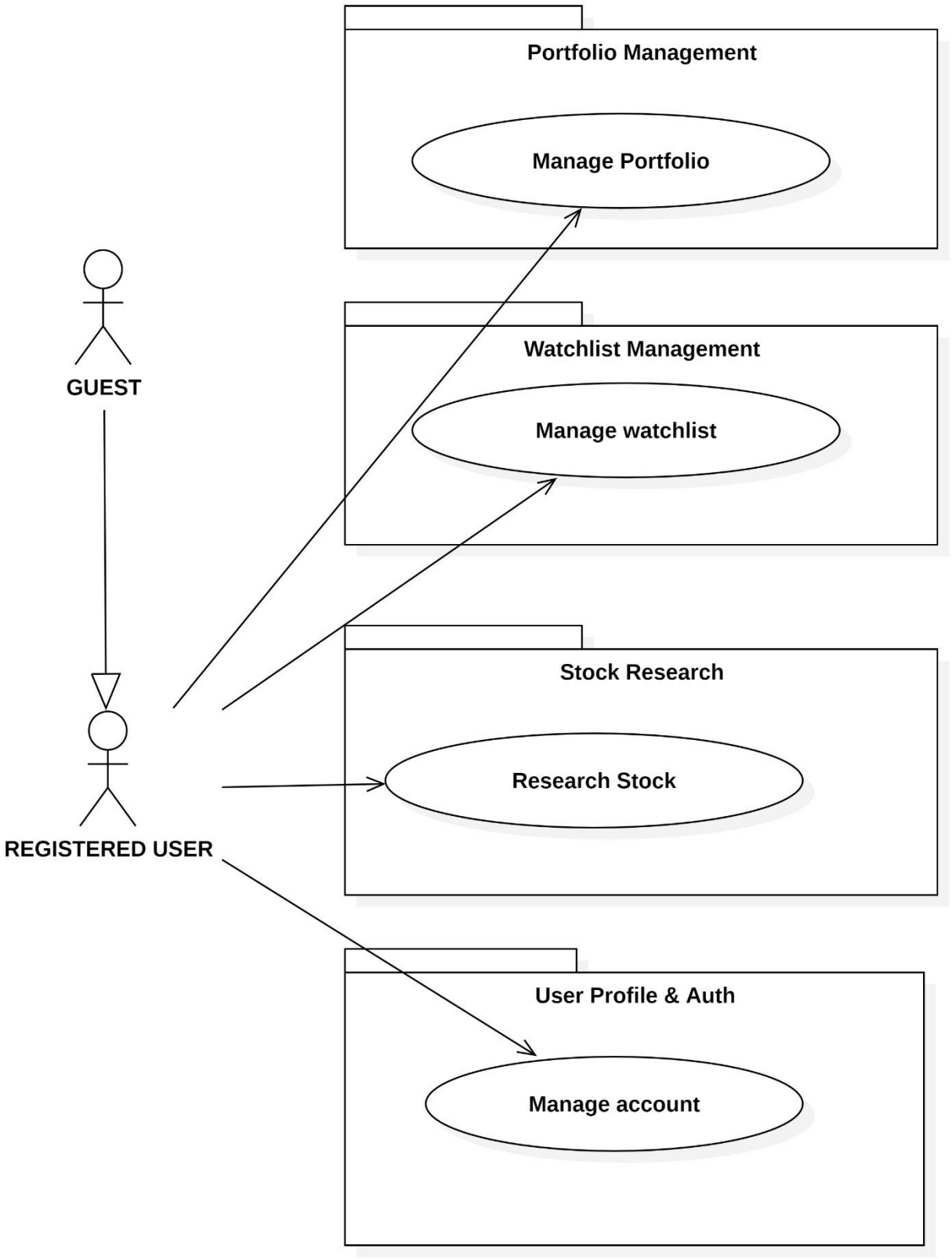
CHAPTER 3. System Analysis and Design

This chapter focuses on the system analysis and design of the Stock Portfolio Management Platform. It defines system actors, use cases, and interaction flows, and models system behavior using use case, activity, sequence, class, and state diagrams. The chapter ensures that system structure and behavior are clearly represented to support accurate implementation and future scalability.

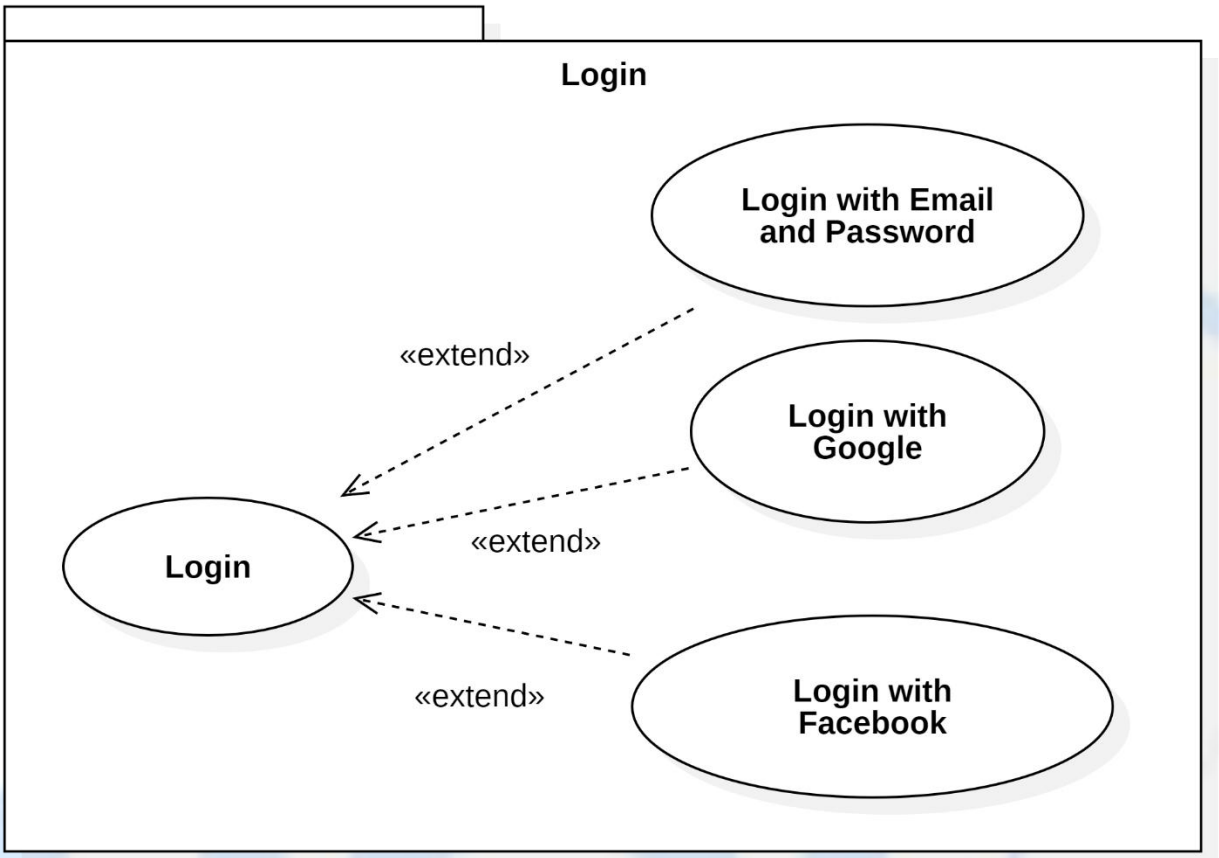
3.1. Use Case Diagram



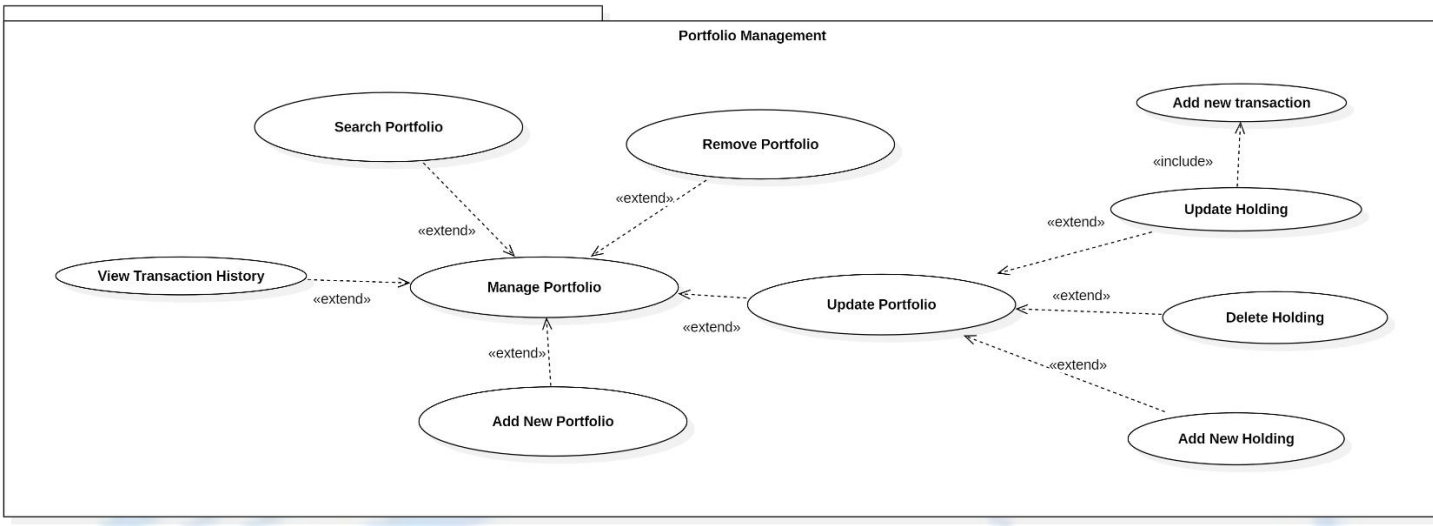
Usecase Diagram 1: Use Case Diagram for GUEST



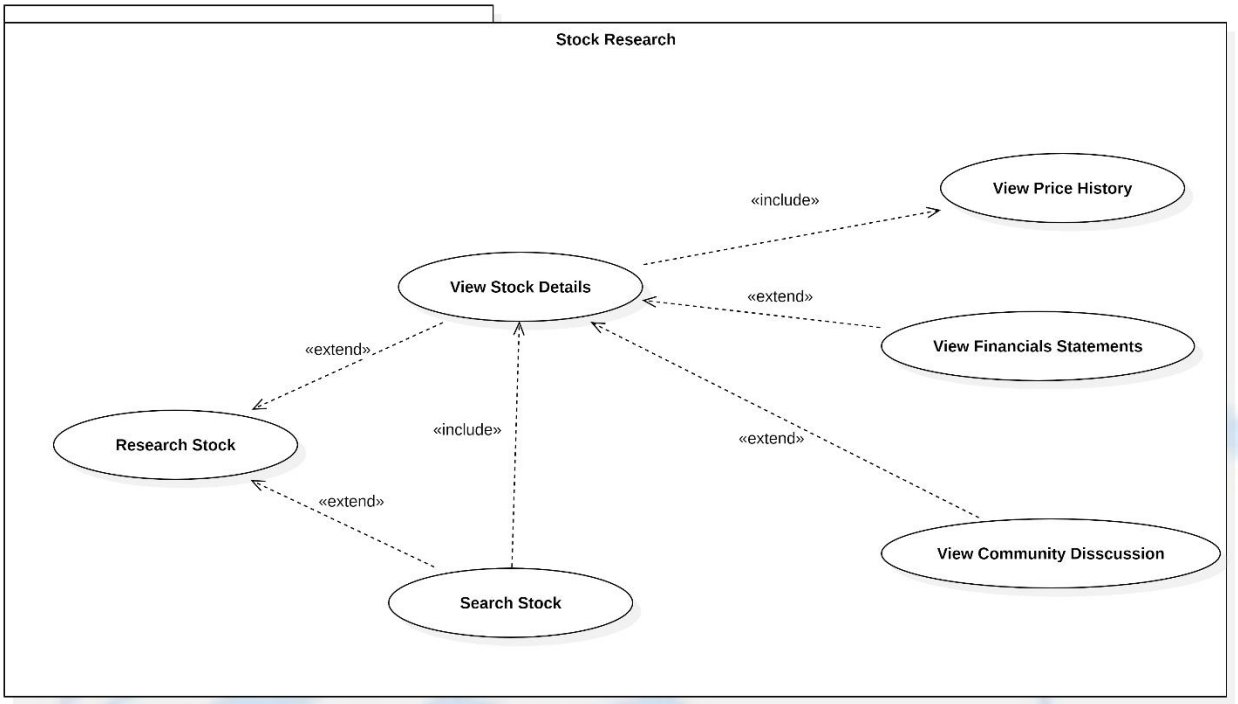
Usecase Diagram 2: Use Case Diagram for REGISTERED USER



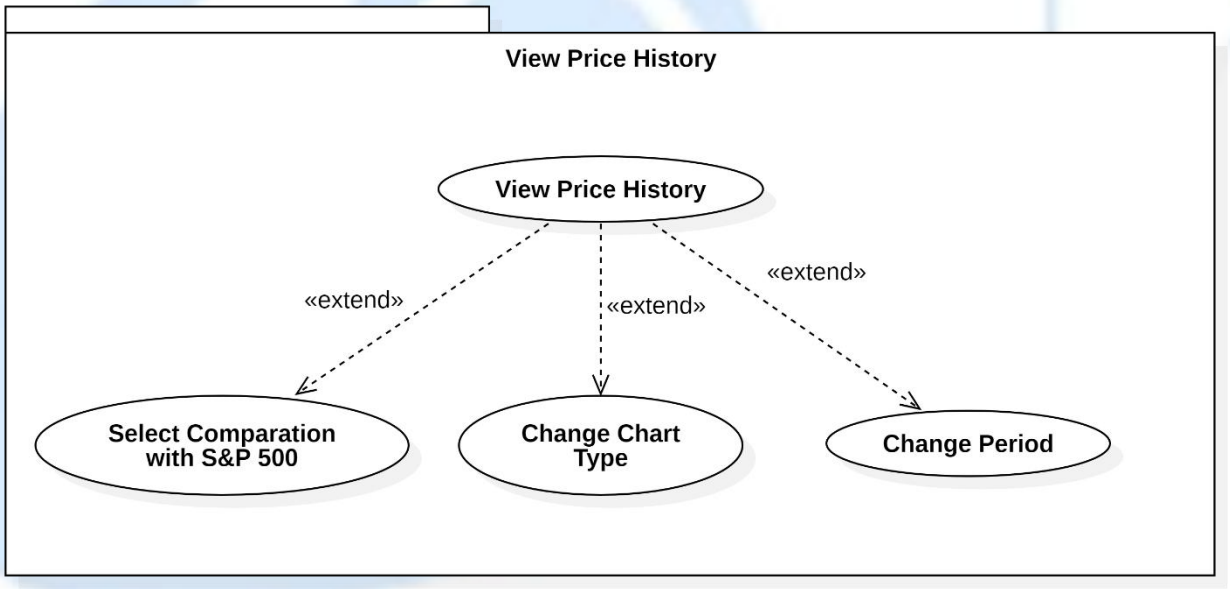
Usecase Diagram 3: Login



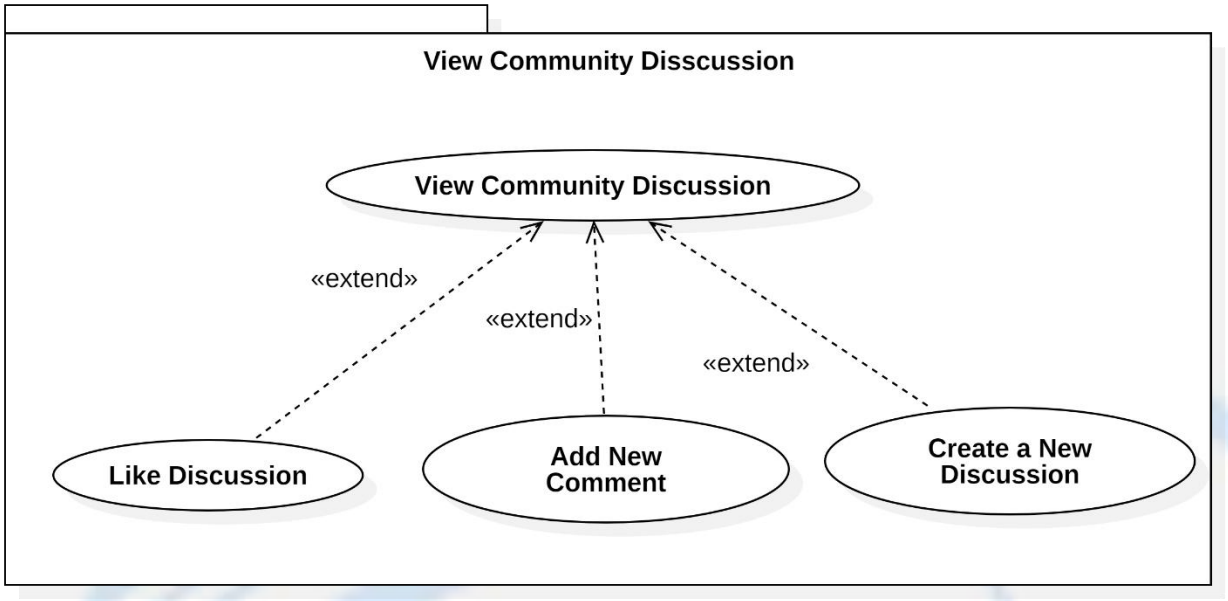
Usecase Diagram 4: Portfolio Management



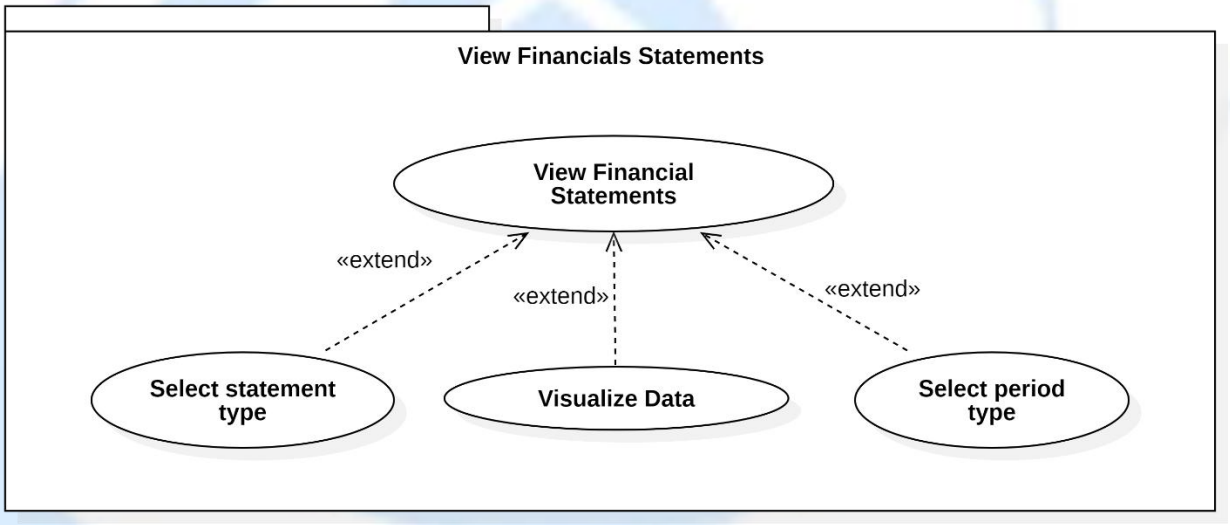
Usecase Diagram 5: Stock Research



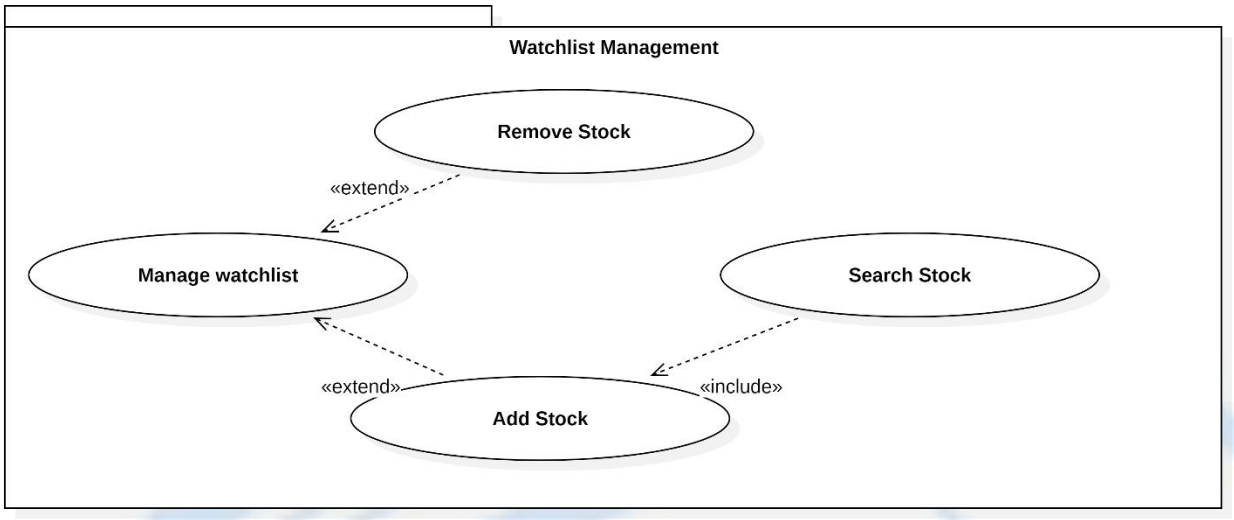
Usecase Diagram 6: View Price History



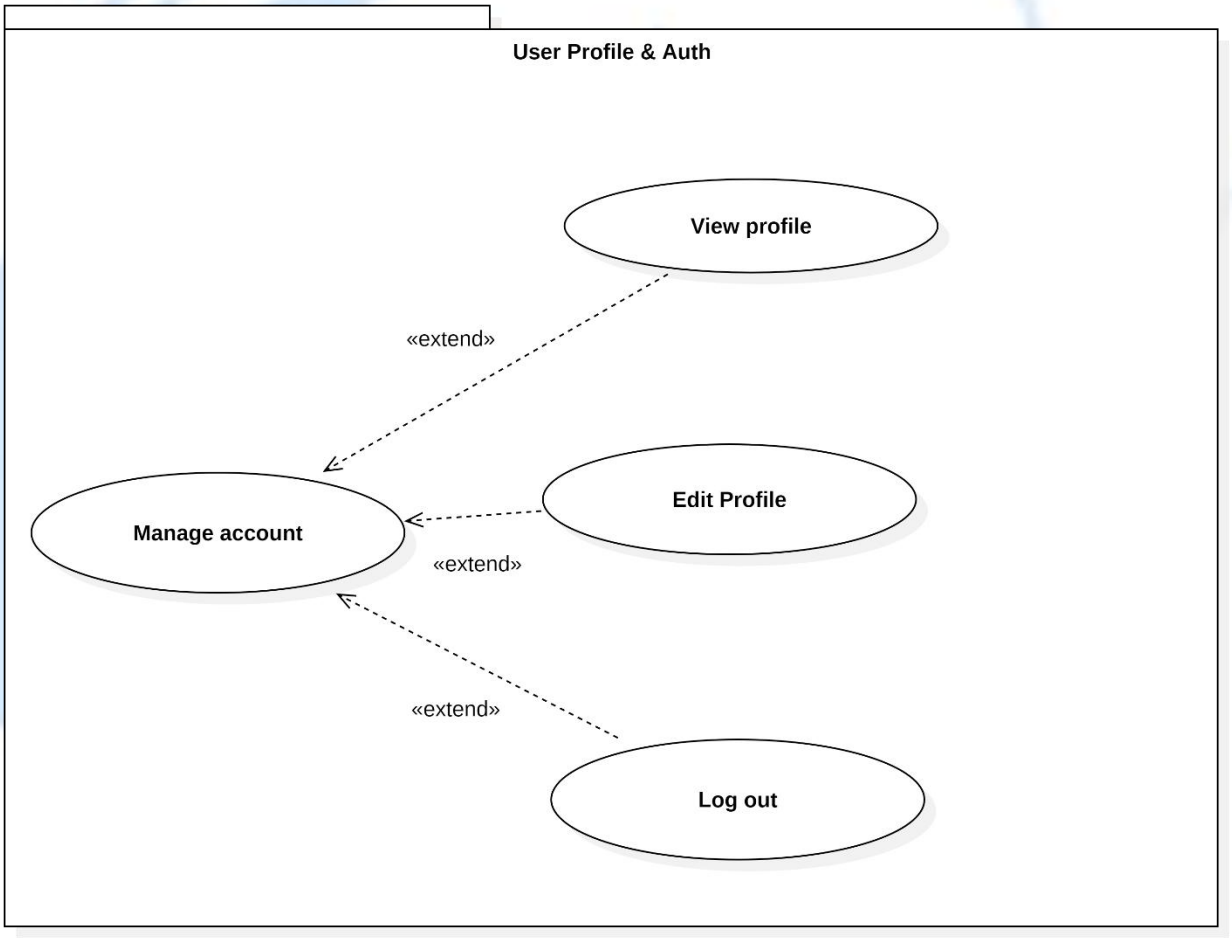
Usecase Diagram 7: View Community Discussion



Usecase Diagram 8: View Financials Statements



Usecase Diagram 9: Watchlist Managemen



Usecase Diagram 10: User Profile & Auth

3.1.1. List of actors

Table 3.1: Table list of actors

Index	Actors	Meaning
1	Guest	People who can access limited features of the system without signing in.
2	Registered User	People who are signed in and have full access to all services of the system.

3.1.2. List of use-case

1. GUEST

Table 3.2: Table usecase of Guest

Index	Use-case	Description
1	View Dashboard	Guests can view the dashboard with basic market data and information.
2	Register	Guests can sign up to access full system services.
3	Verify Registration	Guests can verify account
4	Login with email and password	Guests can log in to access their account and other functionalities.
5	Login with Google	Guests can log in to their account using their Google

		account credentials.
6	Login with Facebook	Guests can log in to their account using their Facebook account credentials.

2. REGISTERED USER

Table 3.3: Table Registered User

Index	Use-case	Description
1	Manage Portfolio	Users can add, remove, and update their portfolio, manage positions, and track performance.
2	Manage Watchlist	Users can add stocks to their watchlist and remove them.
3	Research Stock	Users can search for stocks, view details, financial statements, and related news.
4	Manage Account	Users can view and edit their profile and log out.

Table 3.4: Detailed Use-case Breakdown for Registered User

Index	Use-case	Sub-use-cases	Description
1	Manage	Search Portfolio	Users can search through their existing

	Portfolio		portfolios.
		Add New Portfolio	Users can create new portfolios.
		Remove Portfolio	Users can delete a portfolio.
		Add New Holding	Modify the number of stocks in their portfolio.
		Delete Holding	Remove stocks from the portfolio.
		Update Holding	Add additional stocks to the portfolio.
		View Transaction History	View transaction of each holding
2	Research Stock	Search Stock	Users can search for a specific stock by its symbol or name.
		View Price History	Access historical price data for a stock.
		View Financial Statements	Review the financial performance of the company.
		View Community Discussion	View discussions and opinions about the stock from the community.
3	Manage Account	View Profile	Users can view their account profile details.
		Edit Profile	Users can edit their personal information and settings.
		Log Out	Users can log out from the platform.

4	Manage Watchlist	Search Stock	Users can search and add stocks to their watchlist.
		Add Stock	Add stocks to the watchlist.
		Remove Stock	Remove stocks from the watchlist.

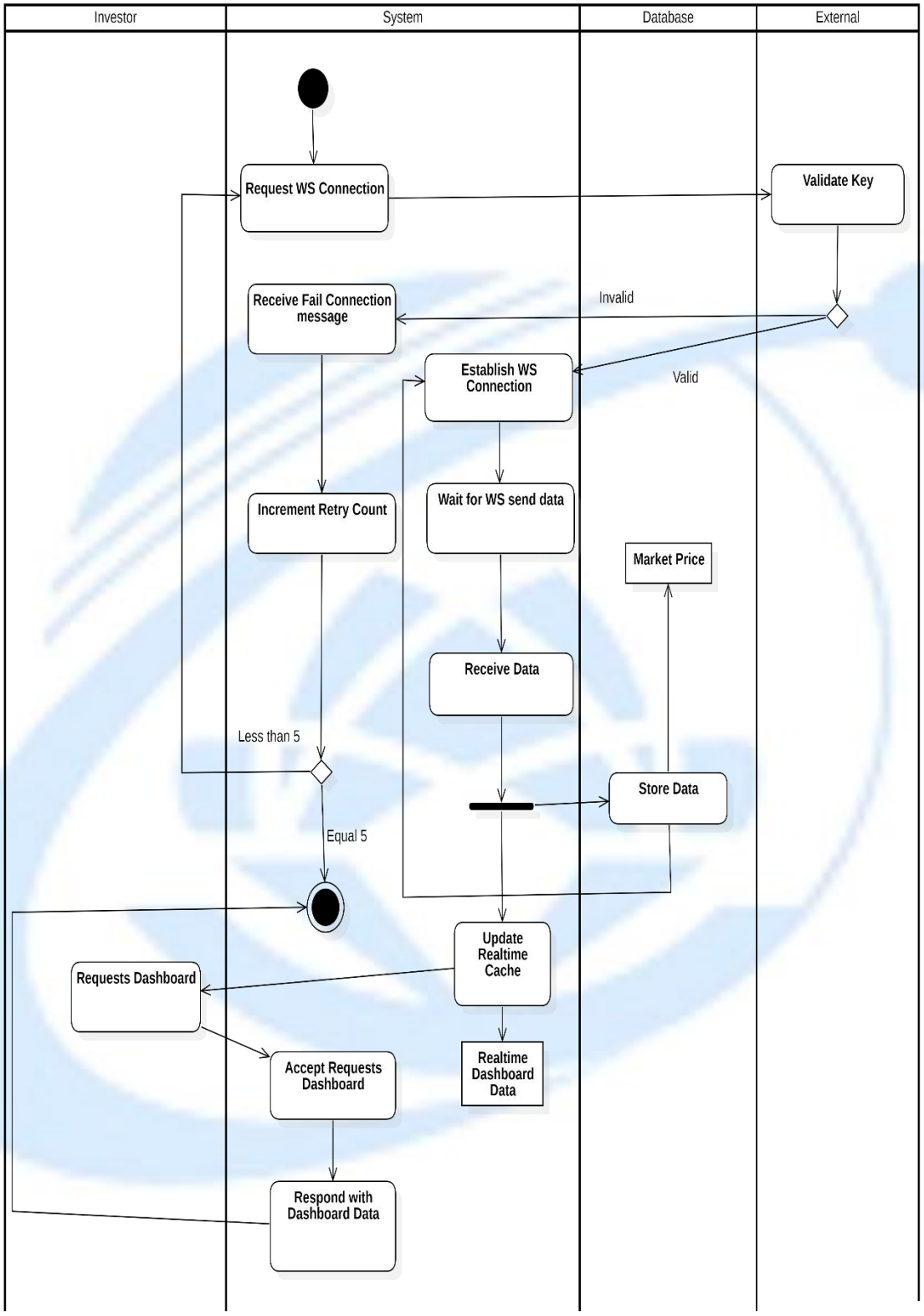
3.2. Use Case Specification and Activity Diagram

3.2.1. View Dashboard

Table 3.5: View Dashboard usecase description

Attribute	**Description**
Use-case Name	Load Realtime Dashboard
Description	The system maintains a persistent WebSocket connection with an external market data provider to continuously receive real-time market prices. Incoming data is validated, stored in the database, and cached for real-time access. When the investor requests the dashboard, the system retrieves the latest cached data and returns it immediately without re-establishing the WebSocket connection.
Trigger	The investor access to homepage.
Pre-condition	The system has a valid API key for the external market data service. The system is able to establish or retry a WebSocket connection.
Post-condition	The dashboard displays the most recent real-time market data.

	Real-time cache is updated continuously in the background.
Basic flow	1. The system requests a WebSocket connection to the external market data service. 2. The external service validates the API key. 3. The system establishes the WebSocket connection. 4. The system waits for market data sent via WebSocket. 5. The system receives real-time market price data. 6. The system stores the data in the database. 7. The system updates the real-time cache. 8. The investor requests the dashboard. 9. The system accepts the dashboard request. 10. The system retrieves real-time data from the cache. 11. The system responds with dashboard data to the investor.
Alternative flow	2a. The system receives a failed connection message. 2b. The system increments the retry counter. The use case continues with step 1
Exception flow	



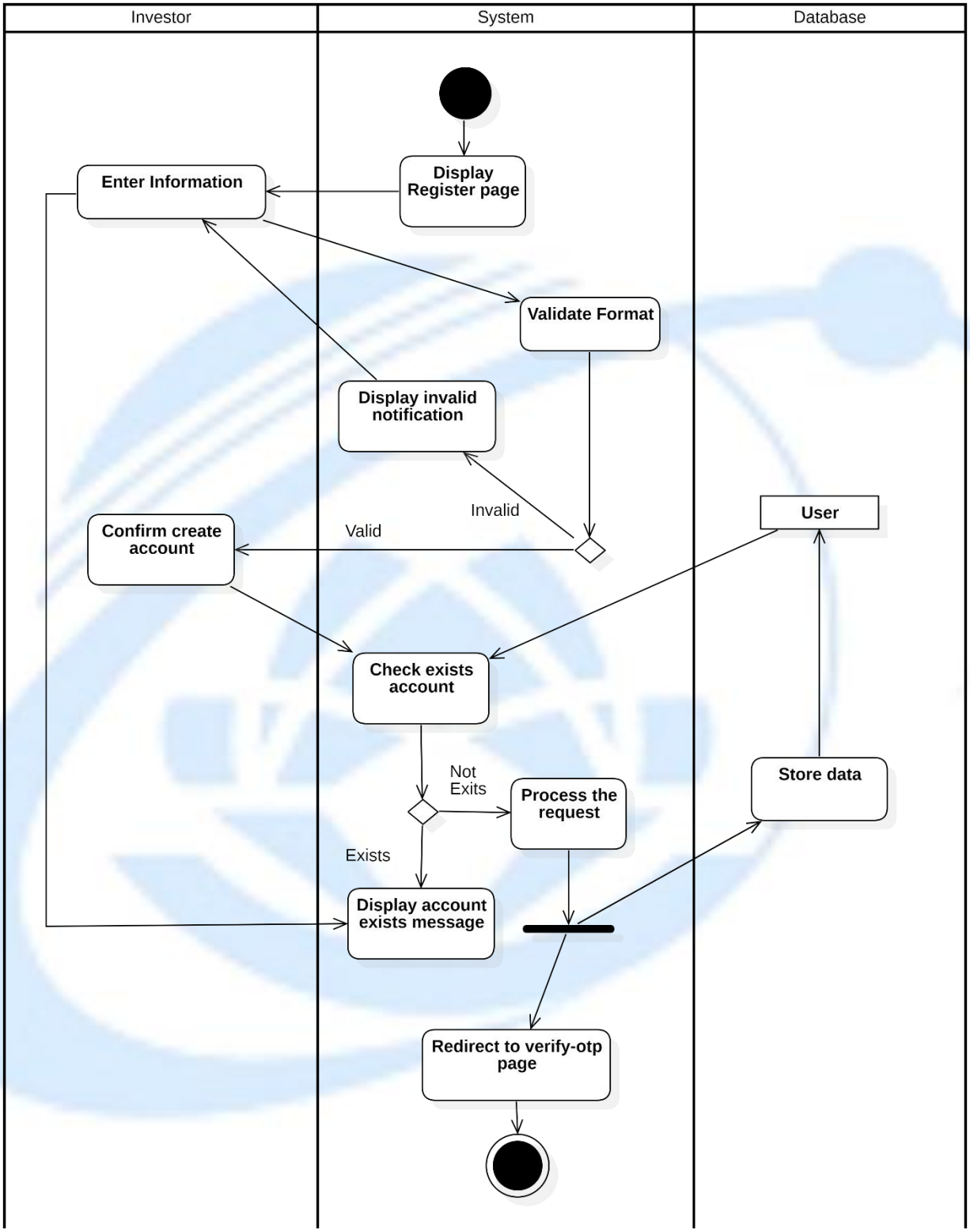
Activity Diagram 1: View Dashboard

3.2.2. Register

Table 3.6: User Registration uscase description

Attribute	Description
Use-case Name	Register Account
Description	The system allows a user to register a new account by entering their information. The system validates the input data, checks if the account already exists, and redirects the user to an OTP verification page upon successful registration.
Trigger	The user clicks to "Register" button.
Pre-condition	The user is not already registered in the system. The user has the necessary data to create an account.
Post-condition	The user is registered in the system. The system stores the user data in the database. The user is redirected to the OTP verification page.
Basic flow	1. The system displays the registration page. 2. The user enters their information (e.g., name, email, password). 3. The system validates the format of the entered data.

	4. The system checks if the account already exists. 5. If the account does not exist, the system processes the request and stores the data in the database. 6. The system redirects the user to the OTP verification page.
Alternative flow	3a. If the format is incorrect, the system displays an invalid notification. The use case continues with step 2 6a. If the account already exists, the system displays a message indicating that the account is already registered. The use case continues with step 2
Exception flow	



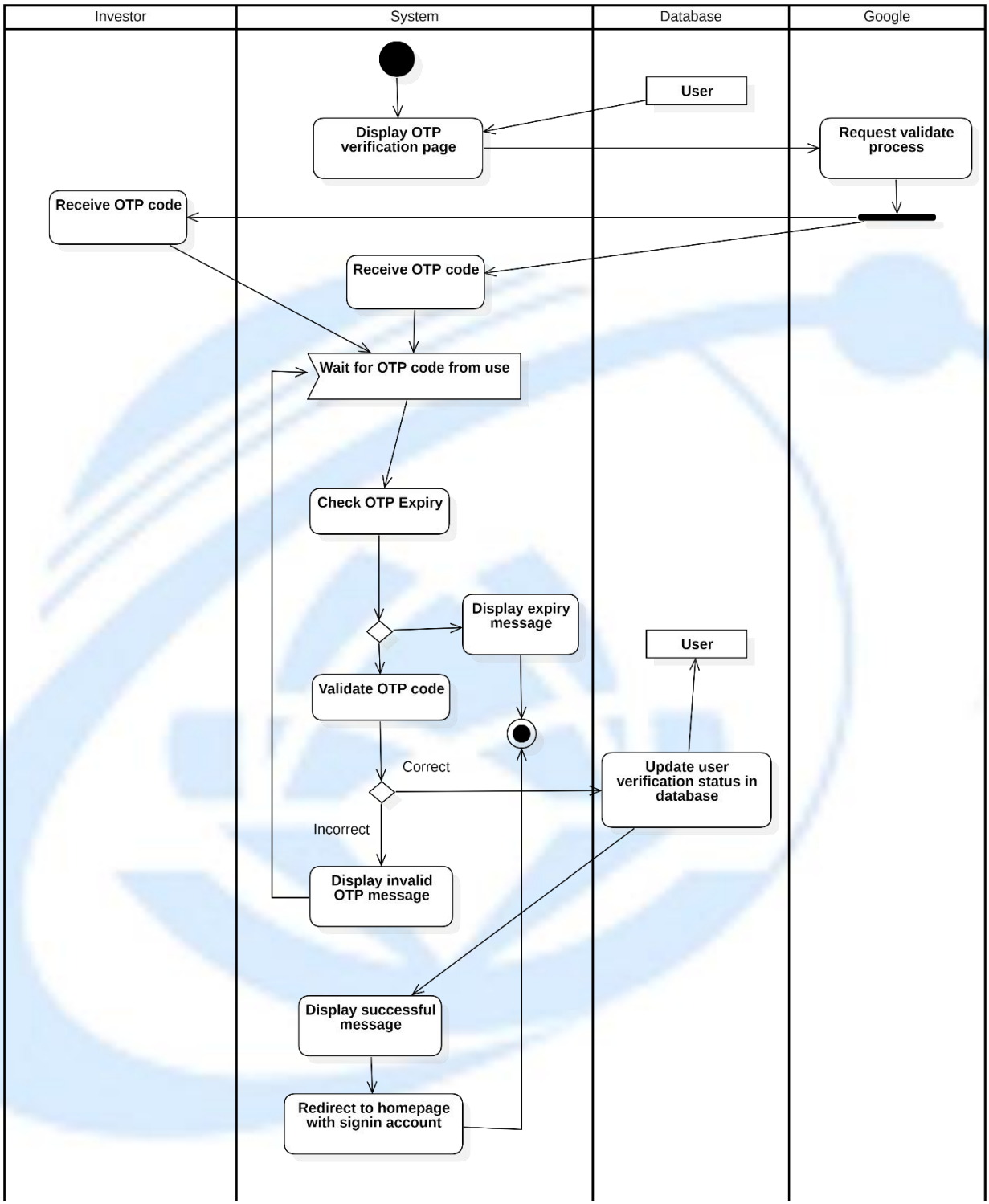
Activity Diagram 2: User Registion

3.2.3. Verify registration

Table 3.7: Verify registration usecase description

Attribute	Description
Use-case Name	OTP Verification for Registration
Description	The system allows an investor to verify their account registration using an OTP code. The system validates the OTP, checks for expiry, and updates the user's verification status in the database upon successful verification.
Trigger	The user enter their information to create account and successfully click button “Create Account”
Pre-condition	The user has entered the registration information and received the OTP code. The OTP code is valid and has not expired.
Post-condition	The user is successfully verified and the verification status is updated in the database. The user is redirected to the homepage with the signed-in account.
Basic flow	1. The system displays the OTP verification page. 2. External send OTP to both user and system.

	3. The user and system receives the OTP code. 4. The system waits for OTP code from user 5. The user enters the OTP code. 6. The system checks if the OTP has expired. 7. The system checks if the OTP is valid. 8. The system updates the user's verification status in the database. 9. The system displays a successful message. 10. The system redirects the user to the homepage with their signed-in account.
Alternative flow	6a. If the OTP has expired, the system displays an expiry message and. End this usecase 7a. If the OTP code is invalid, the system displays an error message and prompts the user to enter the correct code. The use case continues with step 5
Exception flow	



Activity Diagram 3: Verify Registration

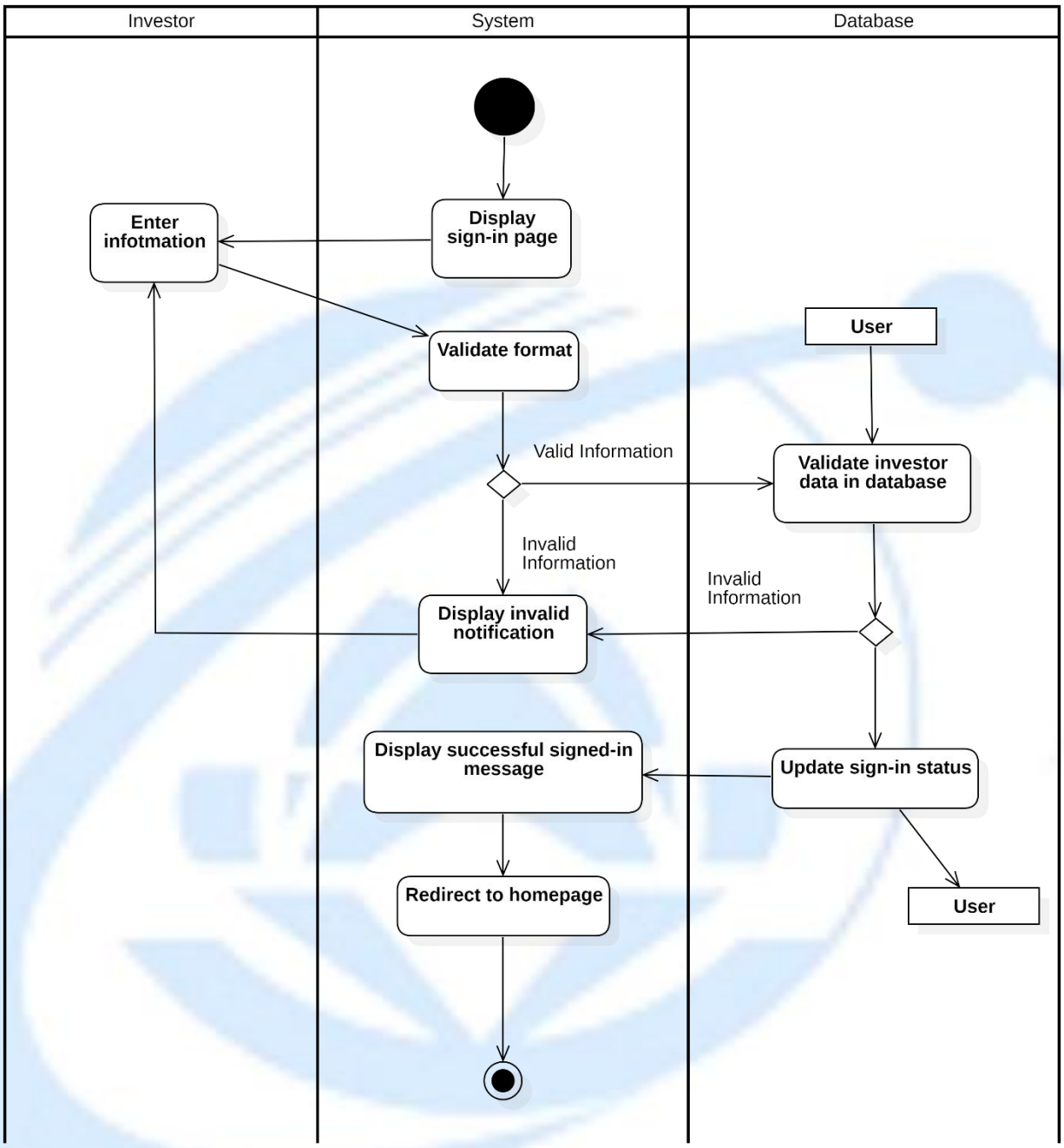
3.2.4. Log in

3.2.4.1. Login with Email and Password

Table 3.8: Login with Email and Password usecase description

Attribute	Description
Use-case Name	User Login
Description	The system allows users to sign in by entering their email and password. The system validates the format of the provided information, checks if the user exists in the database, and updates the user's sign-in status upon successful login.
Trigger	The user navigates to the sign-in page and enters their email and password.
Pre-condition	The user has an existing account in the system. The user has internet access.
Post-condition	The user is successfully signed in. The system updates the sign-in status of the user. The user is redirected to the homepage.
Basic flow	1. The system displays the sign-in page. 2. The user enters their email and password. 3. The system validates the format of the entered information (e.g., checks if the email is valid, password meets requirements). 4. The system checks if the user exists in the database by validating

	the entered information. 5. If the format is incorrect, the system displays an invalid notification. 6. If the user exists and the information is valid, the system updates the user's sign-in status. 7. The system displays a successful sign-in message. 8. The user is redirected to the homepage.
Alternative flow	3a. If the email format is invalid, the system displays an error message prompting the user to correct it. The use case continues with step 2 4a. If the username or email does not exist, the system notifies the user. The use case continues with step 2
Exception flow	



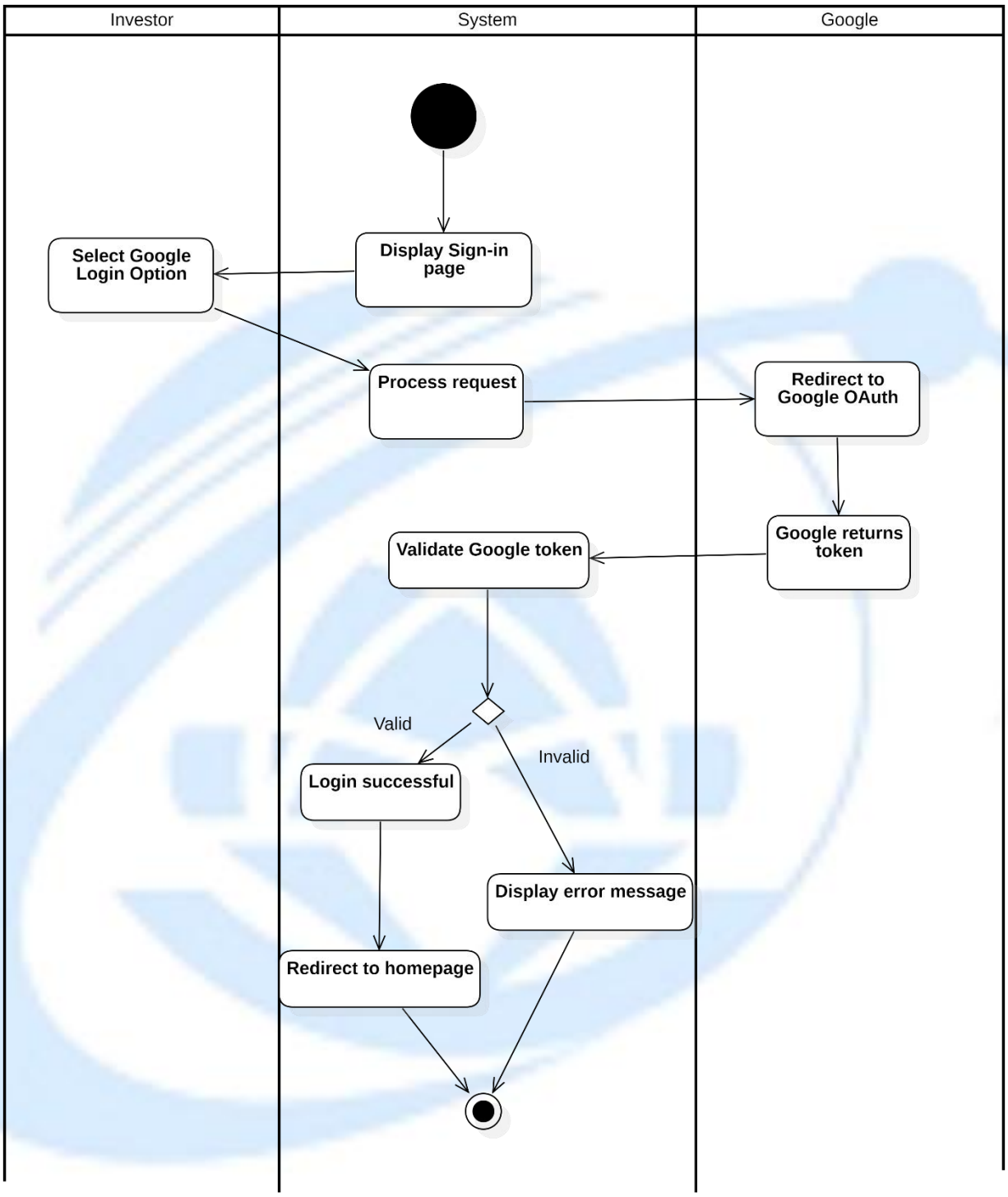
Activity Diagram 4: Login with Email and Password

3.2.4.2. Login with Google

Table 3.9: Login with Google usecase description

Attribute	Description
Use-case Name	Login with Google
Description	The system allows users to log in using their Google account. The system redirects the user to Google's OAuth service, validates the returned token, and signs in the user upon successful validation.
Trigger	The user selects the Google login option on the sign-in page.
Pre-condition	The user has a valid Google account and is connected to the internet. The system has the Google OAuth integration configured.
Post-condition	The user is successfully logged in via Google. The system redirects the user to the homepage.
Basic flow	1. The system displays the sign-in page with a Google login option. 2. The user selects the Google login option. 3. The system processes the request and redirects the user to Google OAuth. 4. Google returns an authentication token to the system. 5. The system validates the Google token. 6. The system notify successful login. 6. The system redirects the user to the homepage.
Alternative	5a. If the token validation fails, the system displays an error message

flow	and prompts the user to try again. End this usecase
Exception flow	



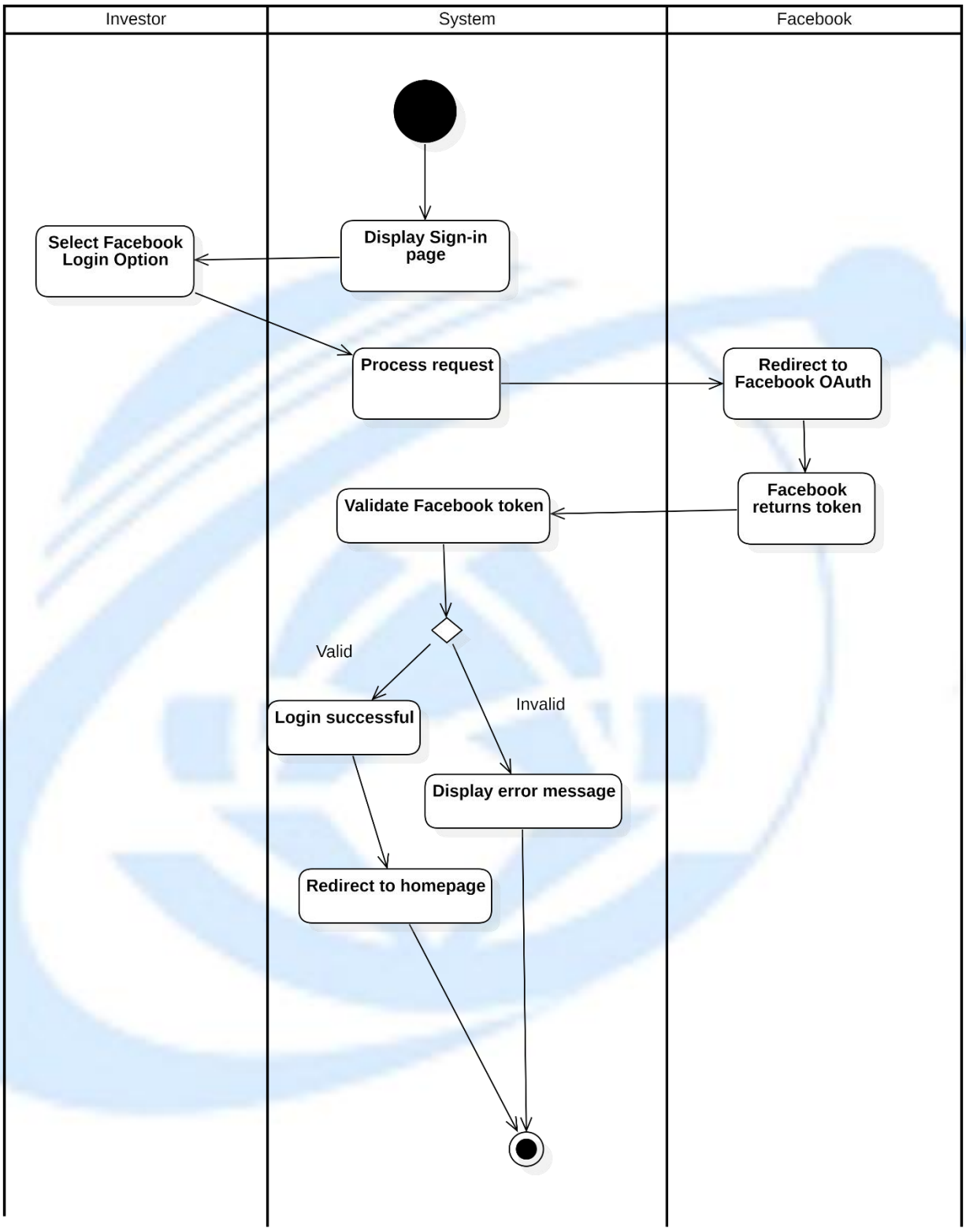
Activity Diagram 5: Login with Google

3.2.4.3. Login with Facebook

Table 3.10: Login with Facebook usecase

Attribute	Description
Use-case Name	Login with Facebook
Description	The system allows users to log in using their Facebook account. The system redirects the user to Facebook's OAuth service, validates the returned token, and signs in the user upon successful validation.
Trigger	The user selects the Facebook login option on the sign-in page.
Pre-condition	The user has a valid Facebook account and is connected to the internet. The system has the Facebook OAuth integration configured.
Post-condition	The user is successfully logged in via Facebook. The system redirects the user to the homepage.
Basic flow	1. The system displays the sign-in page with a Facebook login option. 2. The user selects the Facebook login option. 3. The system processes the request and redirects the user to Facebook OAuth. 4. Facebook returns an authentication token to the system. 5. The system validates the Facebook token. 6. The system redirects the user to the homepage.
Alternative	5a. If the token validation fails, the system displays an error

flow	message and prompts the user to try again. End this usecase
Exception flow	



Activity Diagram 6: Login with Facebook

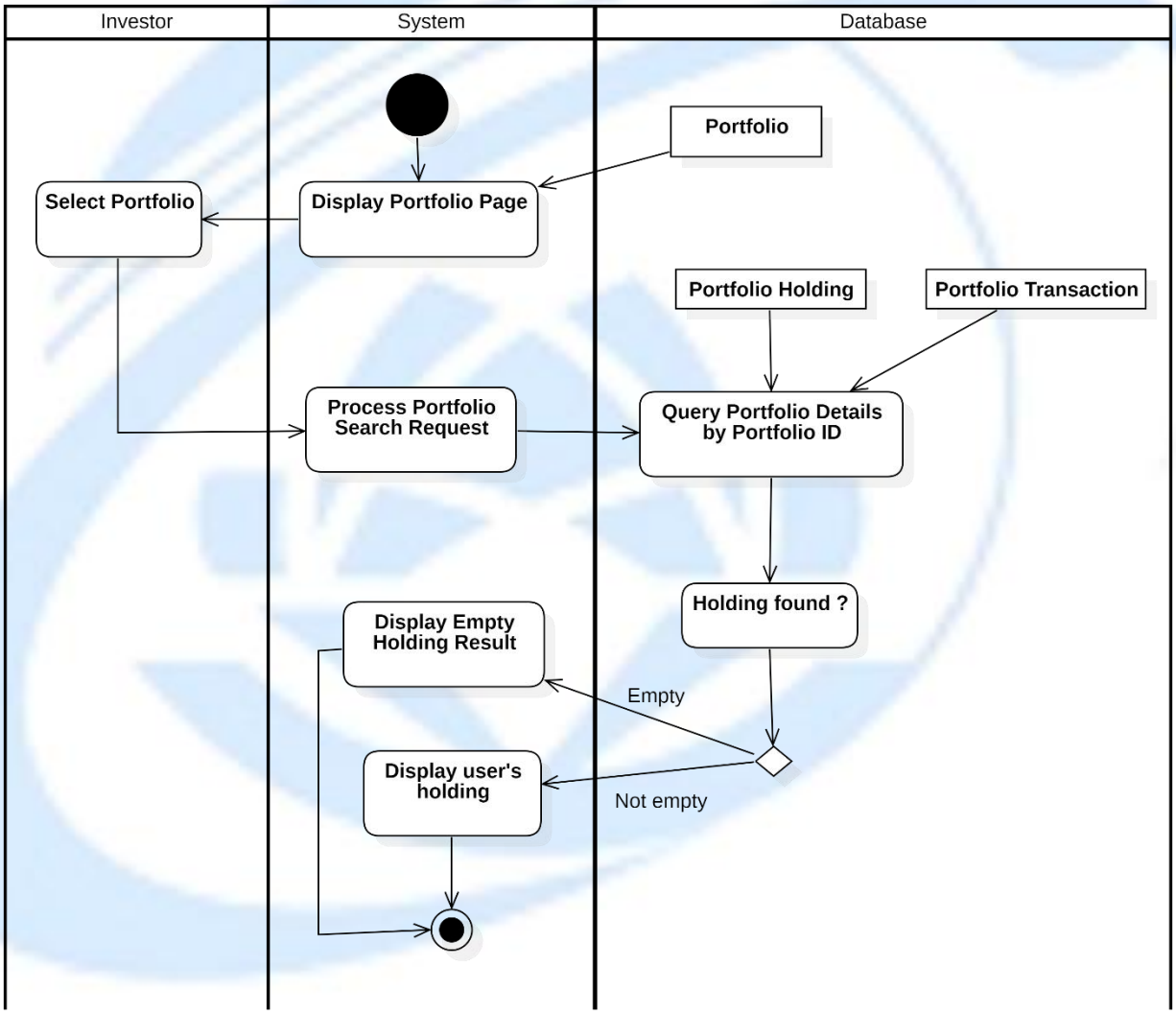
3.2.5. Manage Portfolio

3.2.5.1. View Portfolio

Table 3.11: View Portfolio usecase description

Attribute	Description
Use-case Name	View Portfolio Holdings
Description	The system allows an investor to view the holdings of a selected portfolio. After the investor selects a portfolio, the system retrieves portfolio details, including holdings and related transactions, and displays the results. If no holdings exist, the system shows an empty result message.
Trigger	The investor selects a portfolio.
Pre-condition	The investor is authenticated. The investor owns at least one portfolio.
Post-condition	The portfolio page displays the investor's holdings. If no holdings are found, an empty holding message is shown.
Basic flow	1. The system displays the portfolio page. 2. The investor selects a portfolio. 3. The system processes the portfolio search request. 4. The system queries portfolio details by portfolio ID. 5. The system retrieves portfolio holdings and related transactions. 6. The system checks whether holdings exist.

	7. The system displays the user's holdings on the portfolio page.
Alternative flow	6a. If no holdings are found, the system displays an empty holding result message. End this usecase
Exception flow	



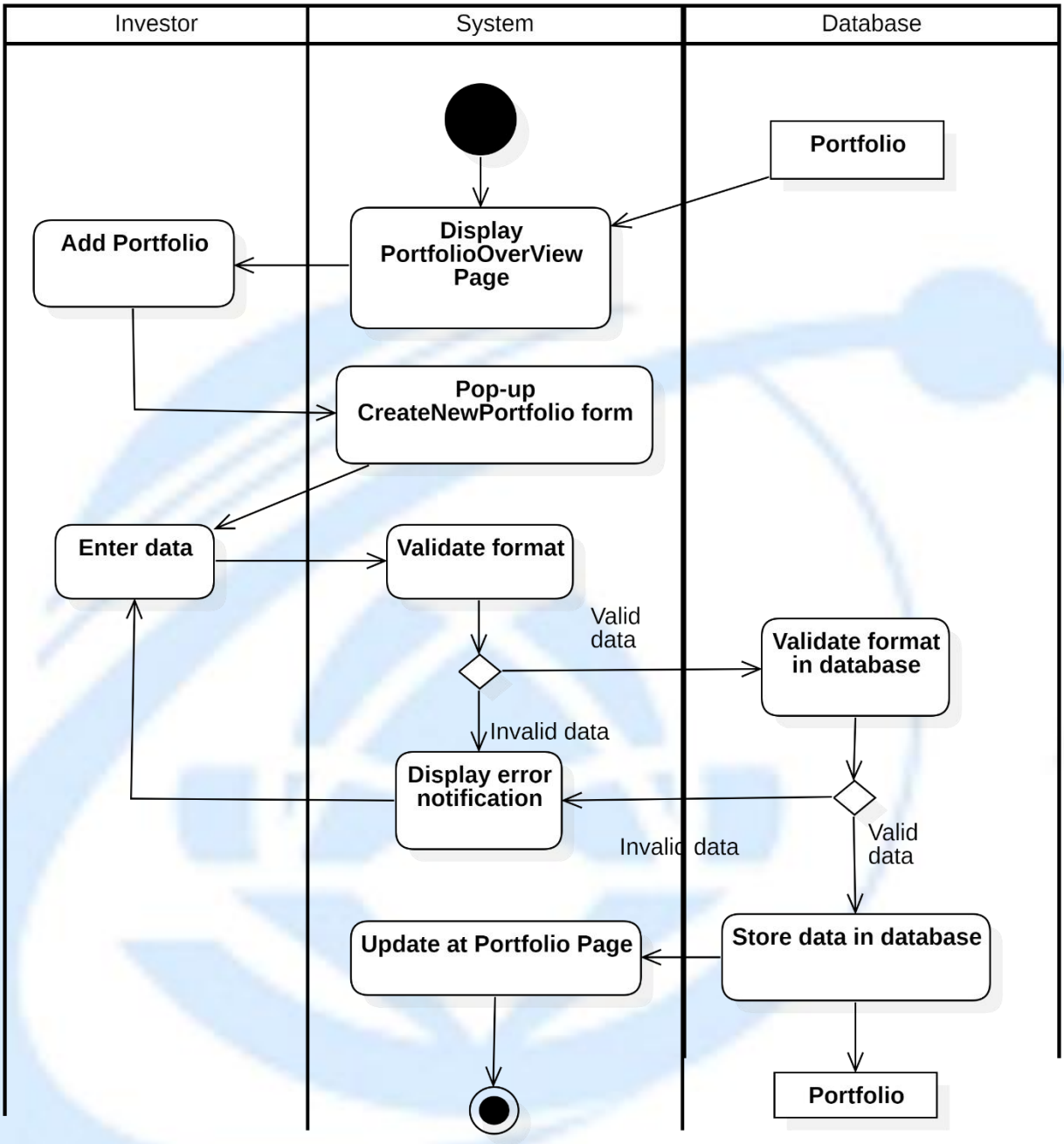
Activity Diagram 7: View Portfolio

3.2.5.2. Add New Portfolio

Table 3.12: Add New Portfolio usecase description

Attribute	Description
Use-case Name	Add Portfolio
Description	The system allows investors to add a new portfolio by entering the required information. The system validates the entered data format and stores it in the database upon successful validation.
Trigger	The user selects button "New Portfolio" on PortfolioOverview page.
Pre-condition	The user is logged into the system.
Post-condition	The portfolio is successfully added to the database. The portfolio page is updated with the newly added portfolio information.
Basic flow	1. The system displays the portfolio page. 2. The user selects the option to add a portfolio. 3. The system displays the CreateNewPortfolio form. 4. The user enters the portfolio data (e.g., portfolio name, description). 5. The system validates the format of the entered data. 6. The system validates the format in the database. 7. The system stores the portfolio information in the database. 8. The system updates the portfolio page with the new portfolio

	information.
Alternative flow	5a. If the entered data format is invalid, the system displays an error notification. The use case continues with step 4. 6a. If the entered data format is invalid, the system displays an error notification. The use case continues with step 4.
Exception flow	



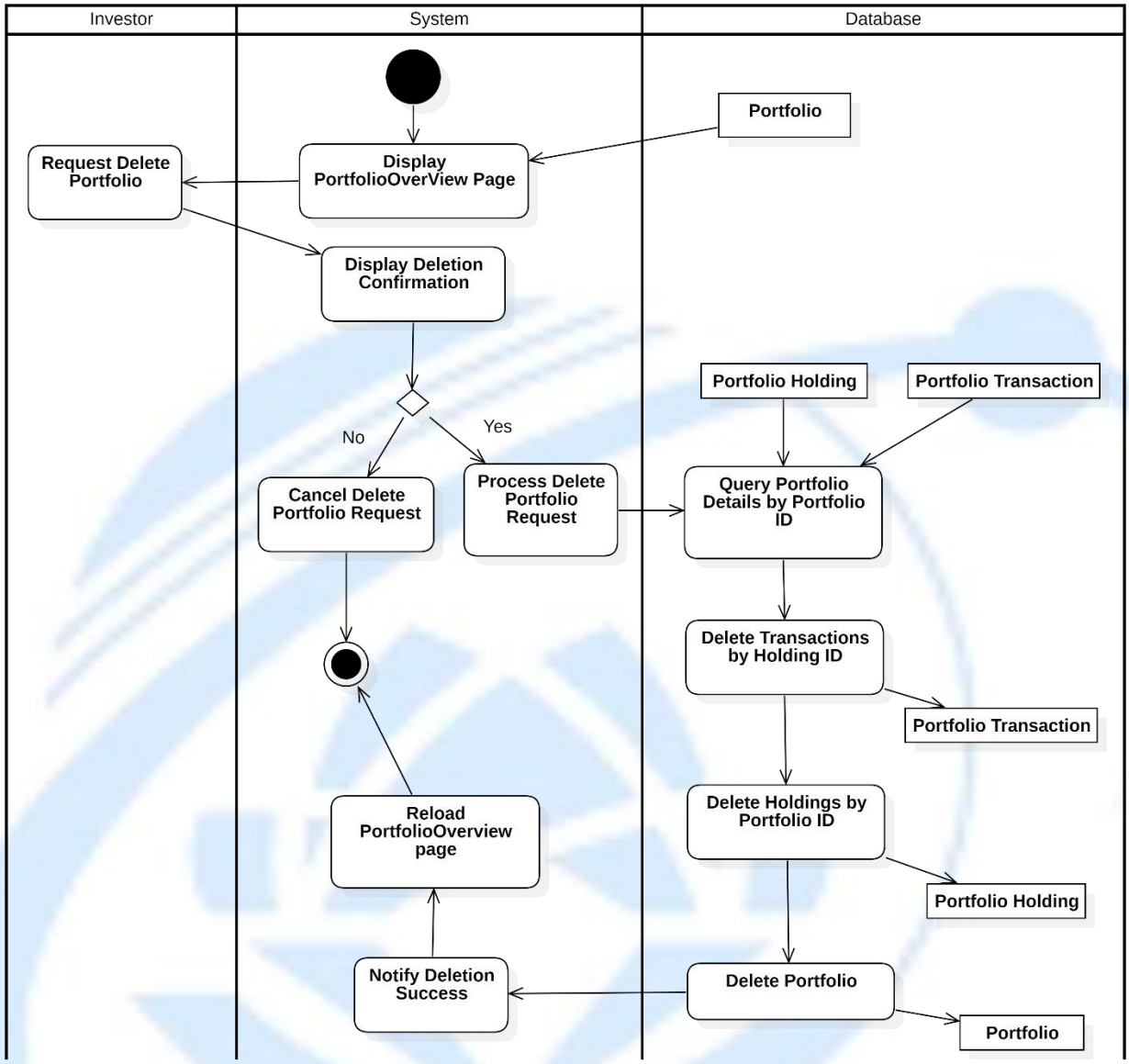
Activity Diagram 8: Add New Portfolio

3.2.5.3. Delete Portfolio

Table 3.13: Delete Portfolio usecase description

Attribute	Description
Use-case Name	Delete Portfolio
Description	This use case allows an investor to delete an existing portfolio. The system asks for confirmation before deletion. If confirmed, all related holdings and transactions of the portfolio are deleted before removing the portfolio itself.
Trigger	The investor requests to delete a portfolio from the portfolio overview page.
Pre-condition	The investor is authenticated. The portfolio exists and belongs to the investor.
Post-condition	The selected portfolio is permanently deleted. All related holdings and transactions are removed from the database. The portfolio overview page is refreshed and shows updated data.
Basic Flow	1. The system displays the Portfolio Overview page. 2. The investor requests to delete a portfolio 3. The system displays a deletion confirmation dialog. 4. The investor confirms the deletion request. 5. The system processes the delete portfolio request. 6. The system queries portfolio details by portfolio ID.

	7. The system deletes all transactions related to the portfolio holdings. 8. The system deletes all holdings related to the portfolio. 9. The system deletes the portfolio. 10. The system reloads the Portfolio Overview page. 11. The system notifies the investor of successful deletion.
Alternative Flow	4a. The investor cancels the delete request. End this usecase
Exception Flow	



Activity Diagram 9: Delete Portfolio

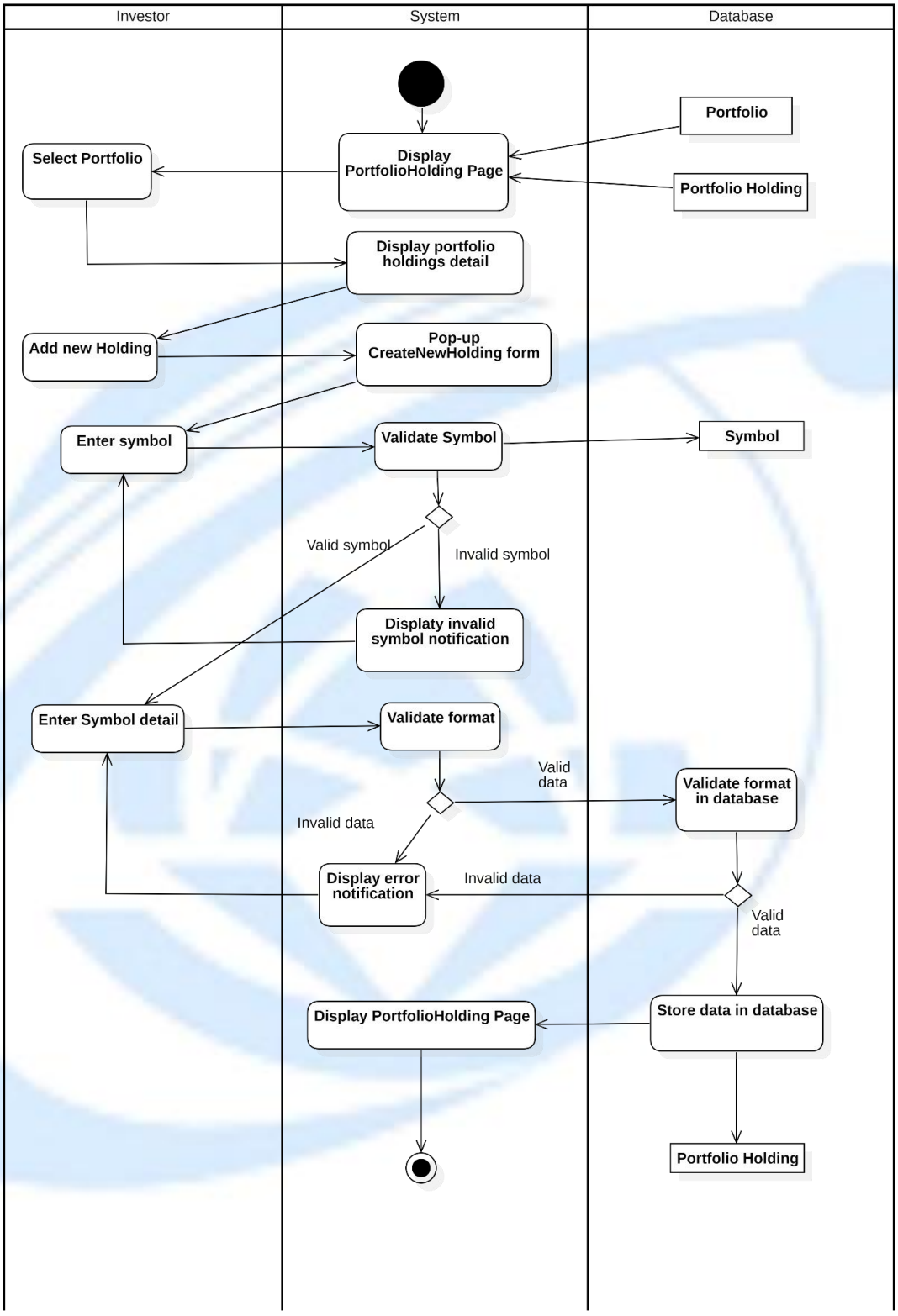
3.2.5.4. Update Portfolio

3.2.5.4.1 Add New Holding

Table 3.14: Add New Holding usecase description

Attribute	Description
Use-case Name	Add New Holding
Description	This use case allows an investor to add a new holding (stock symbol and related details) into an existing portfolio. The system validates the symbol and input format before storing the holding data.
Primary Actor	Investor
Trigger	The investor selects a portfolio and clicks Add New Holding.
Pre-condition	The investor is authenticated. The selected portfolio exists and belongs to the investor.
Post-condition	A new holding is successfully added to the selected portfolio. The portfolio holding page is refreshed with updated data.
Basic Flow	1. The system displays the Portfolio Holding page. 2. The investor selects a portfolio. 3. The system displays portfolio holding details. 4. The investor clicks Add New Holding. 5. The system displays a pop-up Create New Holding form. 6. The investor enters a stock symbol.

	7. The system validates the symbol. 8. The investor enters symbol details (quantity, price, date, etc.). 9. The system validates the input format. 10. The system stores the holding data in the database. 11. The system reloads and displays the Portfolio Holding page with the new holding.
Alternative Flow	7a. The symbol is invalid. The system displays an invalid symbol notification. The use case continues with step 6. 9a. The input format is invalid. The system displays an error notification. The use case continues with step 8
Exception Flow	



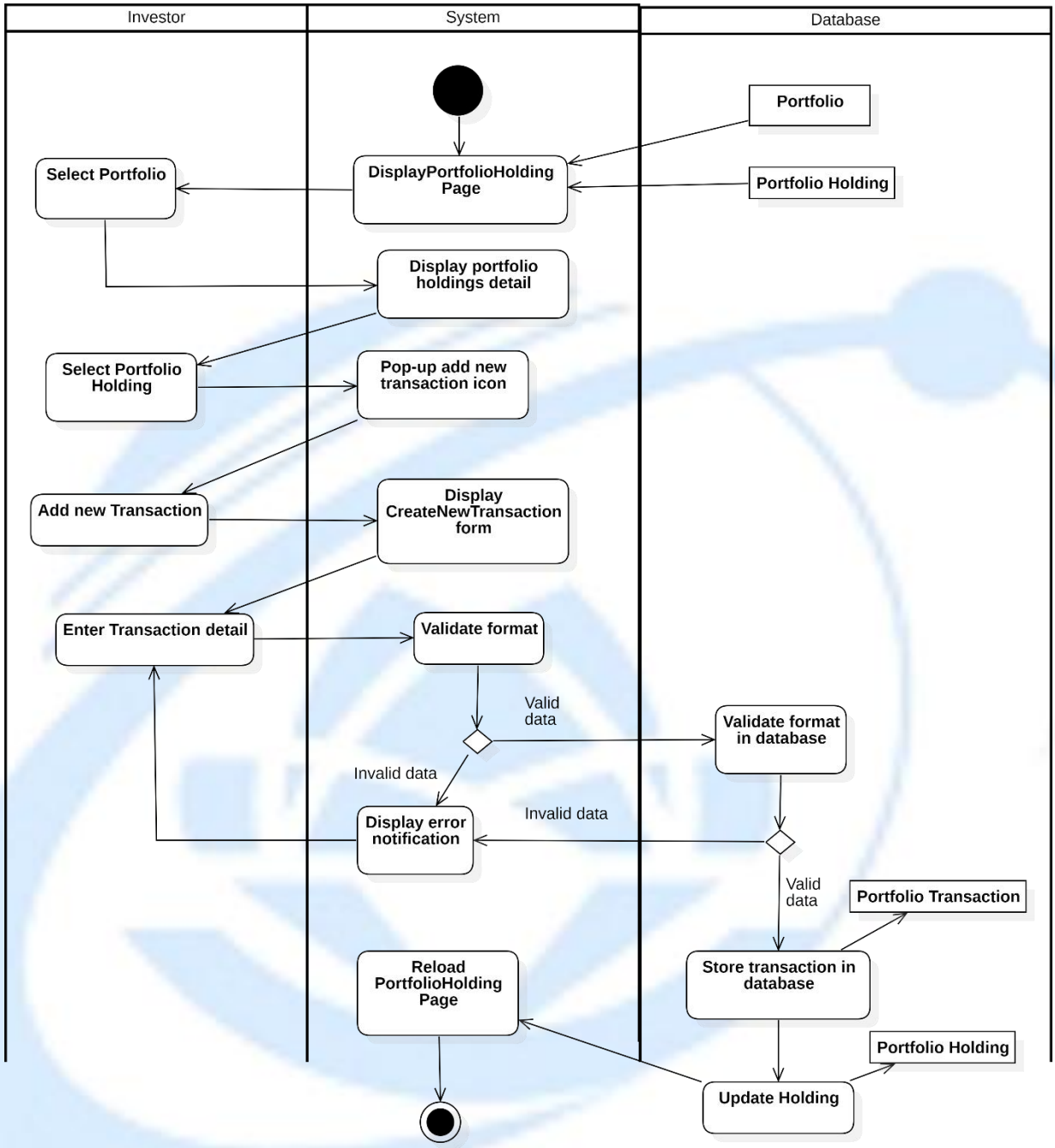
Activity Diagram 10: Add New Holding

3.2.5.4.2 Update Holding / Add new Transaction

Table 3.15: Update Holding usecase description

Attribute	Description
Use-case Name	Add New Transaction
Description	This use case allows an investor to add a new transaction to an existing holding within a selected portfolio. The system validates the transaction data, stores it in the database, updates the holding information, and refreshes the portfolio holding page.
Primary Actor	Investor
Trigger	The investor clicks **Add New Transaction** from a selected portfolio holding.
Pre-condition	The investor is authenticated. A portfolio exists and is selected. A portfolio holding exists and is selected.
Post-condition	A new transaction is successfully stored in the database. The holding information is updated accordingly. The Portfolio Holding page is refreshed with updated data.
Basic Flow	1. The investor selects a portfolio.

	2. The system displays the Portfolio Holding page. 3. The system displays portfolio holdings details. 4. The investor selects a portfolio holding record. 5. The investor clicks the **Add New Transaction** icon. 6. The system displays the Create New Transaction form. 7. The investor enters transaction details. 8. The system validates the transaction data format. 9. The system validates the transaction format in the database. 10. The system stores the transaction in the database. 11. The system updates the corresponding holding data. 12. The system reloads the Portfolio Holding page.
Alternative Flow	8a. The transaction data format is invalid. The system displays an error notification. The use case continues with step 7 9a. The transaction data format is invalid. The system displays an error notification. The use case continues with step 7
Exception Flow	



Activity Diagram 11: Update Holding

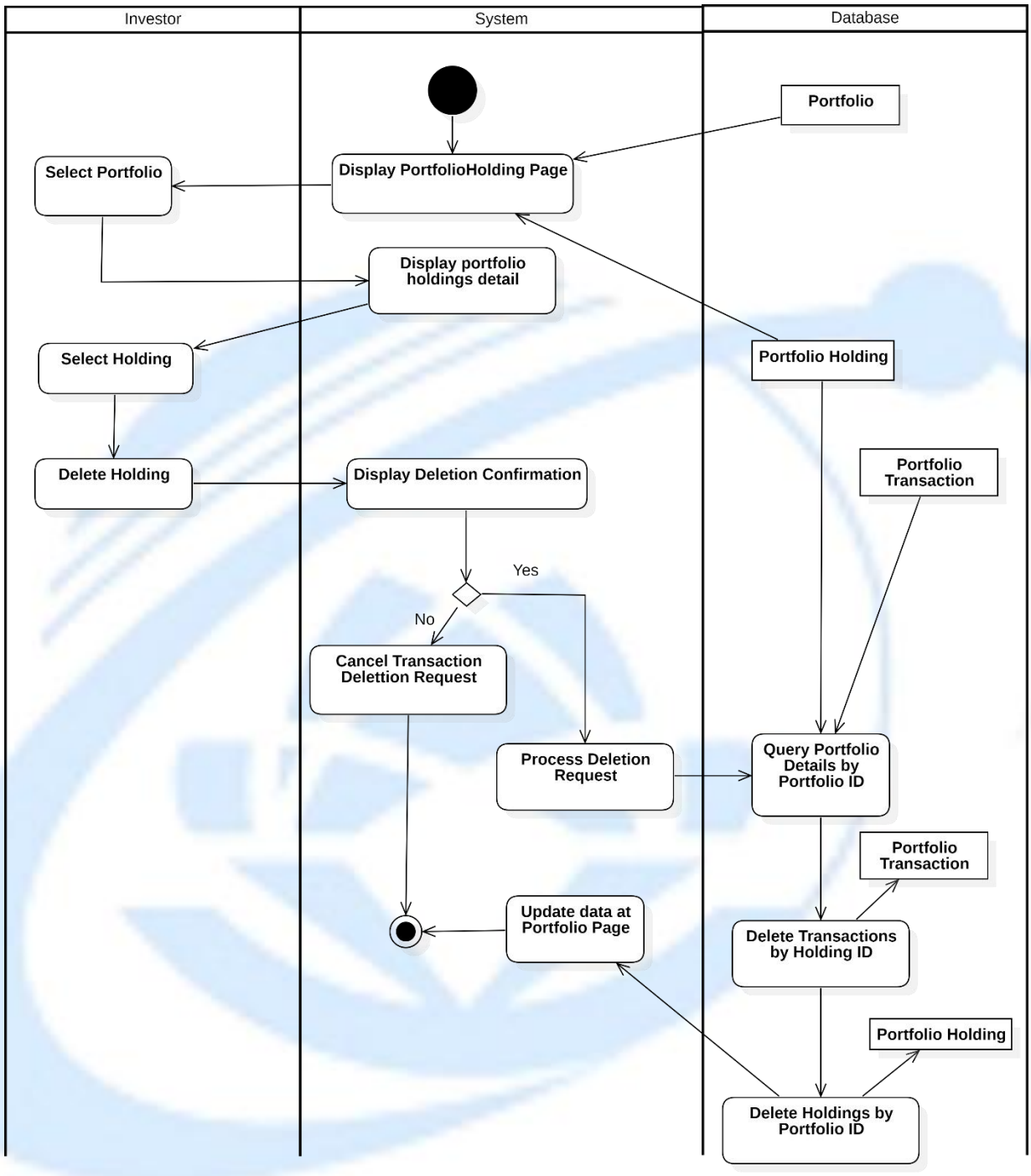
3.2.5.4.3 Delete Holding

Table 3.16: Delete Holding usecase description

Attribute	Description

Use-case Name	Delete Holding
Description	This use case allows an investor to delete a selected holding from a portfolio. The system requests confirmation before deleting the holding and all related transactions, then updates the portfolio data accordingly.
Primary Actor	Investor
Trigger	The investor selects a holding and chooses button Delete Holding.
Pre-condition	The investor is authenticated. The selected portfolio exists and belongs to the investor. The selected holding exists within the portfolio.
Post-condition	The selected holding and all related transactions are deleted. The portfolio holding page is updated with the latest data.
Basic Flow	1. The system displays the Portfolio Holding page. 2. The investor selects a portfolio. 3. The system displays portfolio holdings details. 4. The investor selects a holding. 5. The investor chooses Delete Holding. 6. The system displays a deletion confirmation dialog. 7. The investor confirms the deletion request. 8. The system processes the deletion request. 9. The system queries portfolio details by portfolio ID.

	10. The system deletes all transactions related to the holding by holding ID. 11. The system deletes the holding by portfolio ID. 12. The system updates data on the Portfolio page.
Alternative Flow	7a. The investor cancels the deletion request. The system returns to the Portfolio Holding page without any changes. End this usecase
Exception Flow	



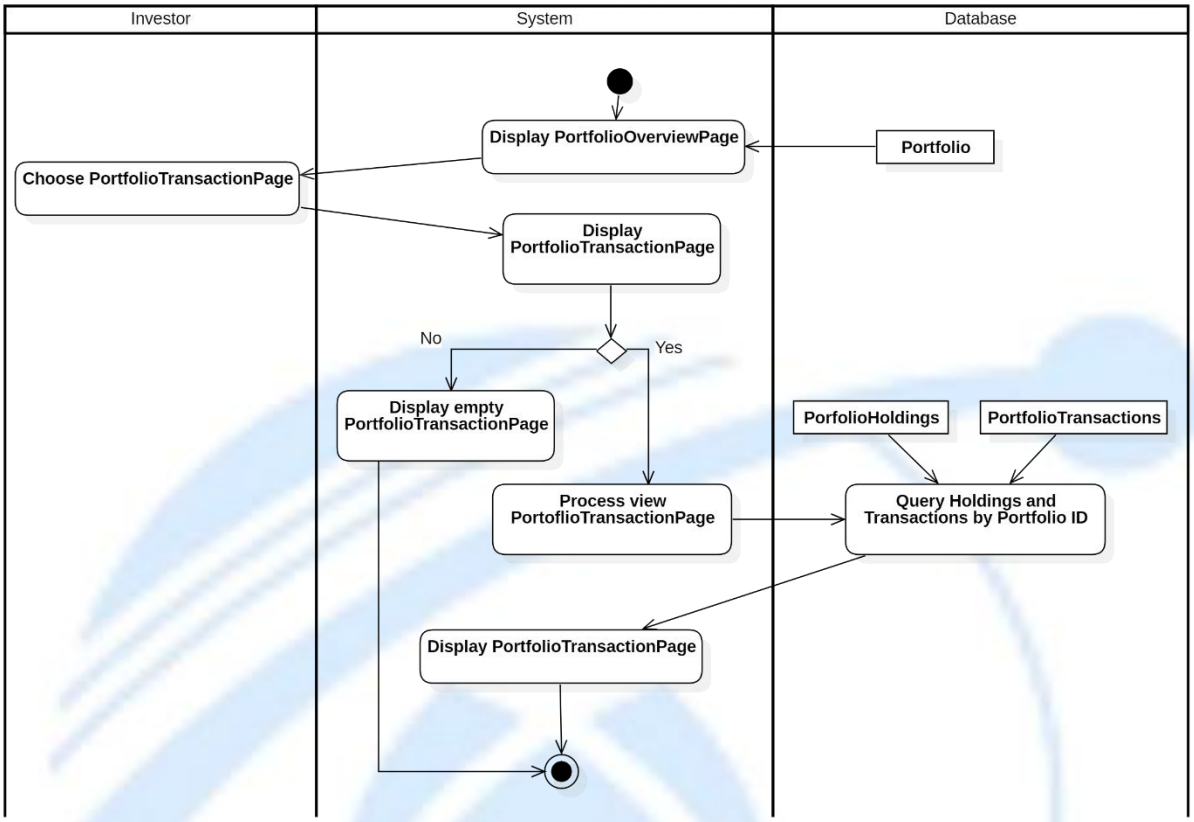
Activity Diagram 12: Delete Holding

3.2.5.5. View Transaction History

Table 3.17. View Transaction History usecase description

Attribute	Description
Use-case Name	View Transaction History
Description	This use case allows an investor to view the transaction history of a selected portfolio. The system retrieves and displays all transactions related to the selected portfolio and holding.
Primary Actor	Investor
Trigger	The investor selects a portfolio and chooses to view its transaction history.
Pre-condition	- The investor is authenticated. - The selected portfolio exists and belongs to the investor. - The portfolio contains at least one holding or transaction.
Post-condition	- The transaction history is displayed successfully. - No data is modified in the system.
Basic Flow	1. The system displays the Portfolio Overview page. 2. The investor selects a portfolio. 3. The system displays the Portfolio Holdings page.

	4. The investor selects a holding. 5. The investor selects View Transaction History. 6. The system retrieves transaction data related to the selected holding. 7. The system displays the transaction history list.
Alternative Flow	6a. No transaction exists 6a.1 The system returns an empty transaction list. 6a.2 The system displays a message: *"No transaction history available."*
Exception Flow	- If the system fails to retrieve transaction data, an error message is displayed.- If the portfolio or holding does not exist, the system redirects back to the Portfolio page.



Activity Diagram 13: View Transactions History

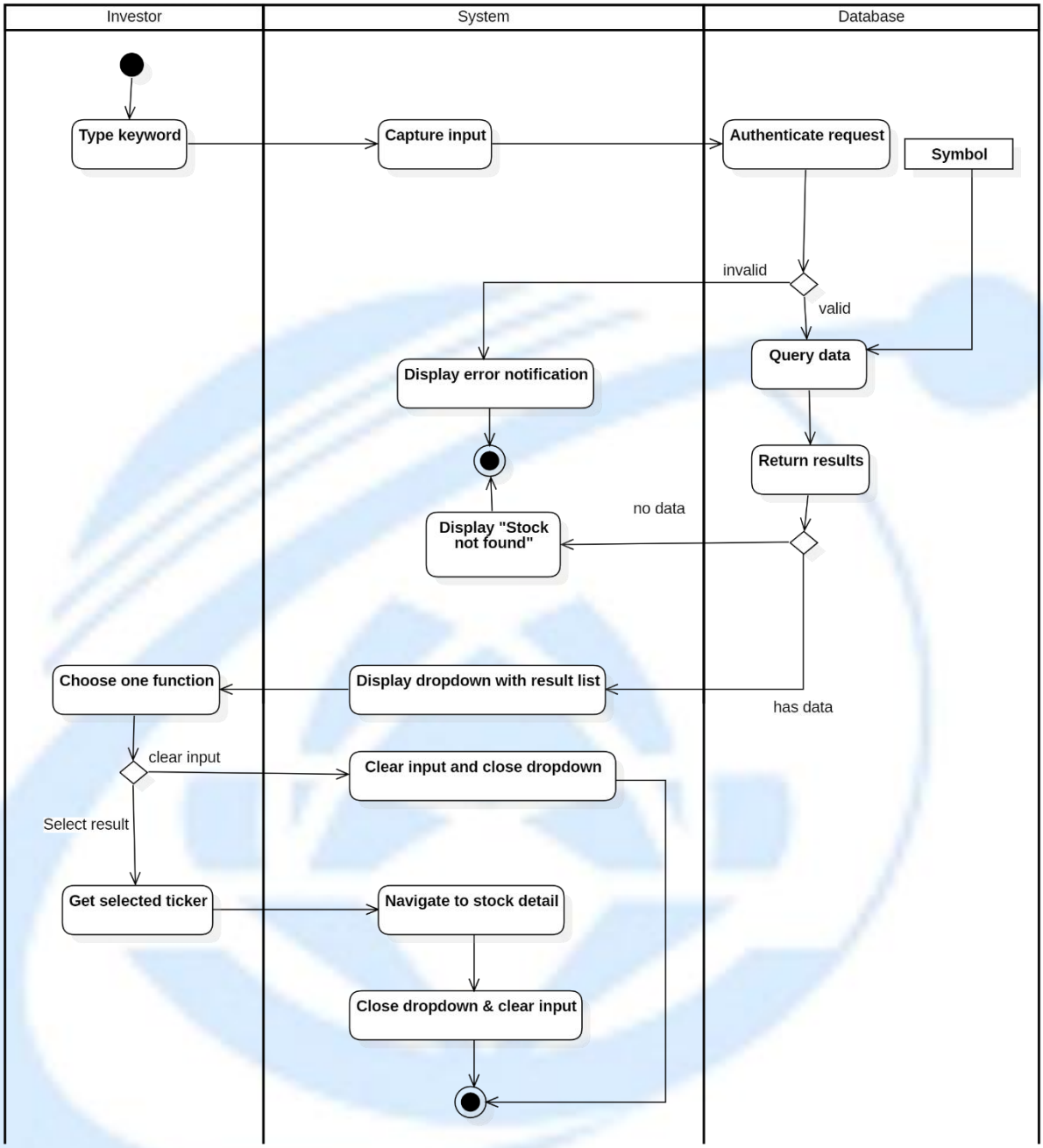
3.2.6. Research Stock

3.2.6.1. Search Stock

Table 3.18: Search Stock usecase description

Attribute	Description
Use-case Name	Search Stock
Description	The system allows an investor to search for a stock by typing a keyword. The system authenticates the search request, queries the stock data, and displays the results, including a dropdown list with the stock options.

Trigger	The user types a keyword to search for a stock.
Pre-condition	The user is logged into the system. The user has internet access to search for stock data.
Post-condition	The system displays the search results for the stock, including relevant options in a dropdown list. The user can select a stock to view more details.
Basic flow	1. The user types a keyword into the search field. 2. The system captures the input. 3. The system authenticates the request and checks if the symbol is valid. 4. If the symbol is invalid, the system displays an error notification. 5. If the symbol is valid, the system queries the database for stock data. 6. If no data is found, the system displays a "Stock not found" message. 7. If data is found, the system returns the results and displays them in a dropdown list. 8. The user selects a stock from the dropdown list. 9. The system navigates to the selected stock's detail page.
Alternative flow	4a. If the symbol is invalid, the system displays an error notification and asks the user to correct the input. 6a. If no stock data is found, the system displays "Stock not found".
Exception flow	



Activity Diagram 14: Search Stock

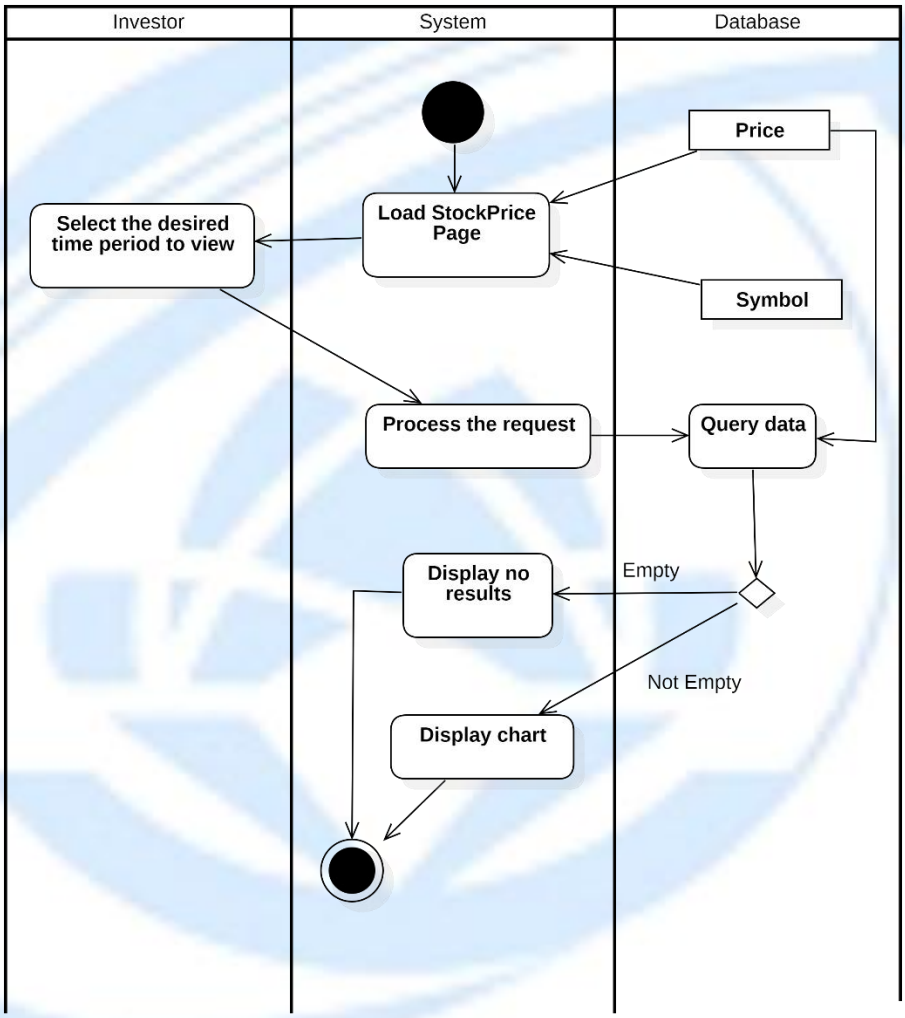
3.2.6.2. View Price History

3.2.6.2.1 Change Period

Table 3.19: Change Period usecase description

Attribute	Description
Use-case Name	View Price History by Period
Description	The system allows an investor to view the price history of a stock for a selected period. The system queries the price data from the database and displays a chart or error message based on the availability of the data.
Trigger	The user clicks to select the period they want to view.
Pre-condition	The user is logged into the system. The user has selected a stock and period to view.
Post-condition	The system displays a chart with the price history of the selected stock for the selected period. If no data exists, an error message is displayed.
Basic flow	1. The user clicks to select the period they want to view. 2. The system processes the request and retrieves the price data from the database. 3. The system queries the data for the selected stock symbol. 4. If data exists, the system displays a chart showing the price history for the selected period.

Alternative flow	4a. If no data is available for the selected period, the system displays an error message. End this usecase
Exception flow	

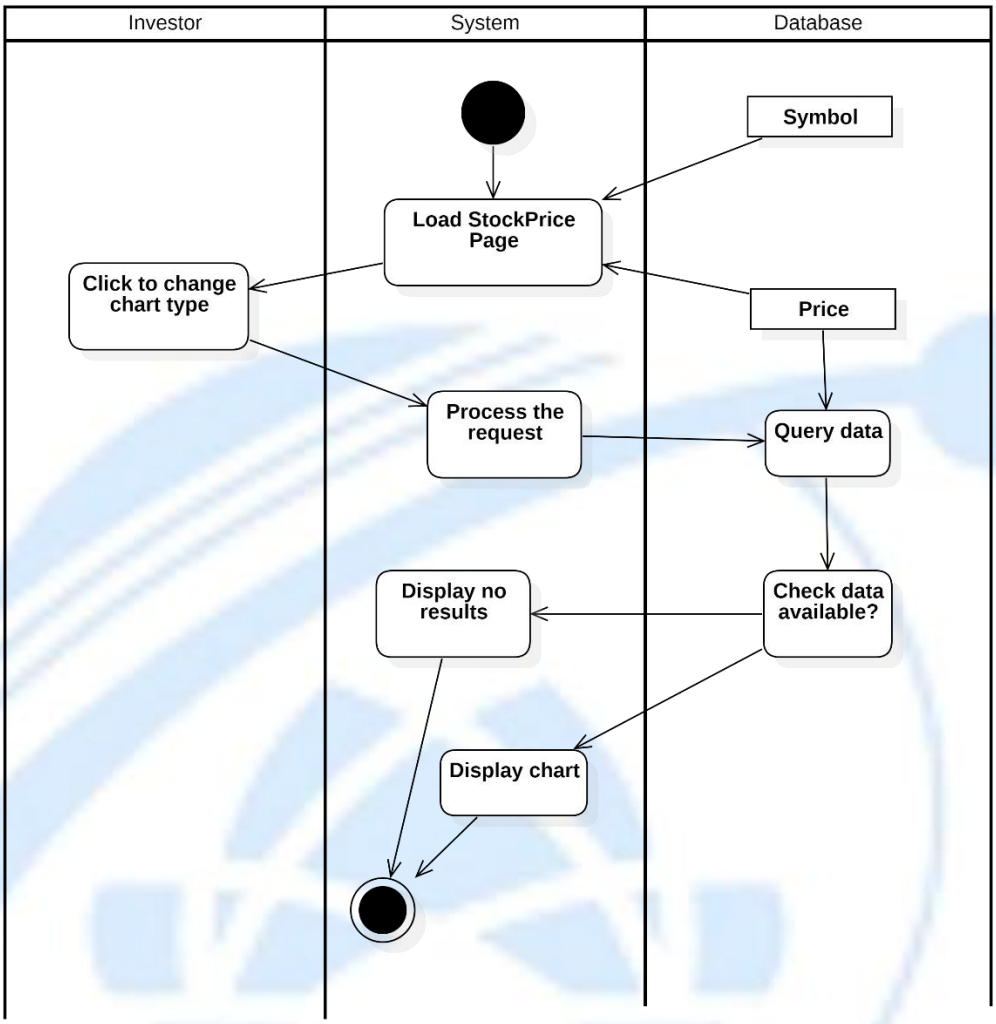


Activity Diagram 15: Change Period

3.2.6.2.2 Change Chart Type

Table 3.20: Change Chart Type usecase description

Attribute	Description
Use-case Name	Change Stock Price Chart Type
Description	The system allows an investor to change the type of chart displaying the stock price. The system processes the request, retrieves the data, and displays the updated chart.
Trigger	The user clicks to change the chart type.
Pre-condition	The user is logged into the system. The user is viewing the stock price page.
Post-condition	The system updates the chart type based on the user's selection. The system displays the updated chart with the stock price data in the new format.
Basic flow	1. The user clicks to change the chart type. 2. The system processes the request. 3. The system retrieves the data for the stock price. 4. The system checks if data is available. 5. If data is available, the system displays the chart in the new selected format.
Alternative flow	5a. If no data is available for the selected chart type, the system displays a "no results" message. End this usecase
Exception flow	



Activity Diagram : Change Chart Type

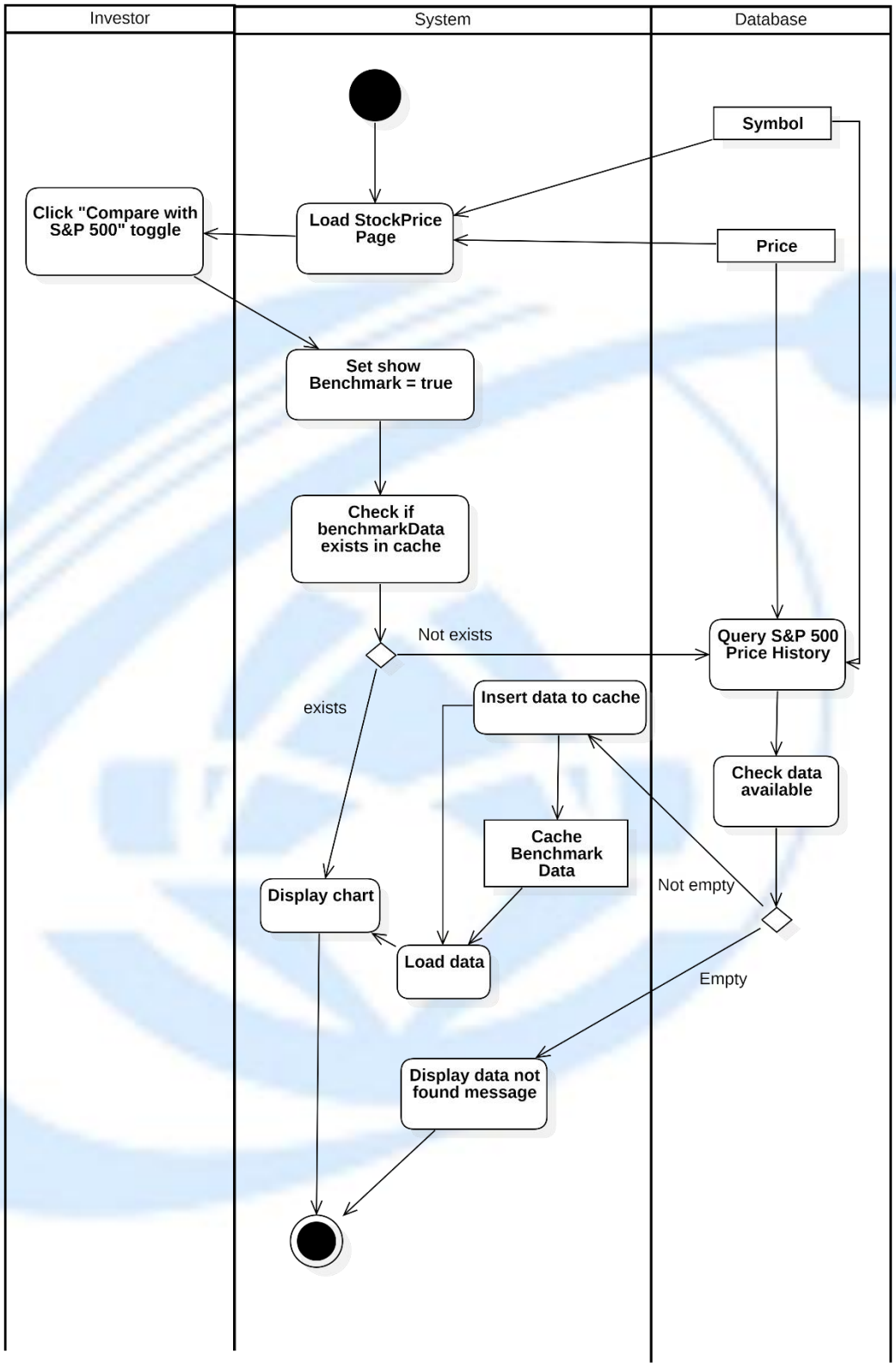
3.2.6.2.3 Select Comparation with S&P 500

Table 3.21: Select Comparation with S&P 500 usecase description

Attribute	Description
Use-case Name	Compare Stock Price with S&P 500
Description	The system allows an investor to compare the stock price with the S&P 500 index. The system first checks the cache for the benchmark data. If the data exists, it displays the chart. If not, it

	queries the S&P 500 price history, stores the data in the cache, and then displays the chart.
Trigger	The user clicks on the "Compare with S&P 500" toggle.
Pre-condition	The user is logged into the system. The user is viewing the stock price page.
Post-condition	The system displays the comparison chart between the stock price and S&P 500. The system stores the benchmark data in the cache for future use.
Basic flow	1. The system loads the stock price page. 2. The user clicks the "Compare with S&P 500" toggle. 3. The system sets the benchmark flag to true. 4. The system checks if the benchmark data (S&P 500 data) exists in the cache. 5. If the data doesn't exist in the cache, the system queries the S&P 500 price history. 6. The system checks if the data is available. 7. If the data is available, the system loads the data and stores it in the cache. 8. The system displays the comparison chart.
Alternative flow	5a. If the data is available in the cache, the system directly displays the comparison chart. End this usecase

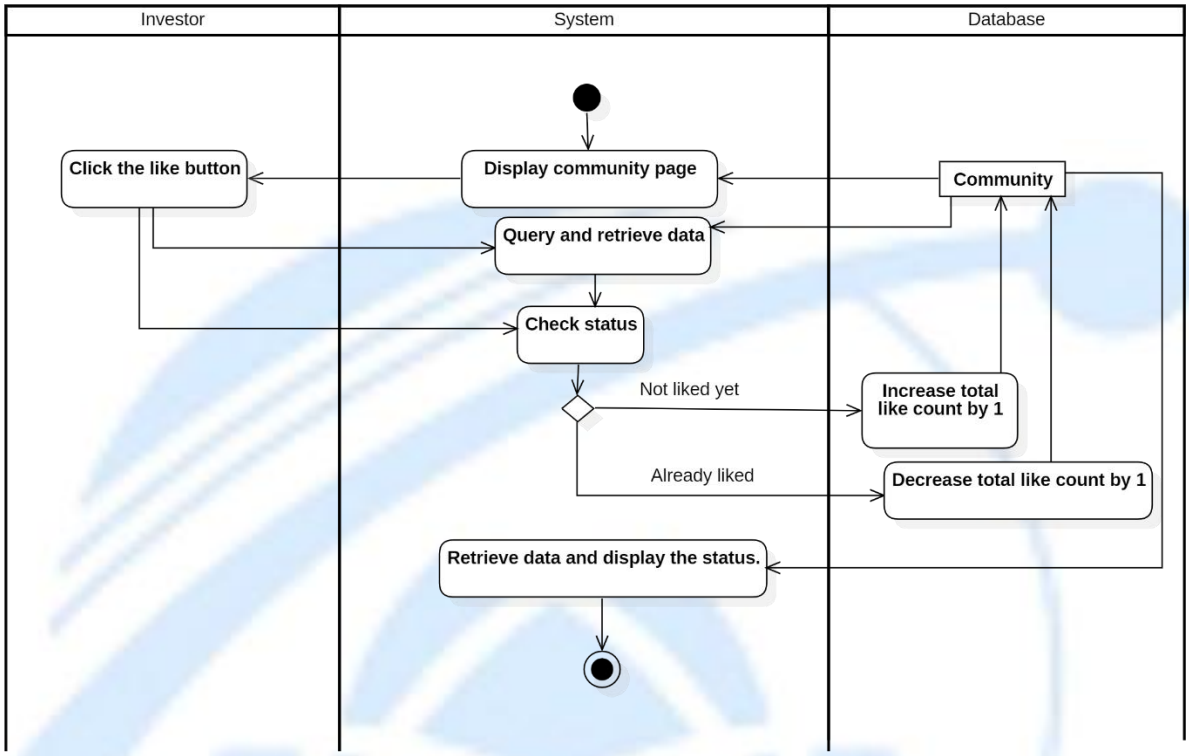
	7a. If no data is found, the system displays a "data not found" message. End this usecase
Exception flow	If the system encounters an error during the data retrieval or visualization process (e.g., database failure), it displays a failure message and prompts the user to try again later.



Activity Diagram : Select Comparation with S&P 500

3.2.6.3. View Community Discussion

3.2.6.3.1 Like Discussion



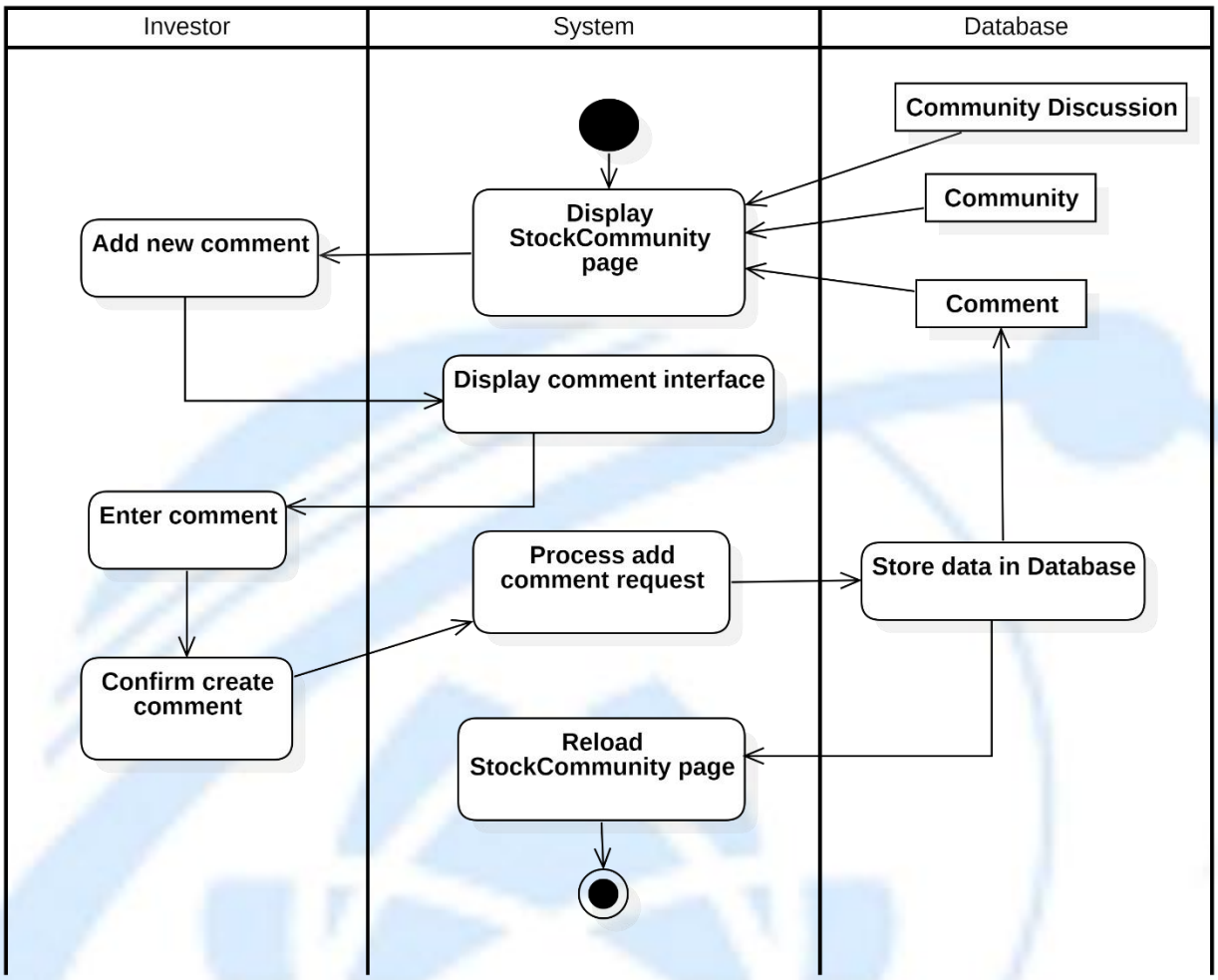
Activity Diagram : Like Discussion

3.2.6.3.2 Add New Comment

Table 3.22: Add New Comment usecase description

Attribute	Description
Use-case Name	Add New Comment
Description	This use case allows an investor to add a new comment to a stock community discussion
Trigger	The investor selects Add new comment on the Stock Community page.

Pre-condition	The investor is authenticated. The Stock Community page is accessible. The related community discussion exists.
Post-condition	The new comment is successfully stored in the database. The Stock Community page is reloaded and displays the newly added comment.
Basic Flow	1. The system displays the Stock Community page. 2. The investor selects Add new comment. 3. The system displays the comment interface. 4. The investor enters comment content. 5. The investor confirms comment creation. 6. The system processes the add comment request. 7. The system stores the comment data in the database. 8. The system reloads the Stock Community page and displays the new comment.
Alternative Flow	
Exception Flow	



Activity Diagram : Add New Comment

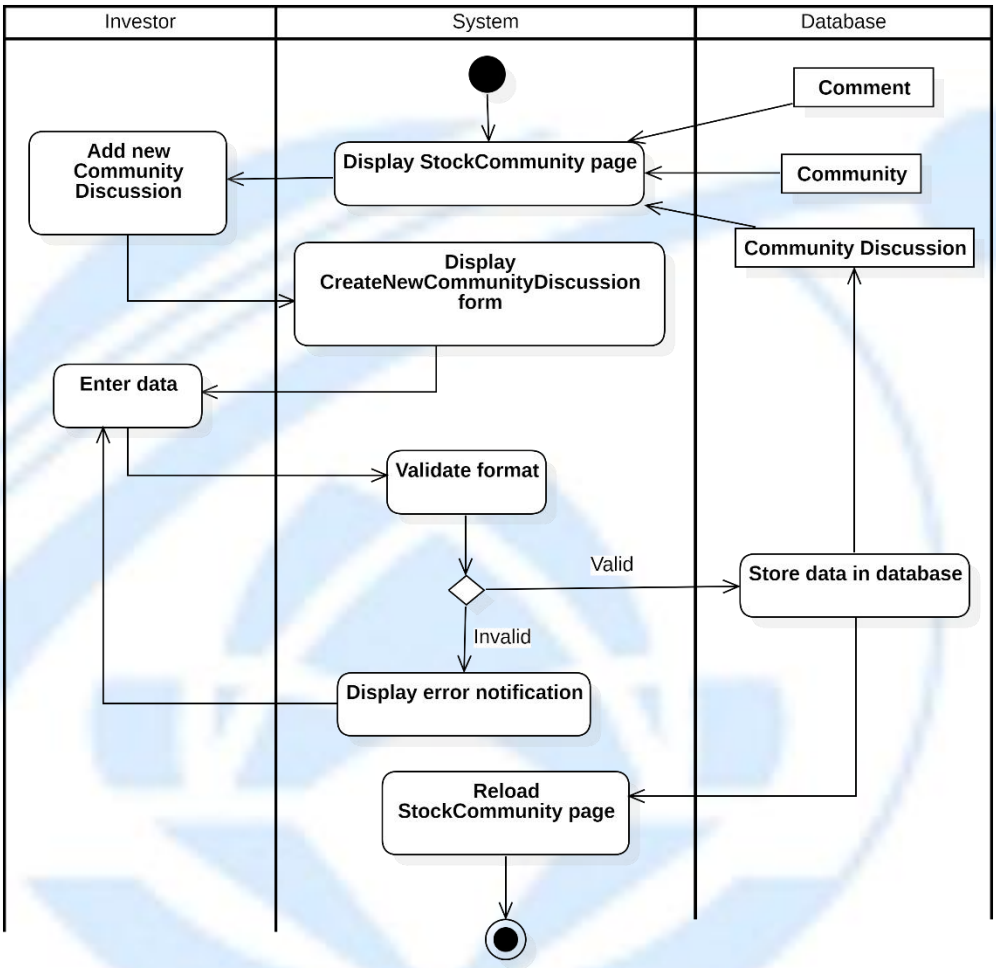
3.2.6.3.3 Create a New Discussion

Table 3.23: Create a New Discussion usecase description

Attribute	Description
Use-case Name	Add New Community Discussion
Description	This use case allows an investor to create a new discussion in the stock community.

Trigger	The investor selects **Add new Community Discussion** on the Stock Community page.
Pre-condition	The investor is authenticated. The Stock Community page is successfully displayed.
Post-condition	A new community discussion is stored in the database. The Stock Community page is reloaded and displays the newly created discussion.
Basic Flow	1. The system displays the Stock Community page. 2. The investor selects **Add new Community Discussion**. 3. The system displays the **CreateNewCommunityDiscussion** form. 4. The investor enters discussion data. 5. The system validates the input format. 6. The system stores the discussion data in the database. 7. The system reloads the Stock Community page and displays the new discussion.
Alternative Flow	5a. The input data is invalid. 5The system displays an error notification. The use case continues with step 4

Exception Flow	



Activity Diagram : Create a New Discussion

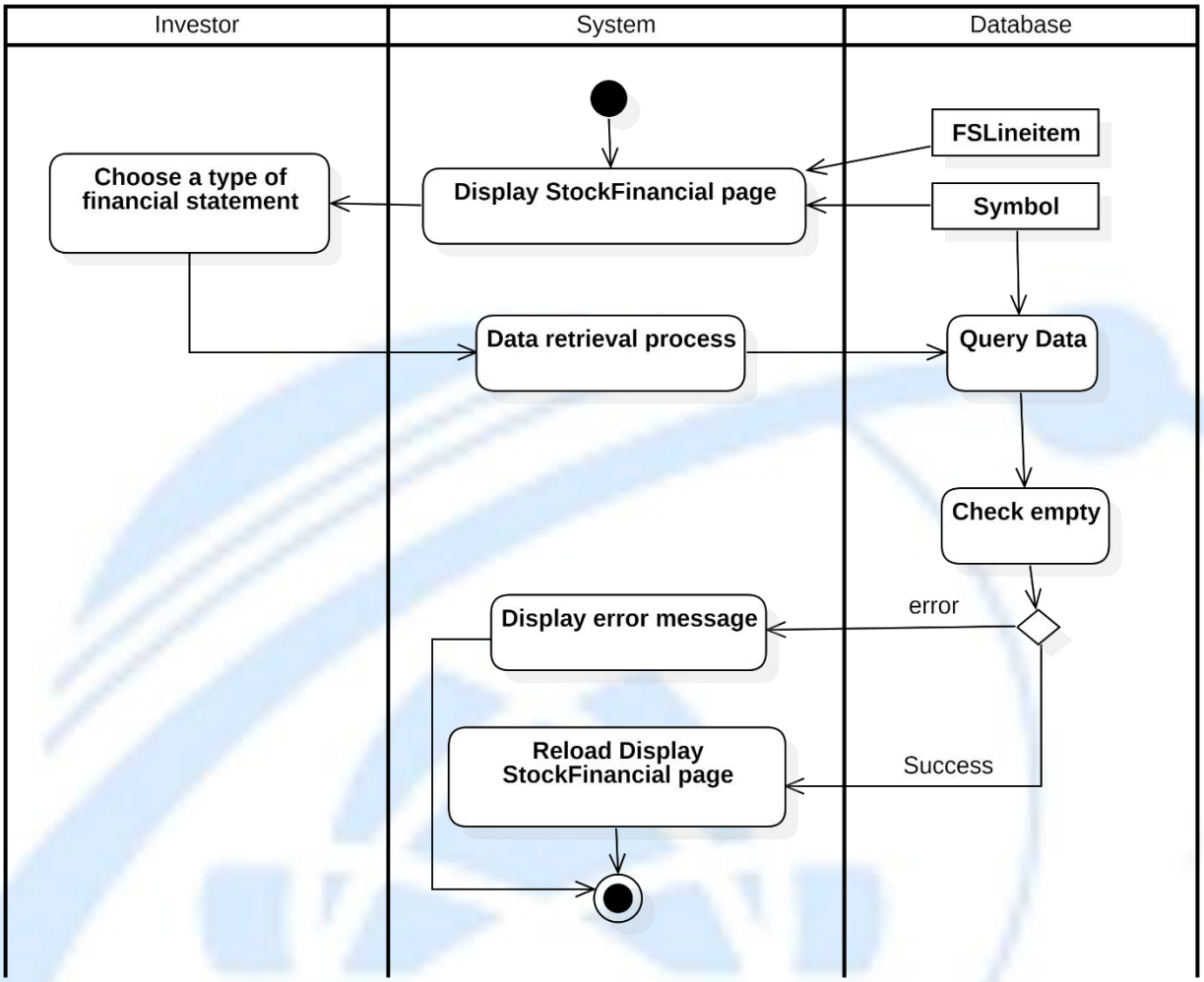
3.2.6.4. View Financial Statement

3.2.6.4.1 Change Report Type

Table 3.24: Change Report Type usecase description

Attribute	Description
Use-case Name	View Financial Statements and Change Report Type

Description	The system allows an investor to view a financial statement report for a selected company. The system retrieves the requested data, checks for availability, and displays the financial statement or an empty result if no data is available.
Trigger	The user selects the type of financial statement to view.
Pre-condition	The user is logged into the system. The user has selected a company and the type of financial statement.
Post-condition	The system displays the financial statement if data is available. If no data is available, the system displays an empty result.
Basic flow	1. The system displays the financial statements page. 2. The user chooses the type of financial statement to view. 3. The system retrieves the data for the selected statement type. 4. The system checks if the data is available. 5. If data exists, the system displays the financial statement.
Alternative flow	5a. If no data is available for the selected report type, the system displays an empty result message. End this usecase
Exception flow	



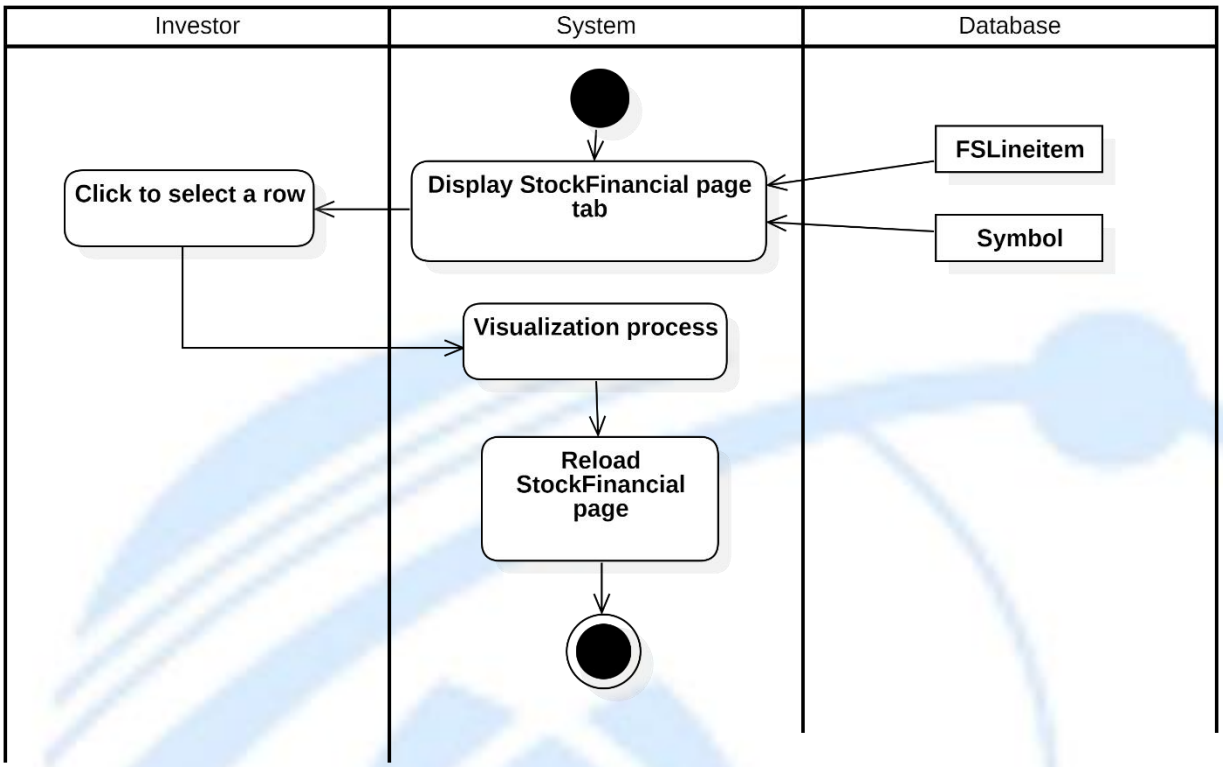
Activity Diagram : Change Report Type

3.2.6.4.2 Visualize Data

Table 3.25: Visualize Data usecase description

Attribute	Description
Use-case Name	Display Financials Statement Visualization
Description	The system allows an investor to visualize financial data by selecting a row from the financial statement page. The system processes the request and displays a bar chart of the selected data.

Trigger	The user clicks to select a row from the financial statement.
Pre-condition	The user is logged into the system. The user is viewing the financial statement page.
Post-condition	The system displays a bar chart representing the selected financial data.
Basic flow	1. The system displays the financial statement page. 2. The user clicks to select a row of financial data. 3. The system processes the request and visualizes the selected data. 4. The system displays the selected data in a bar chart format.
Alternative flow	
Exception flow	



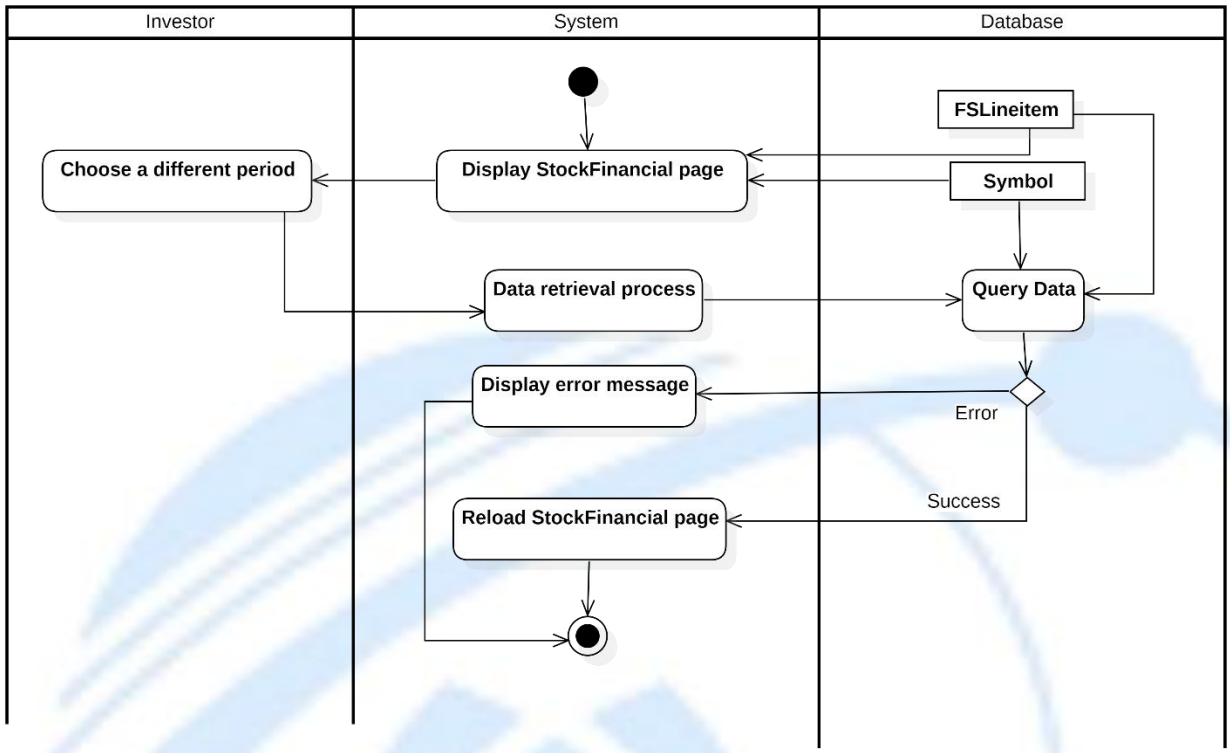
Activity Diagram : Visualize Data

3.2.6.4.3 Select Period

Table 3.26: Select Period usecase description

Attribute	Description
Use-case Name	Change Period for Financial Statement
Description	The system allows an investor to view financial statements for different periods. The system retrieves the data for the selected period and displays the financial statement.
Trigger	The user selects a different period for the financial statement.
Pre-condition	The user is logged into the system. The user is viewing the financial statement tab.

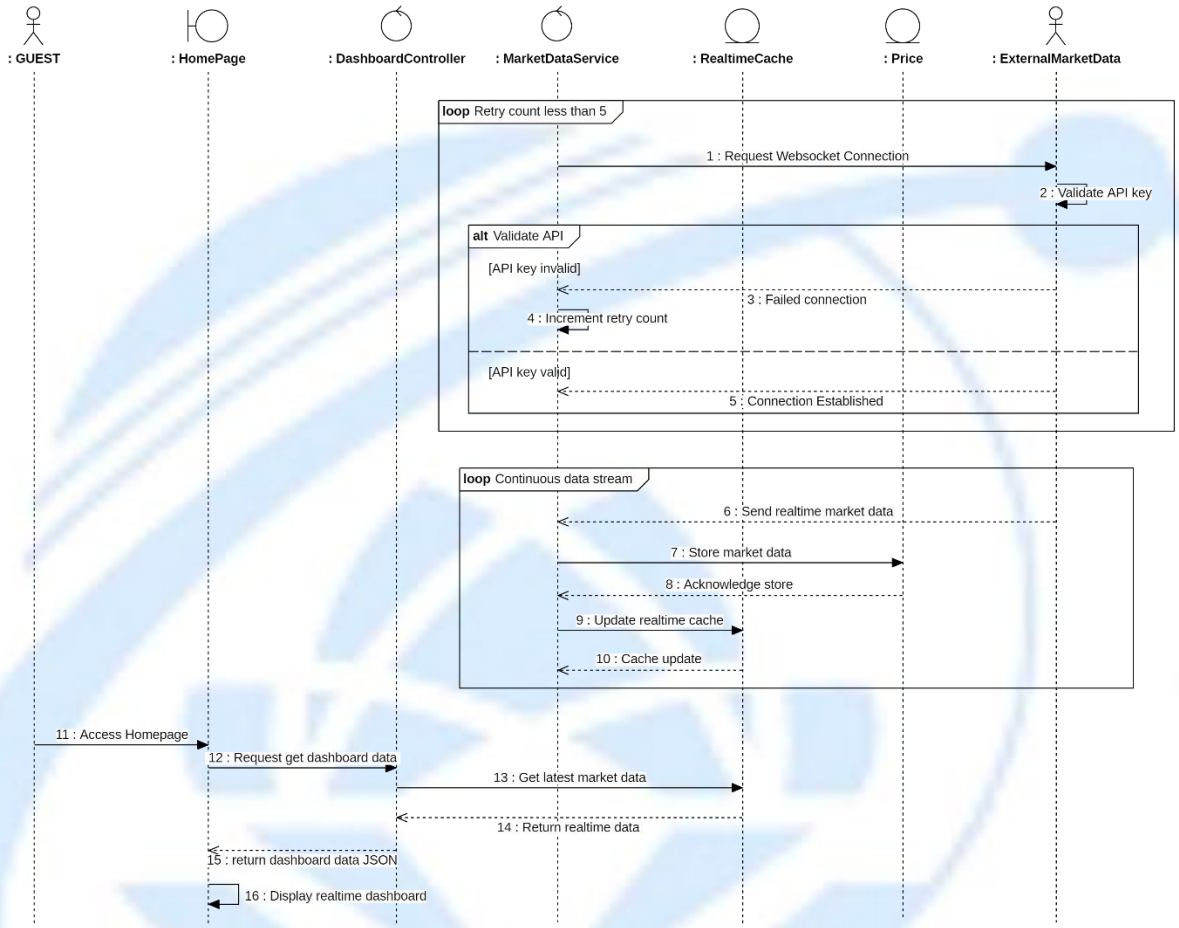
Post-condition	The system displays the financial statement for the selected period. If the data for the selected period is unavailable, the system displays an error message.
Basic flow	1. The system displays the financial statement tab. 2. The user selects a different period to view. 3. The system retrieves the data for the selected period. 4. If the data is available, the system displays the financial statement for the selected period.
Alternative flow	4a. If no data is available for the selected period, the system displays an error message indicating that the data is unavailable. End this usecase
Exception flow	



Activity Diagram : Select Period

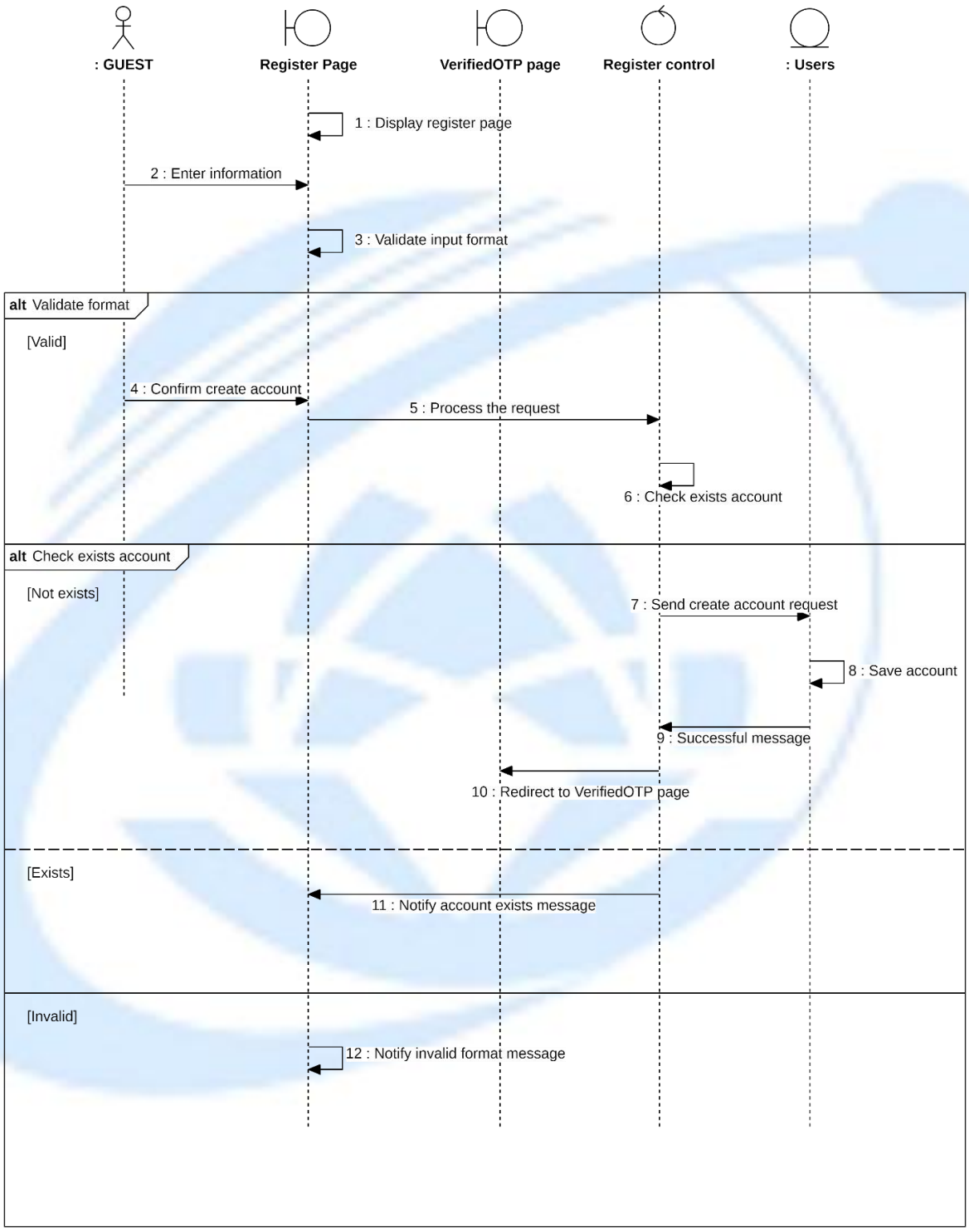
3.3. Sequence Diagram

3.3.1. View Dashboard



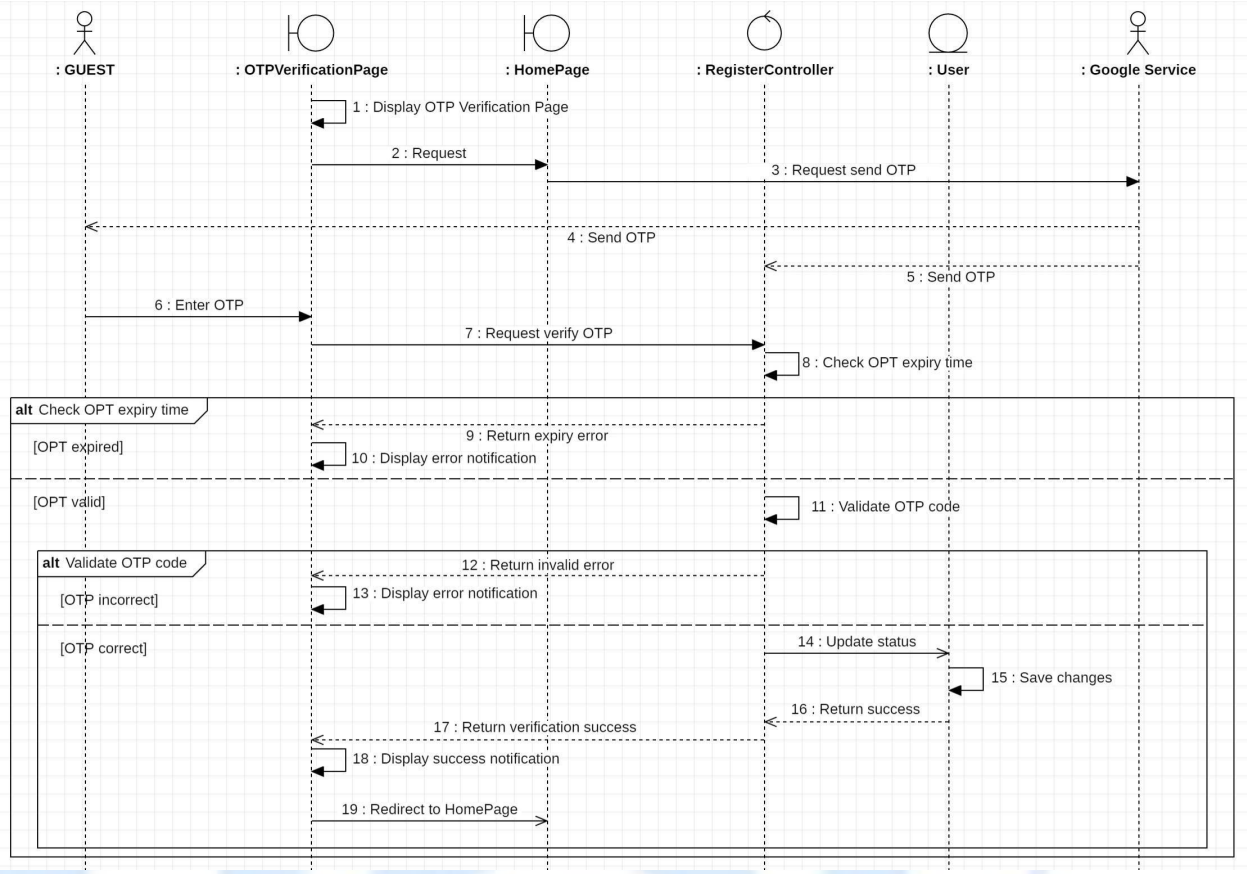
Sequence Diagram 1: View Dashboard

3.3.2. Register



Sequence Diagram 2: Register

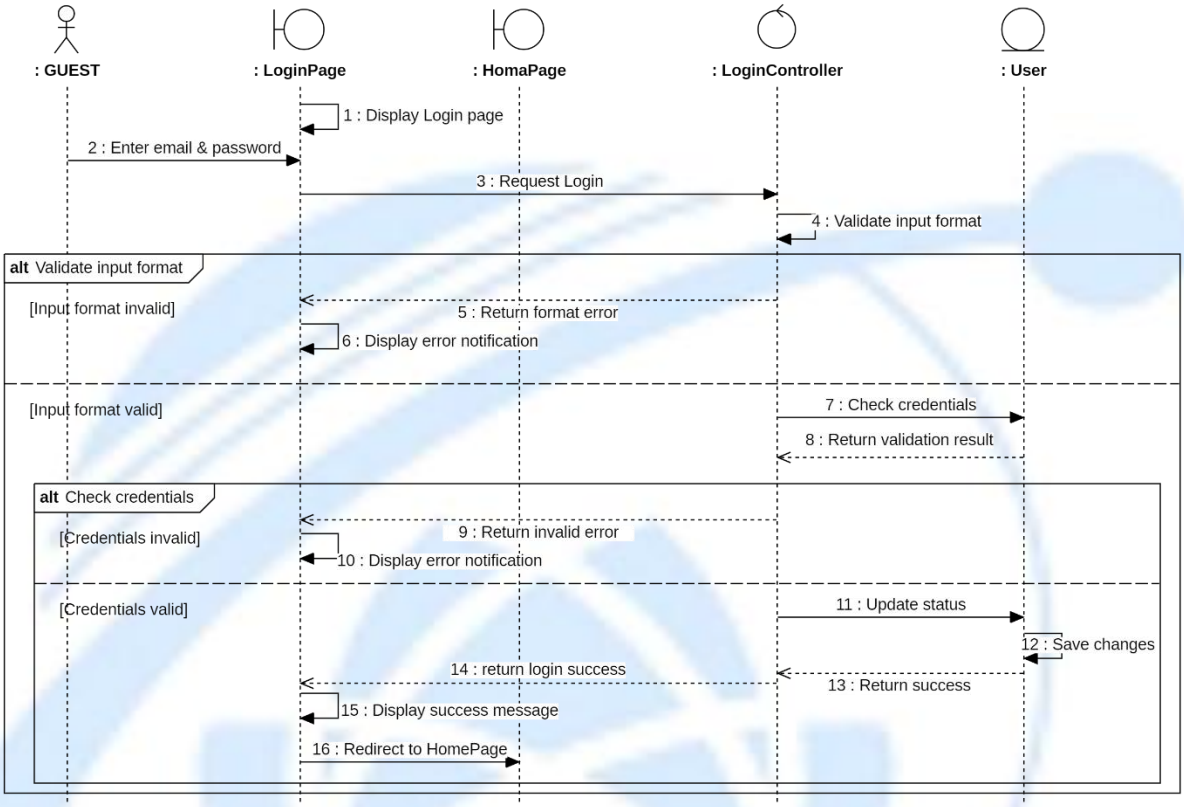
3.3.3. Verify registration



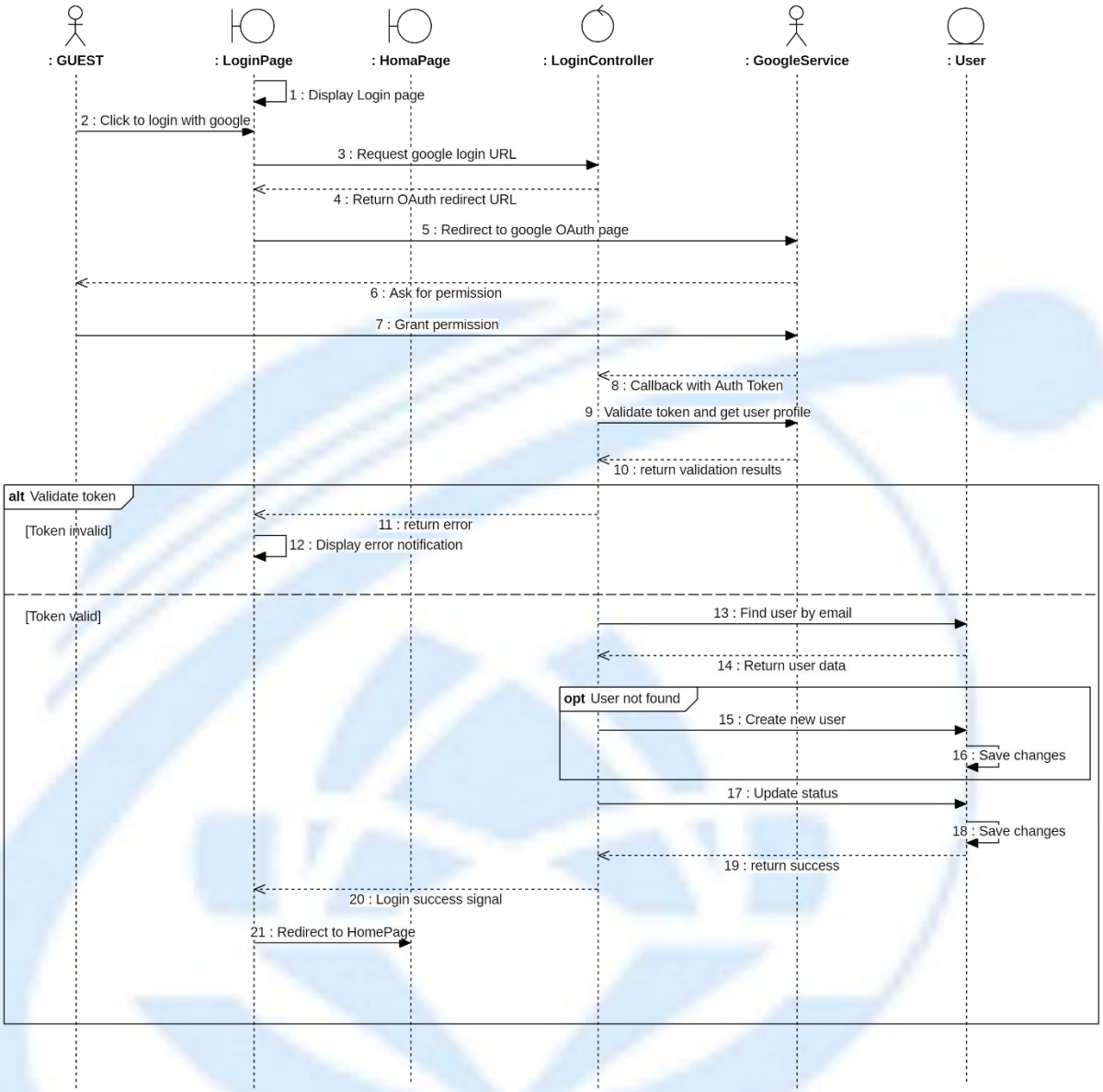
Sequence Diagram 3: Verify registration

3.3.4. Login

3.3.4.1. Login with Email and Password

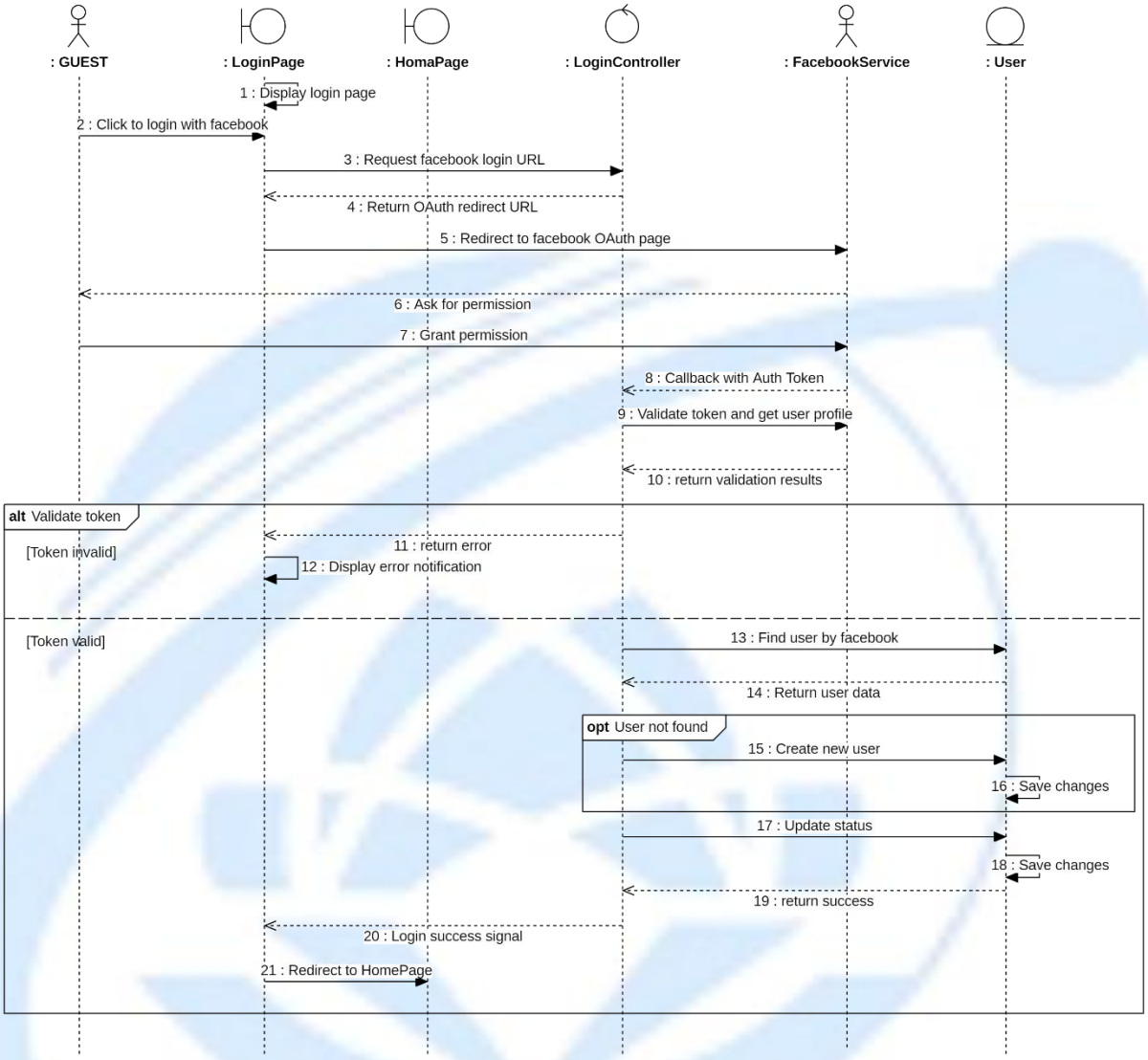


Sequence Diagram 4: Login with Email and Password



Sequence Diagram 5: Login with Google

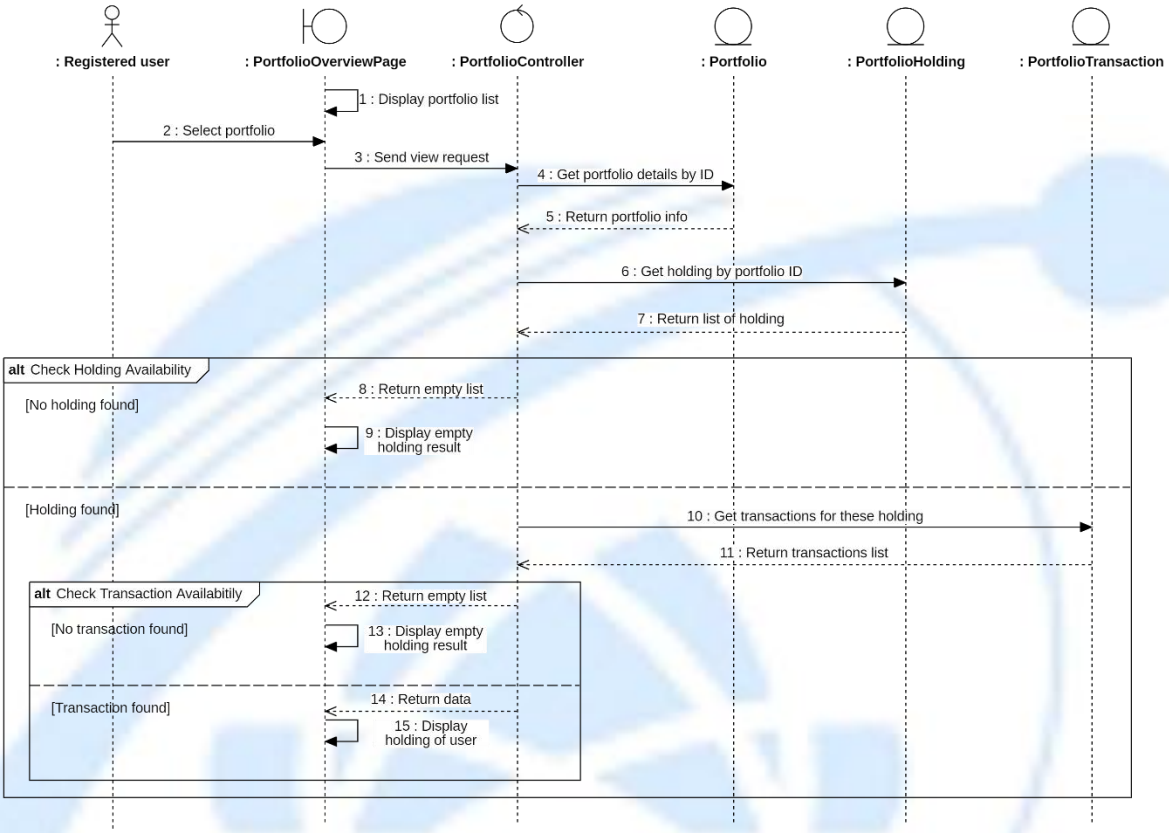
3.3.4.2. Login with Facebook



Sequence Diagram : Login with Facebook

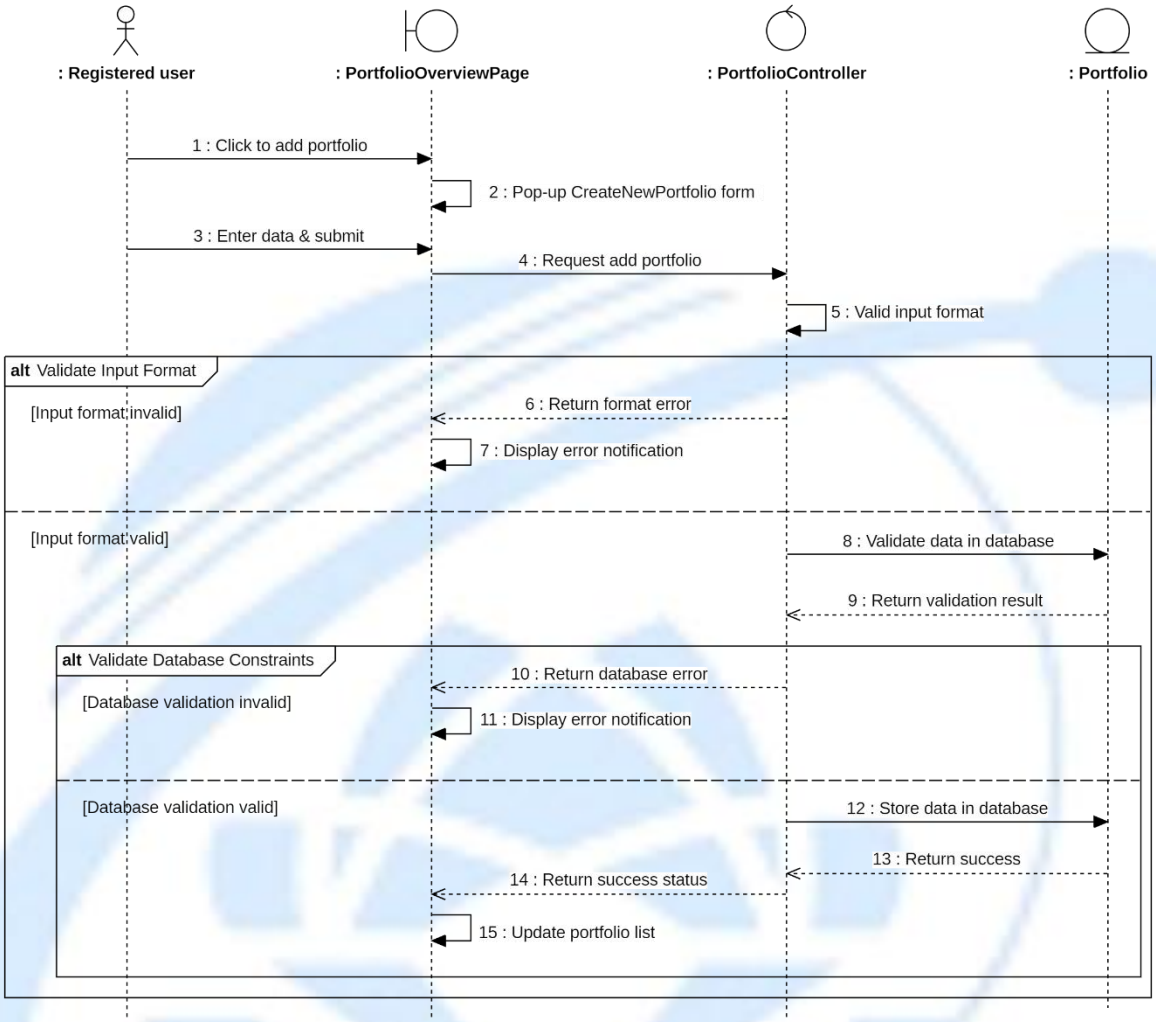
3.3.5. Manage Portfolio

3.3.5.1. View Portfolio



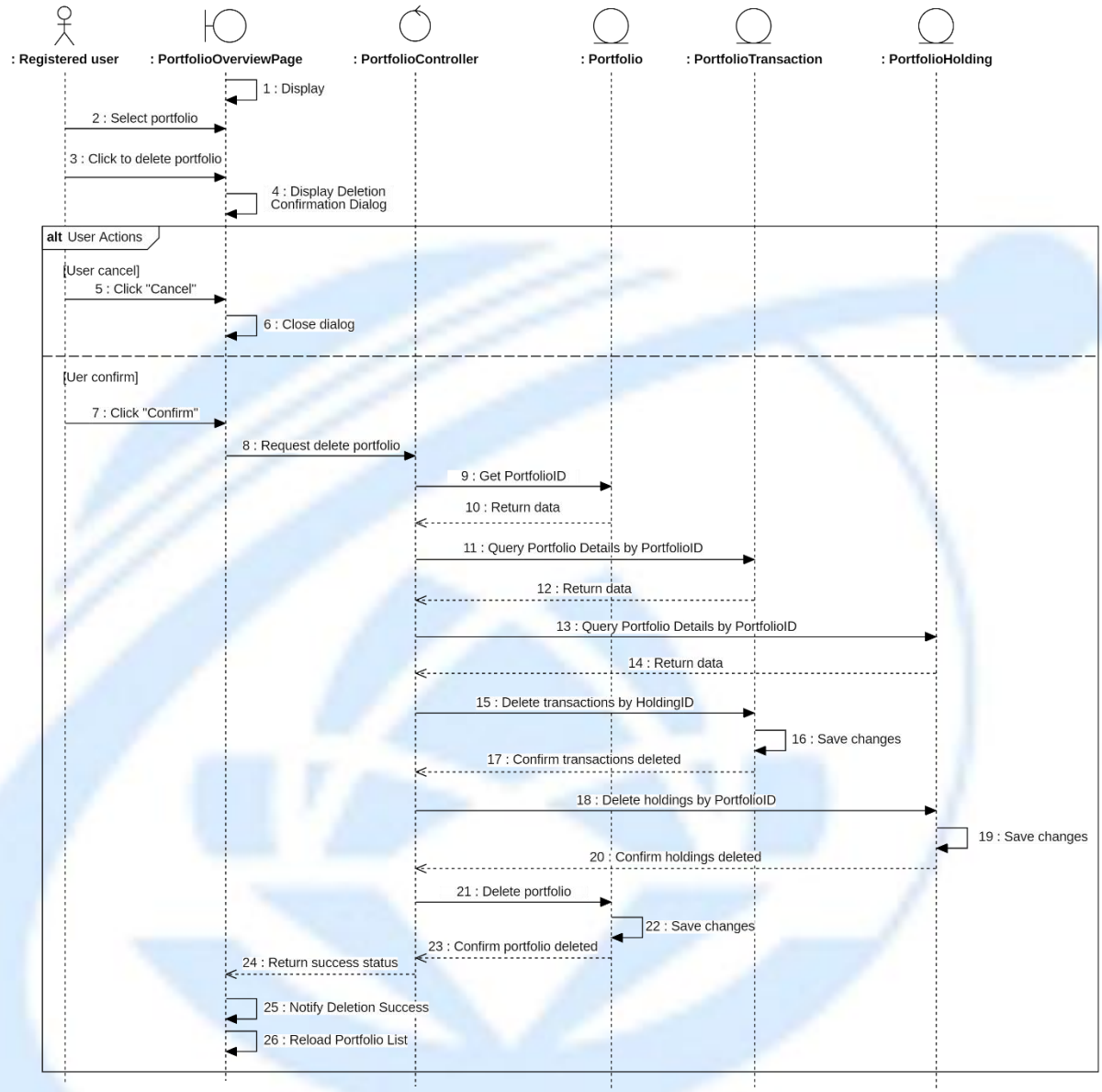
Sequence Diagram : View Portfolio

3.3.5.2. Add New Portfolio



Sequence Diagram : Add New Portfolio

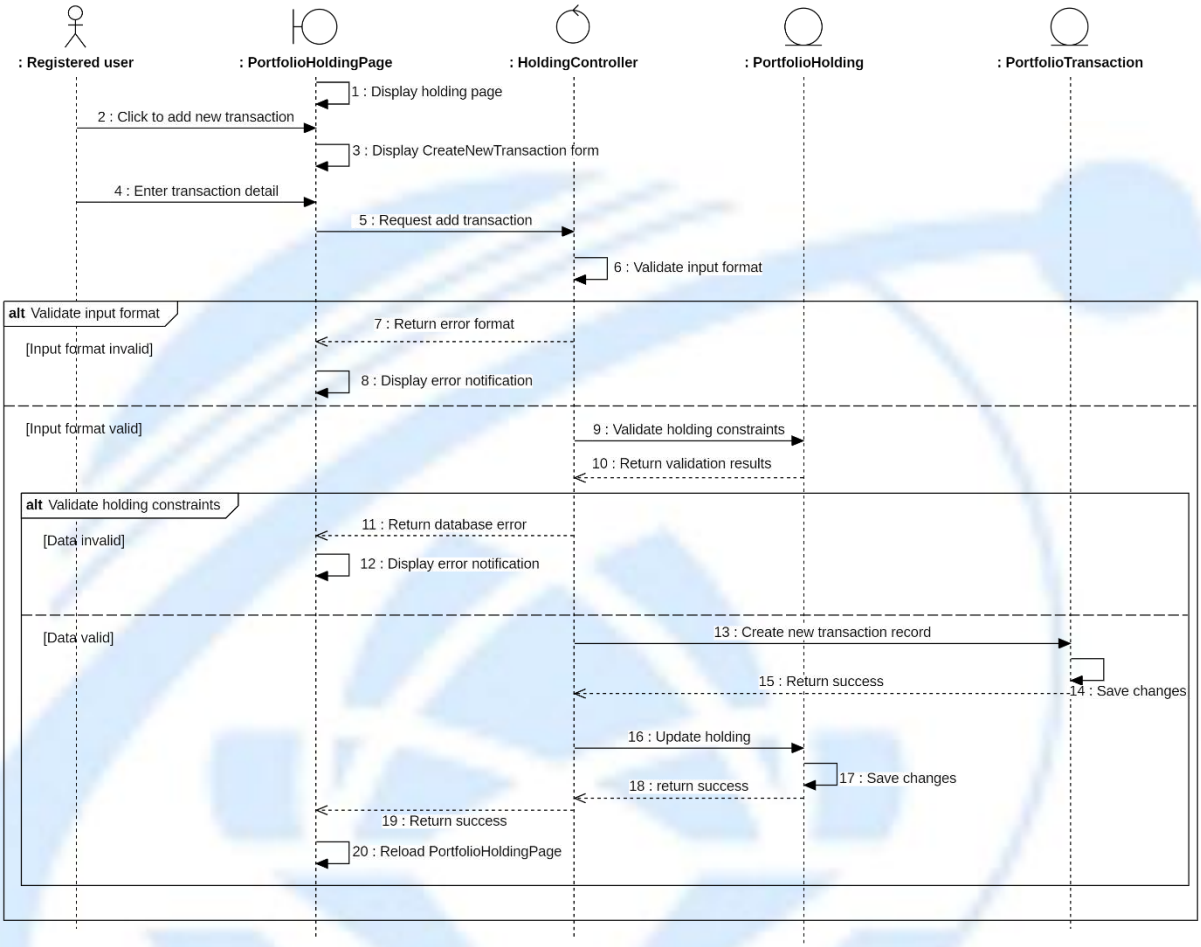
3.3.5.3. Remove Portfolio



Sequence Diagram : Remove Portfolio

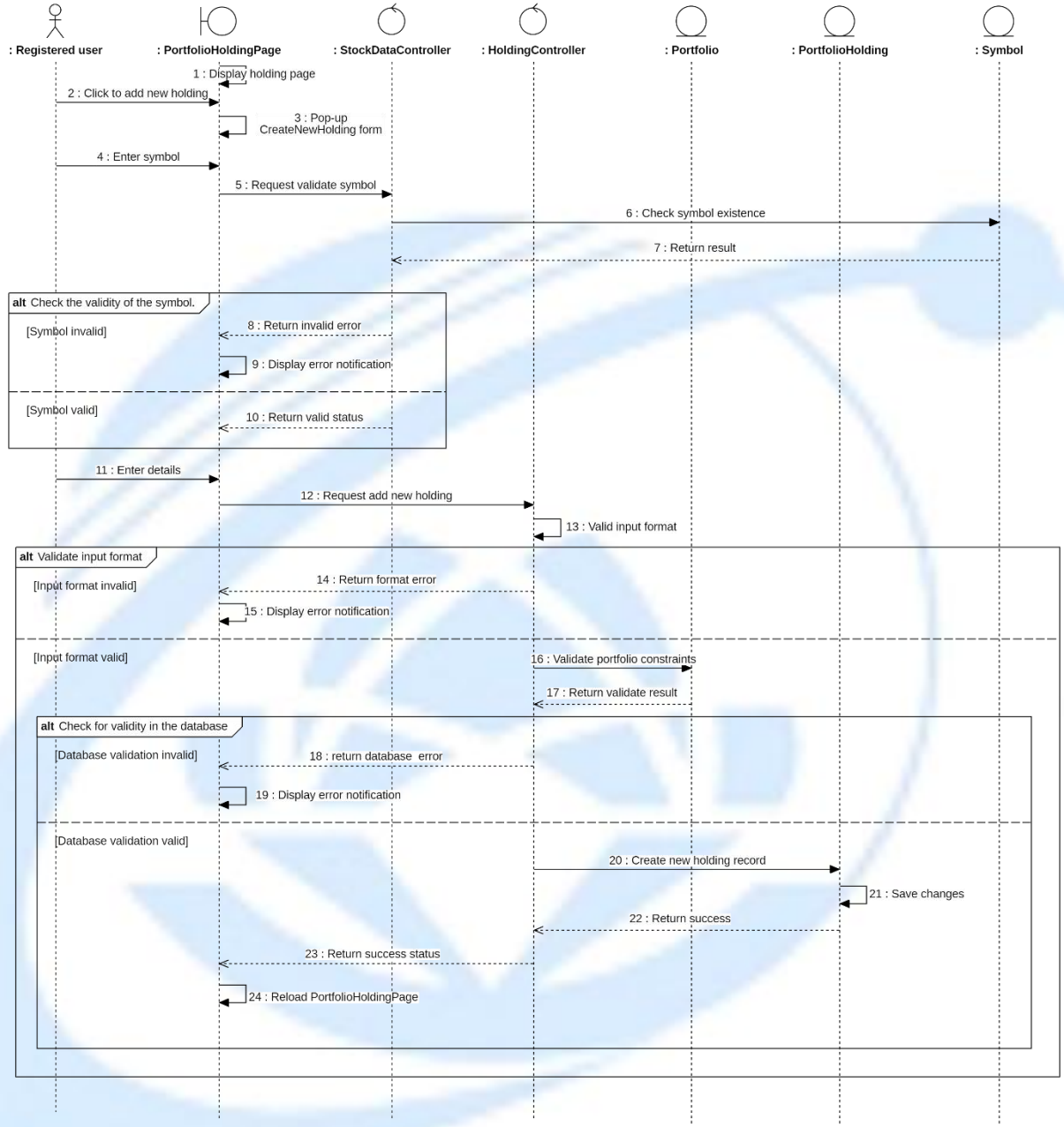
3.3.5.4. Update Portfolio

3.3.5.4.1 Add New Transaction



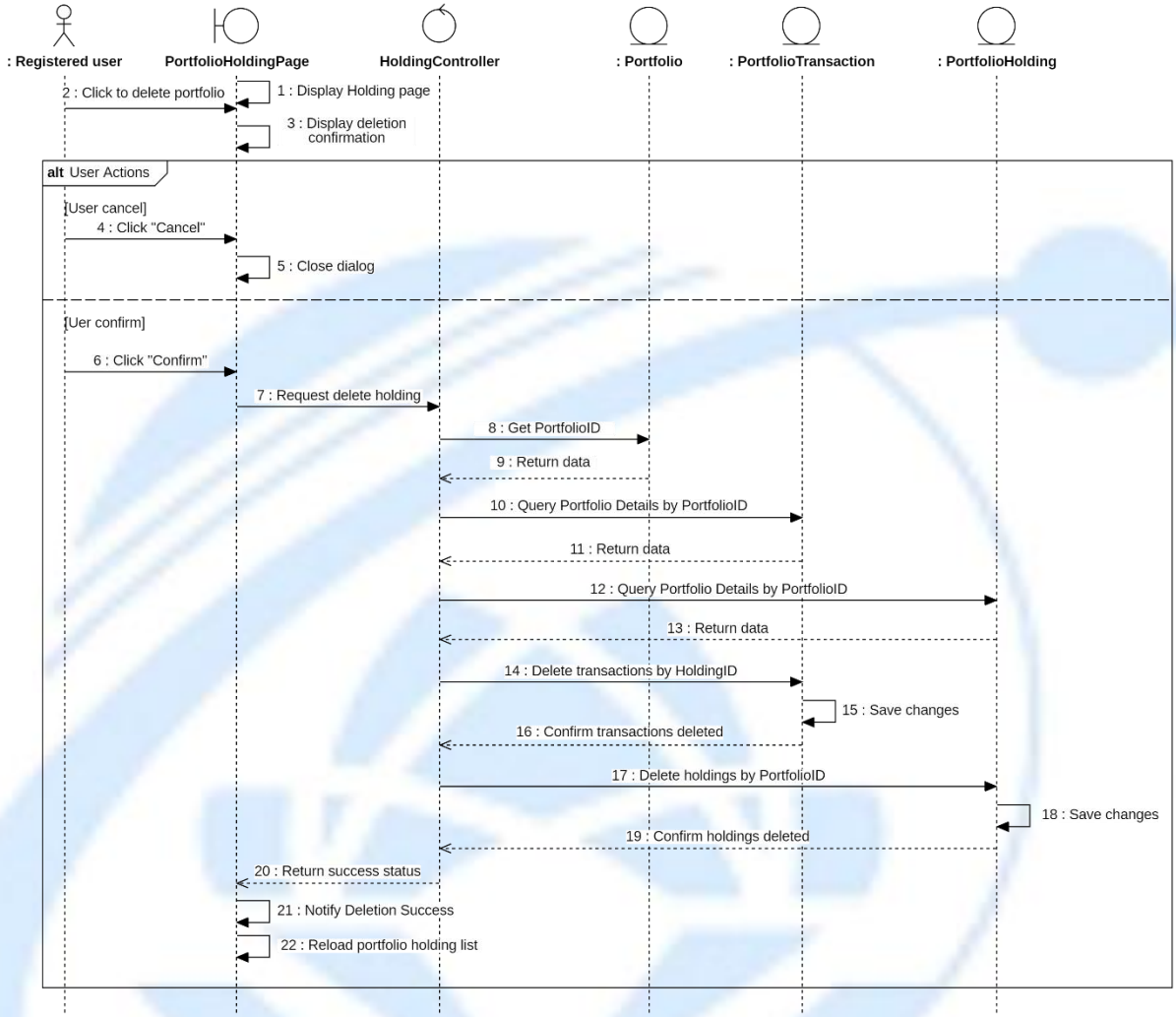
Sequence Diagram : Add New Transaction

3.3.5.4.2 Add New Holding



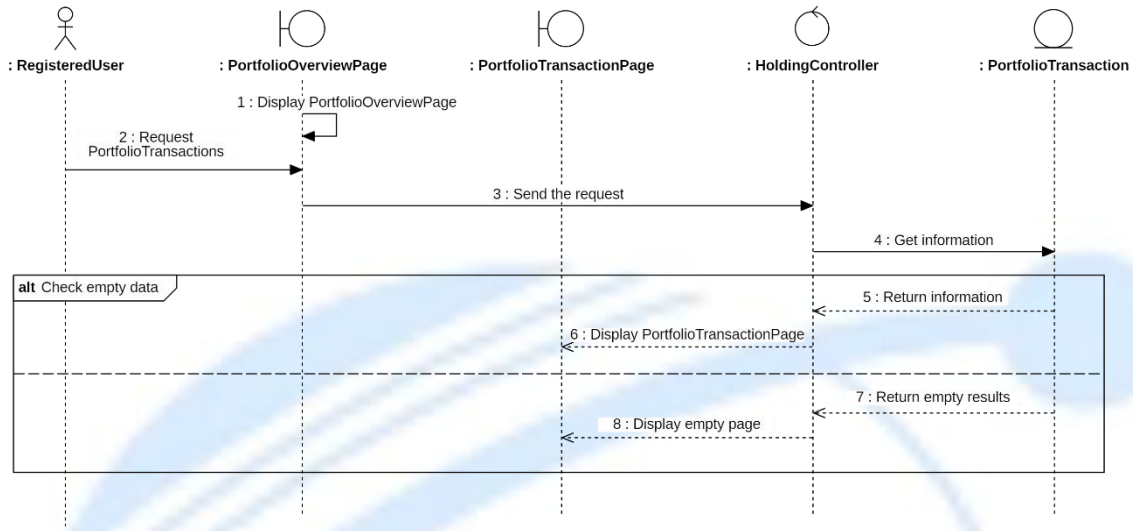
Sequence Diagram : Add New Holding

3.3.5.4.3 Delete Holding



Sequence Diagram : Delete Holding

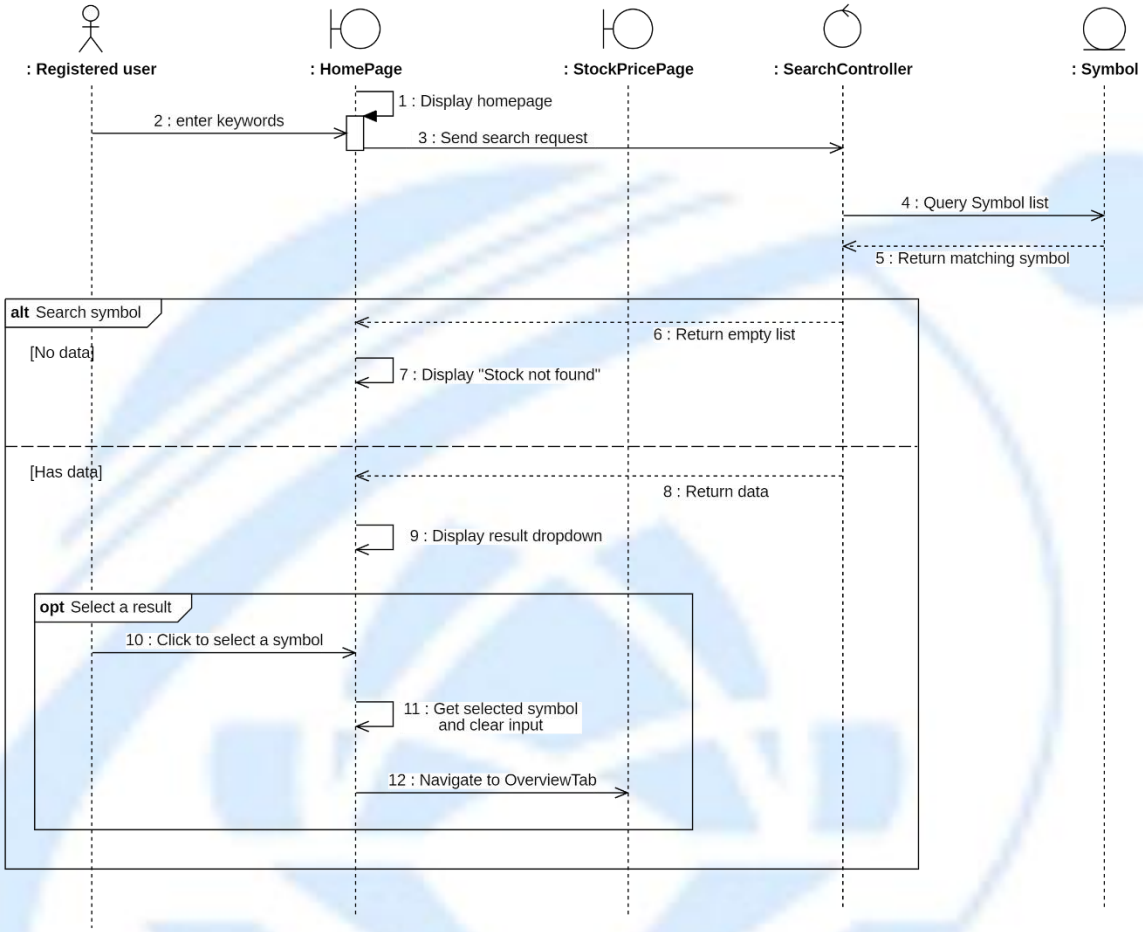
3.3.5.5. View Transaction History



Sequence Diagram : View Transaction History

3.3.6. Research Stock

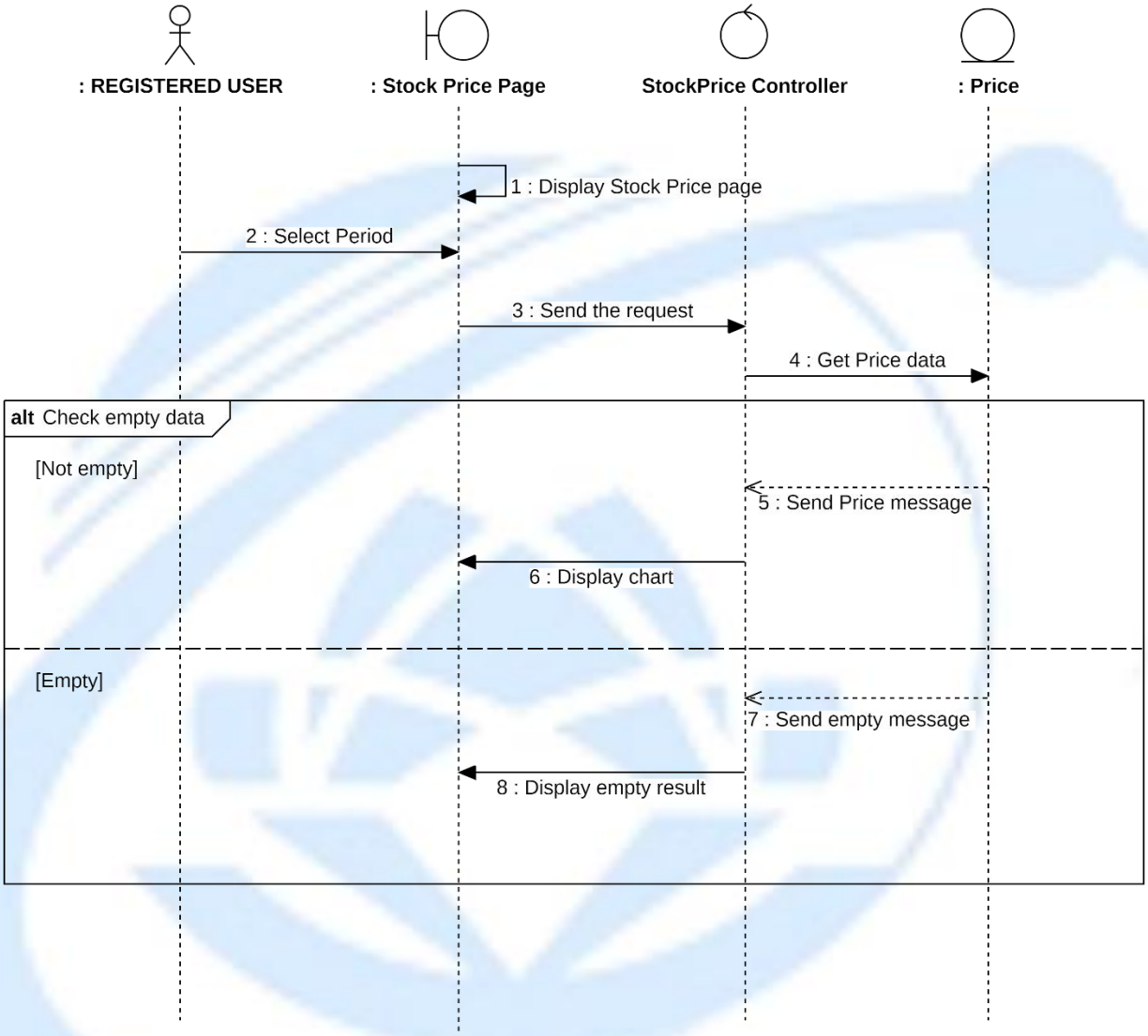
3.3.6.1. Search Stock



Sequence Diagram : Search Stock

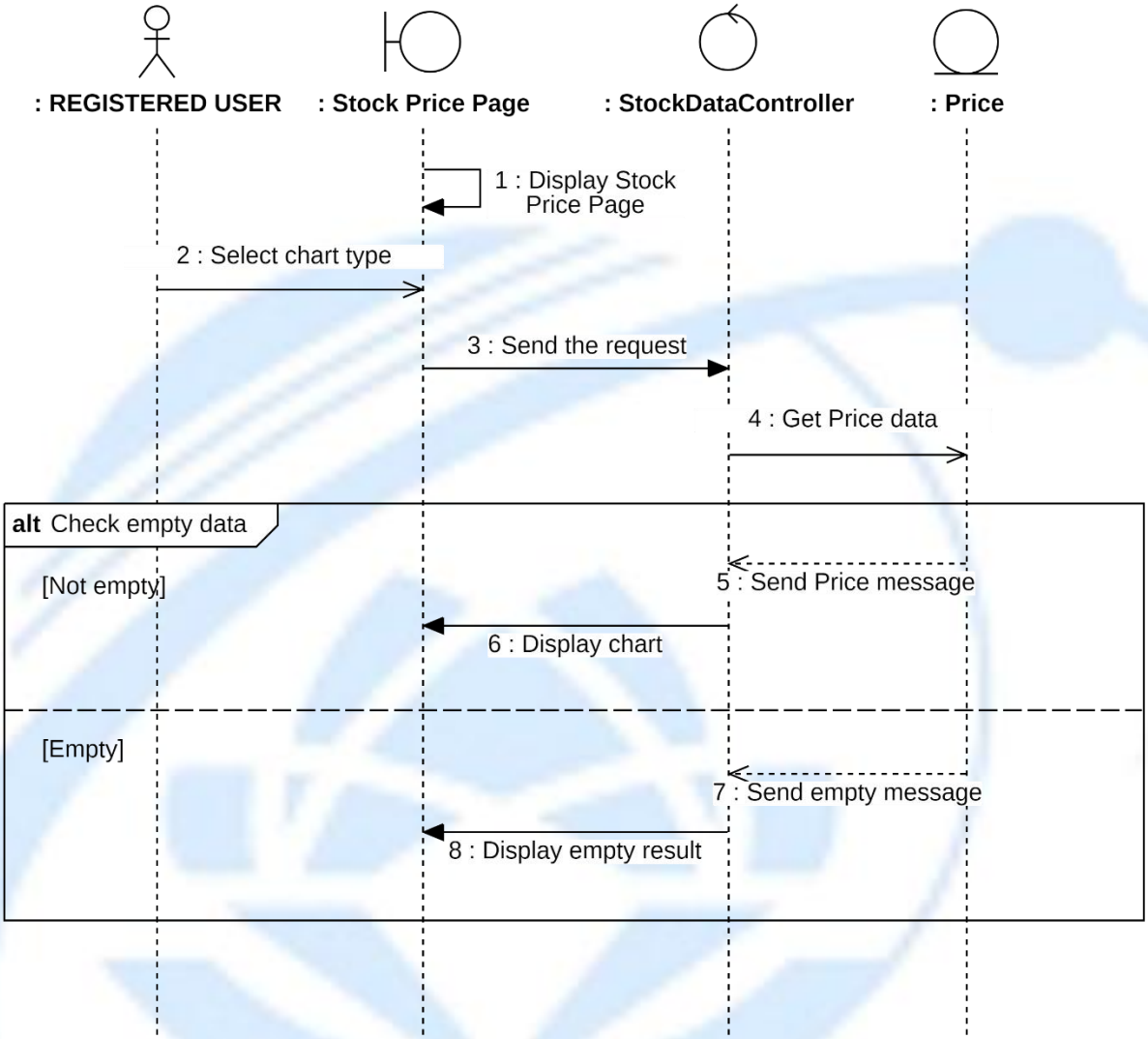
3.3.6.2. View Price History

3.3.6.2.1 Change Period



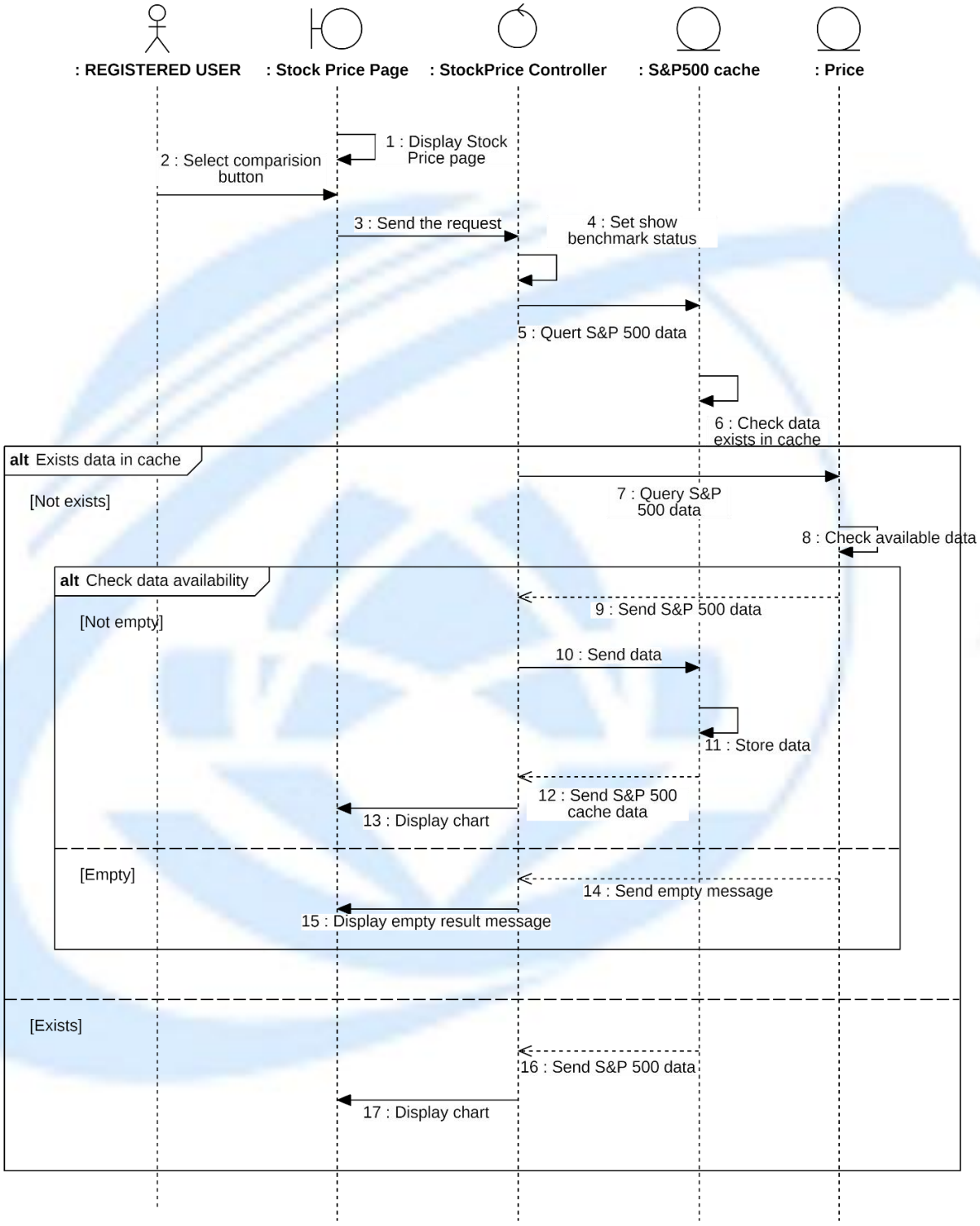
Sequence Diagram : Change Period

3.3.6.2.2 Change Chart Type



Sequence Diagram : Change Chart Type

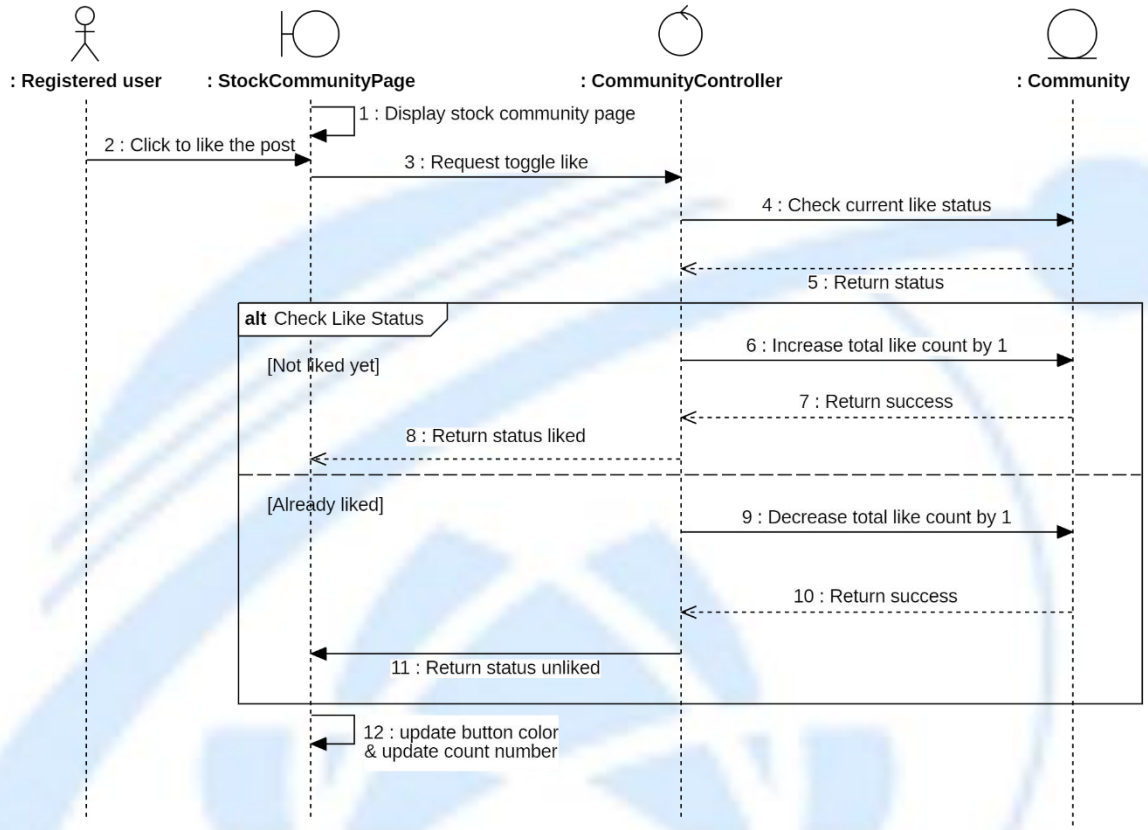
3.3.6.2.3 Select Comparition with S&P 500



Sequence Diagram : Select Comparition with S&P 500

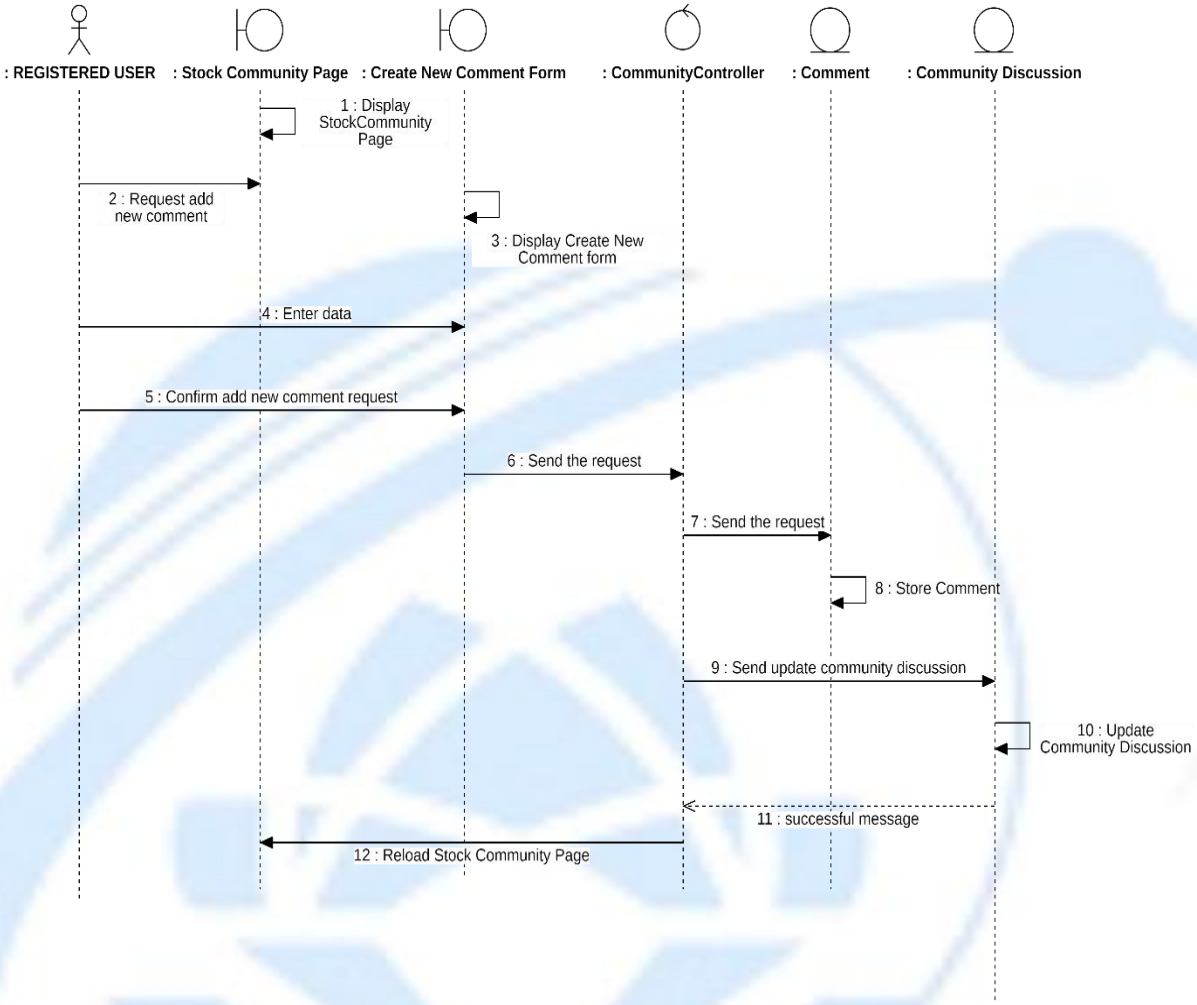
3.3.6.3. View Community Discussion

3.3.6.3.1 Like Discussion



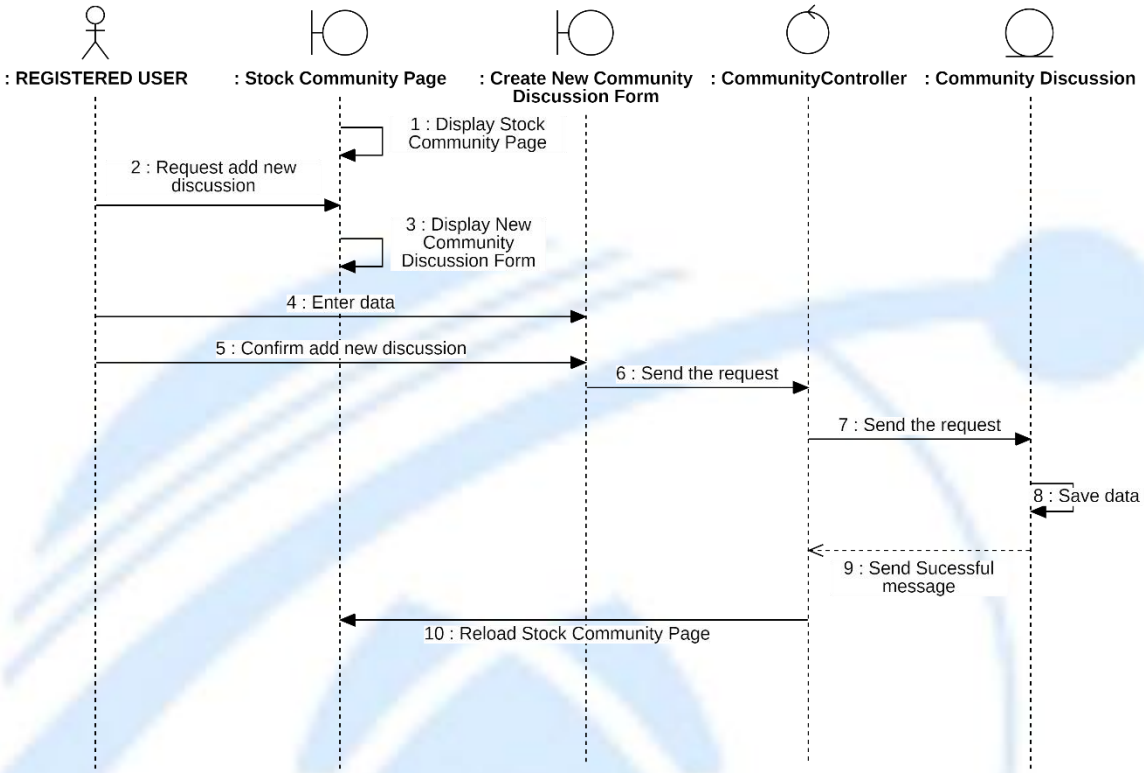
Sequence Diagram : Like Discussion

3.3.6.3.2 Add New Comment



Sequence Diagram : Add a New Comment

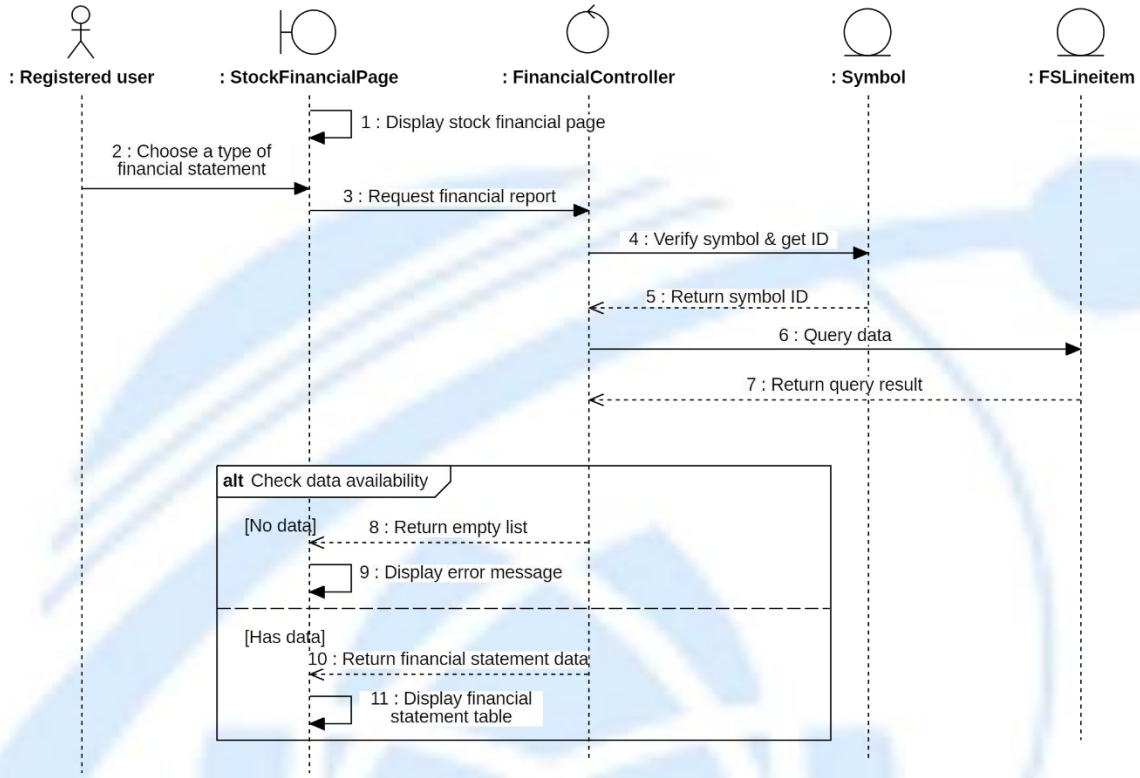
3.3.6.3.3 Create a New Disscussion



Sequence Diagram : Create a New Disscusion

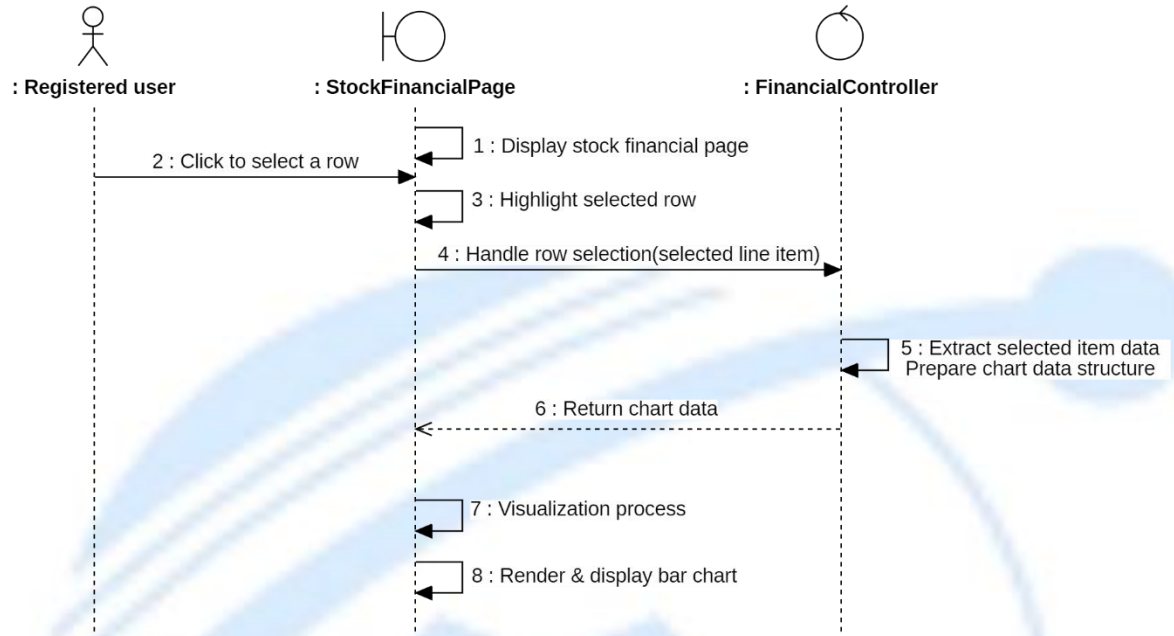
3.3.6.4. View Financials Statements

3.3.6.4.1 Change Report Type



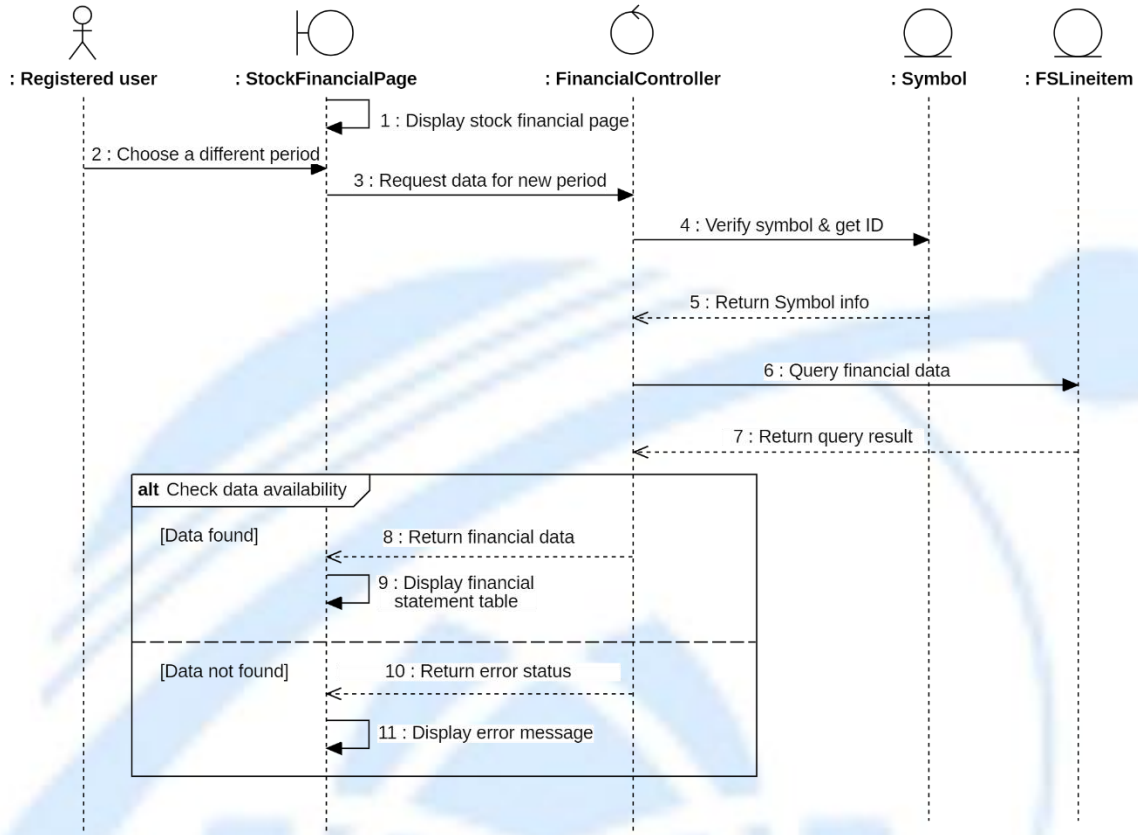
Sequence Diagram : Change Report Type

3.3.6.4.2 Visualize Data



Sequence Diagram : Visualize Data

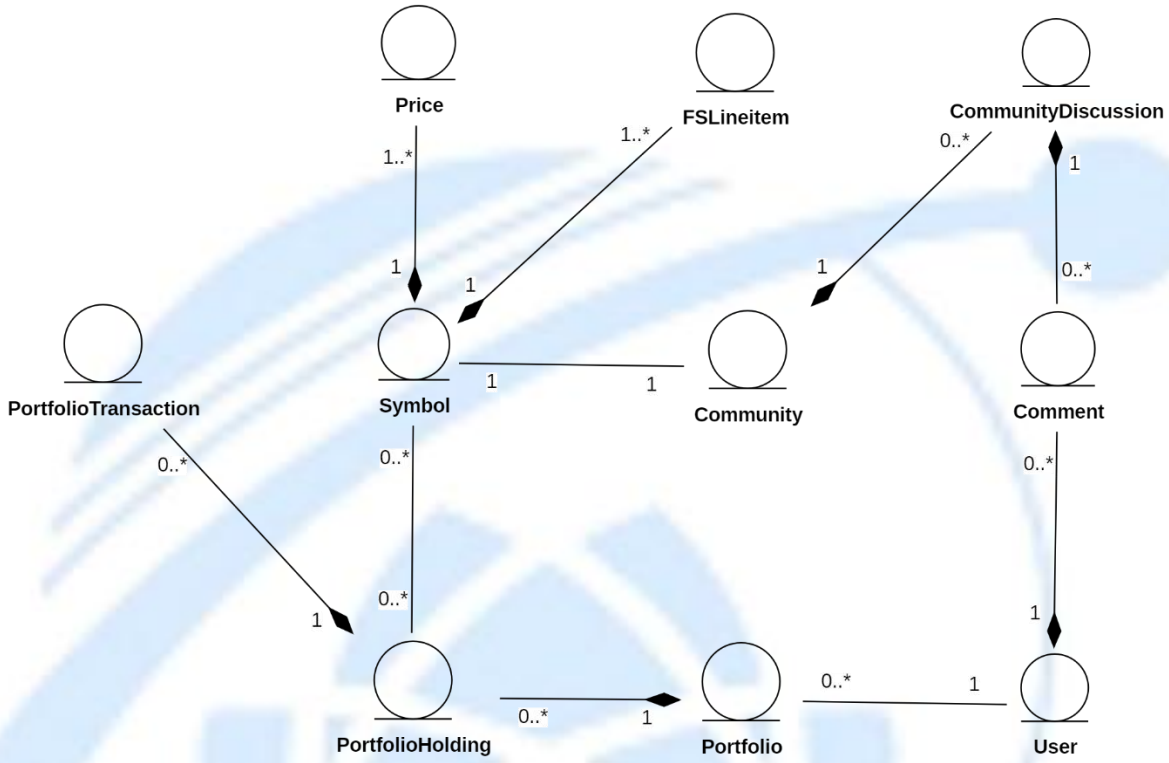
3.3.6.4.3 Select Period



Sequence Diagram : Seclect Pediod

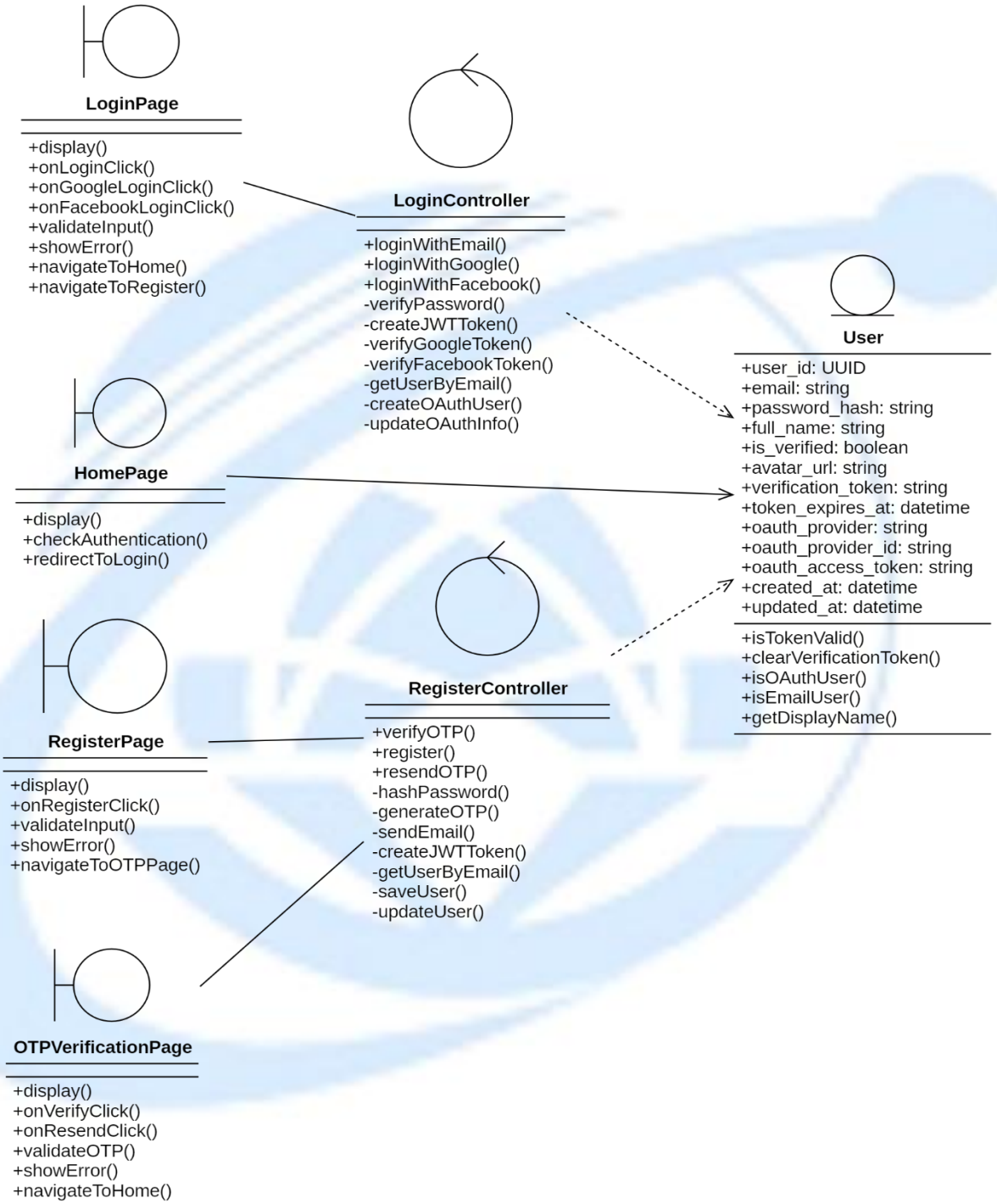
3.4. Class Diagram and Entity Class Diagram

3.4.1. Entity Class Diagram



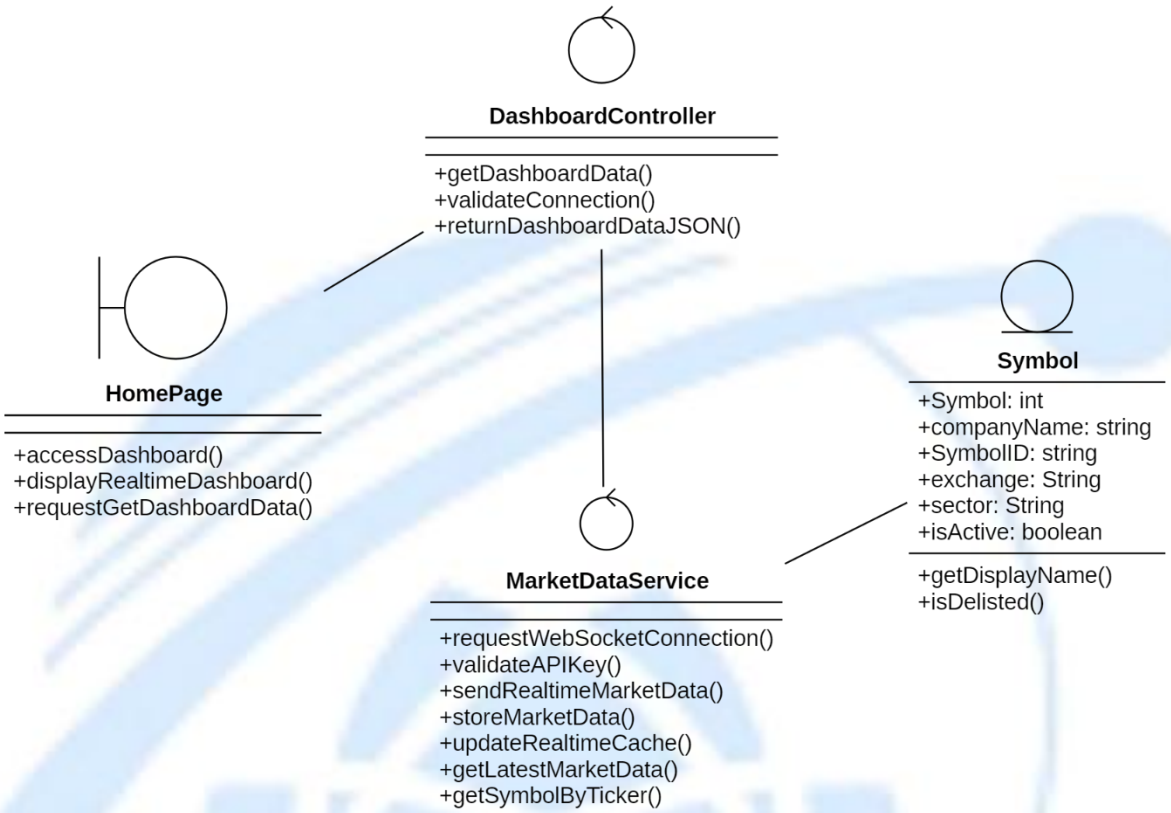
Class Diagram 1: Entity Class Diagram

3.4.2. Authentication



Class Diagram 2: Authentication

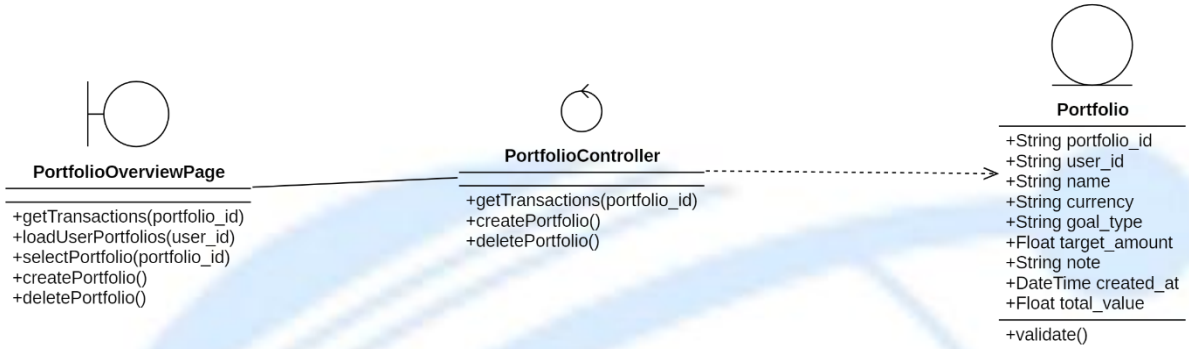
3.4.3. View Dashboard



Class Diagram 3: View Dashboard

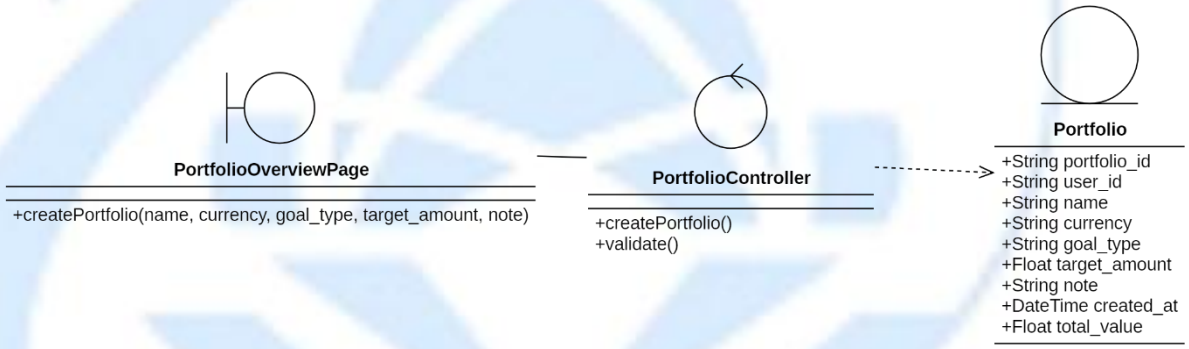
3.4.4. Manage Portfolio

3.4.4.1. View Portfolio



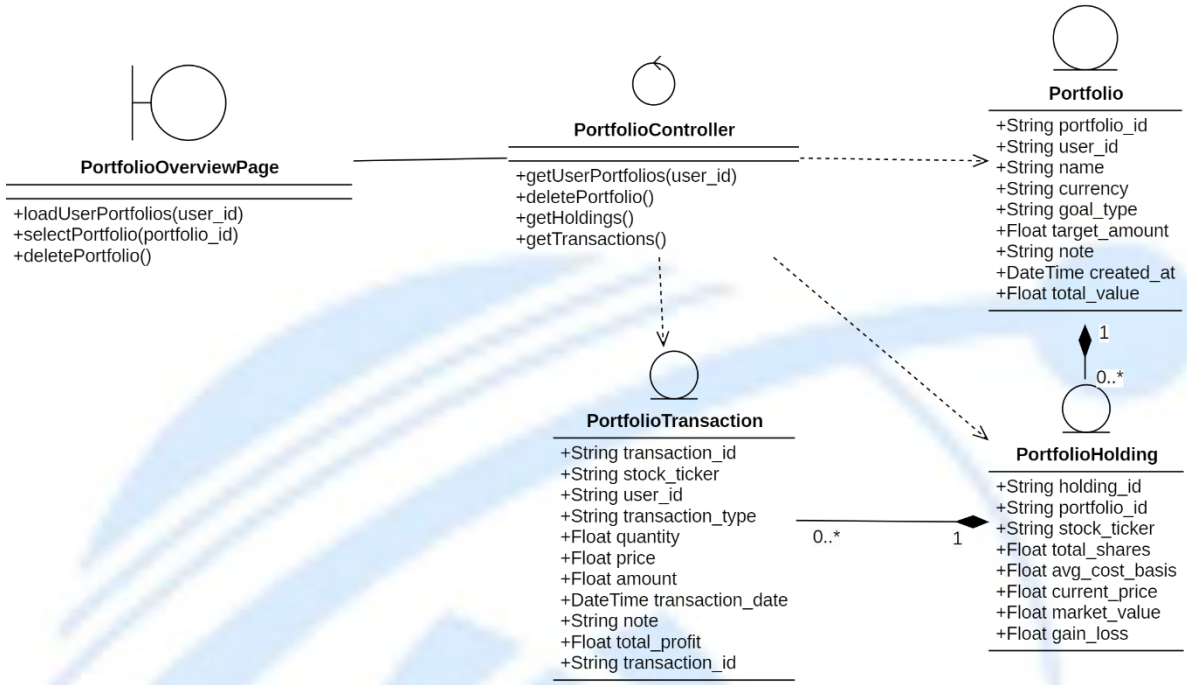
Class Diagram 4: View Portfolio

3.4.4.2. Add New Portfolio



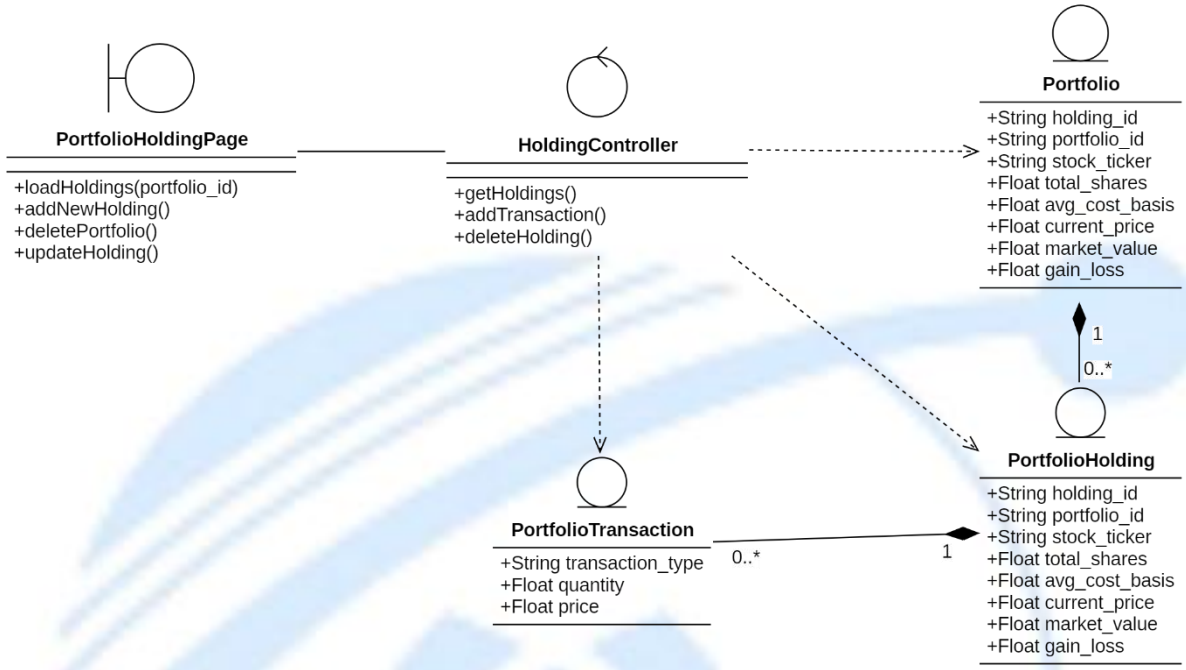
Class Diagram 5: Add New Portfolio

3.4.4.3. Remove Portfolio



Class Diagram 6:Remove Portfolio

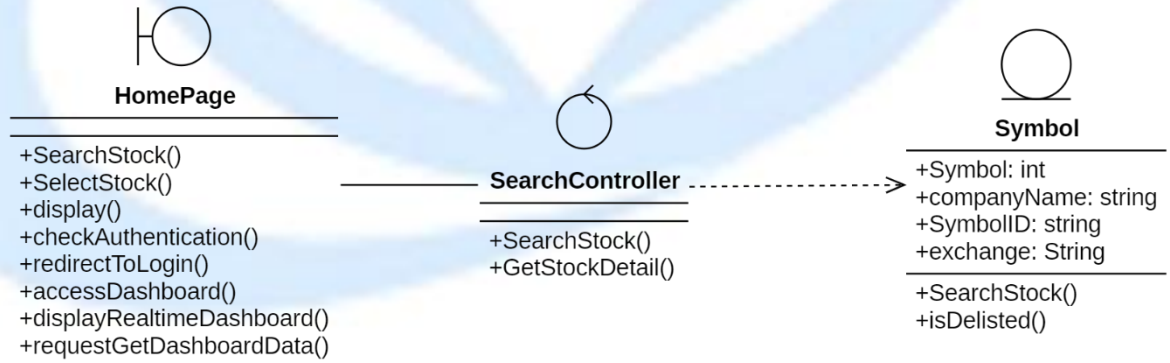
3.4.4.4. Update Portfolio



Class Diagram 7: Update Portfolio

3.4.5. Research Stock

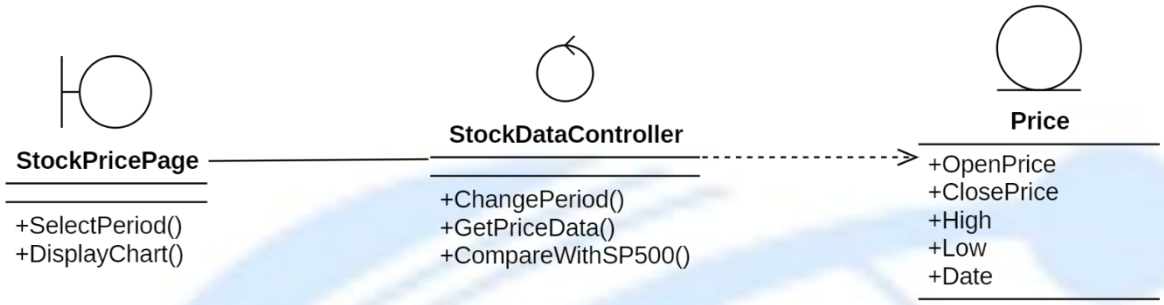
3.4.5.1. Search Stock



Class Diagram 8: Search Stock

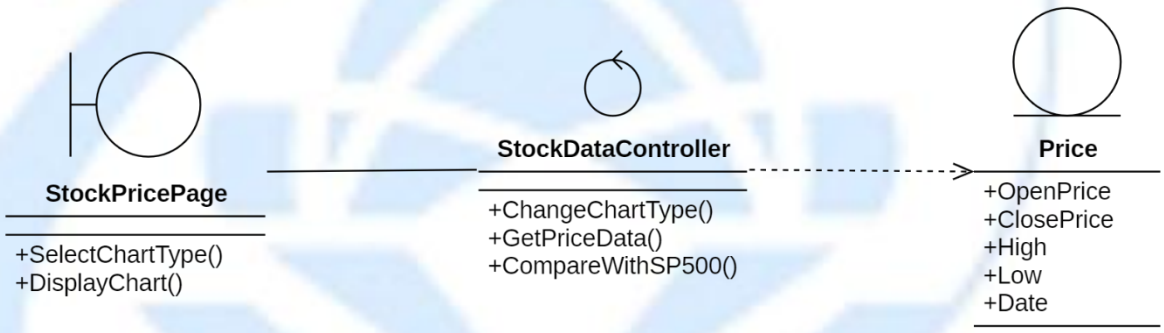
3.4.5.2. View Price History

3.4.5.2.1 Change Period



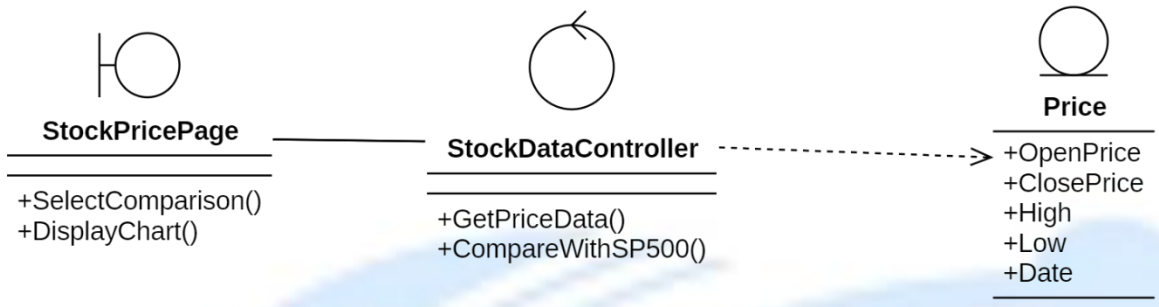
Class Diagram 9: Change Period

3.4.5.2.2 Change Chart Type



Class Diagram 10: Change Chart Type

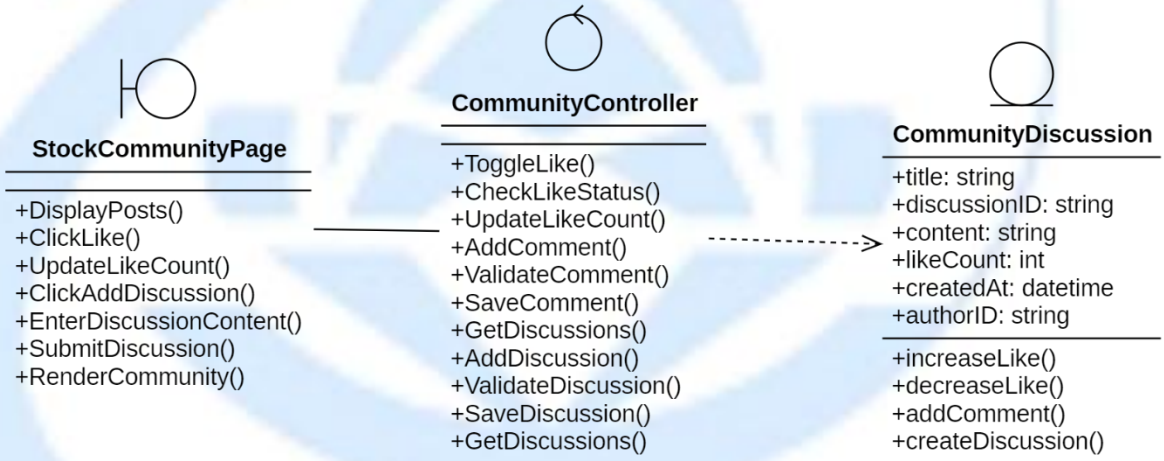
3.4.5.2.3 Select Comparition with S&P 500



Class Diagram 11: Select Comparition with S&P 500

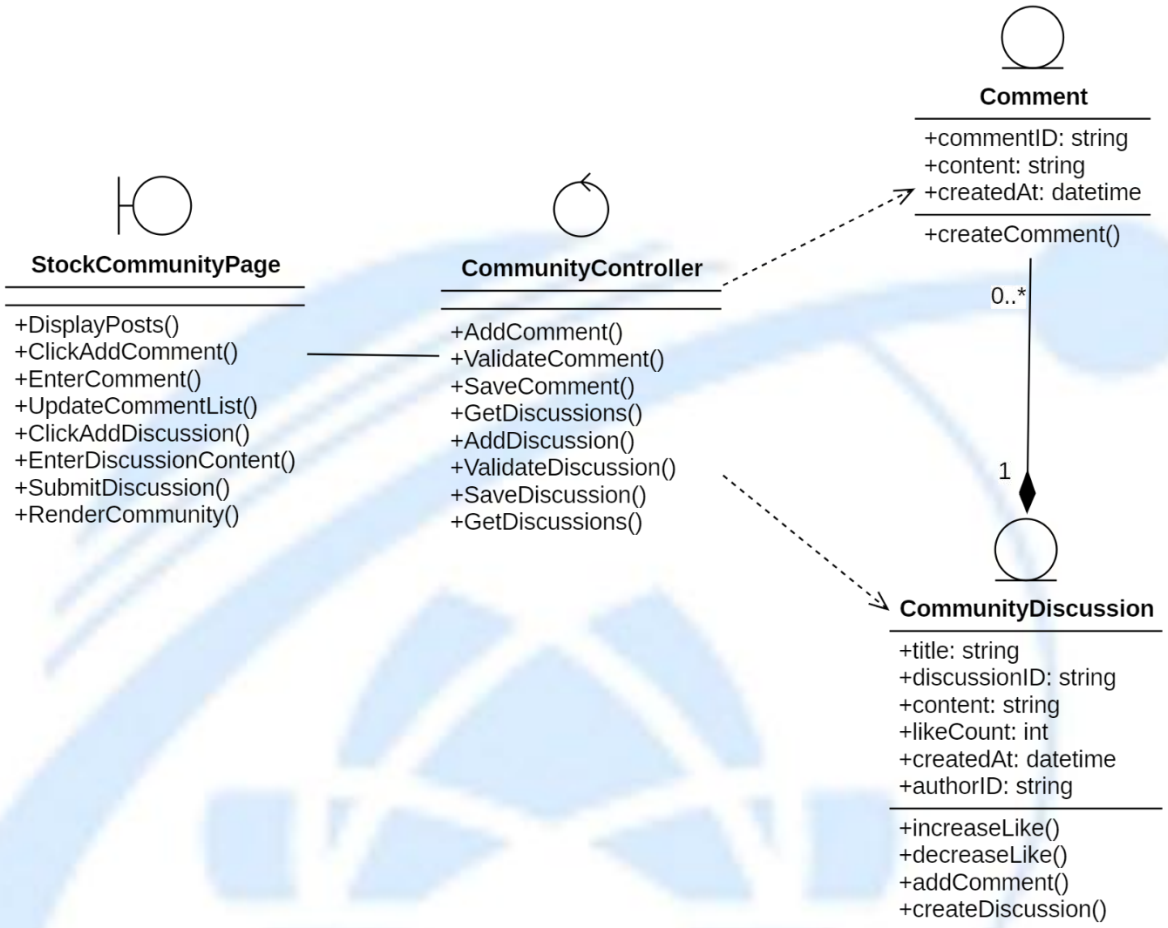
3.4.5.3. View Community Discussion

3.4.5.3.1 Like Discussion



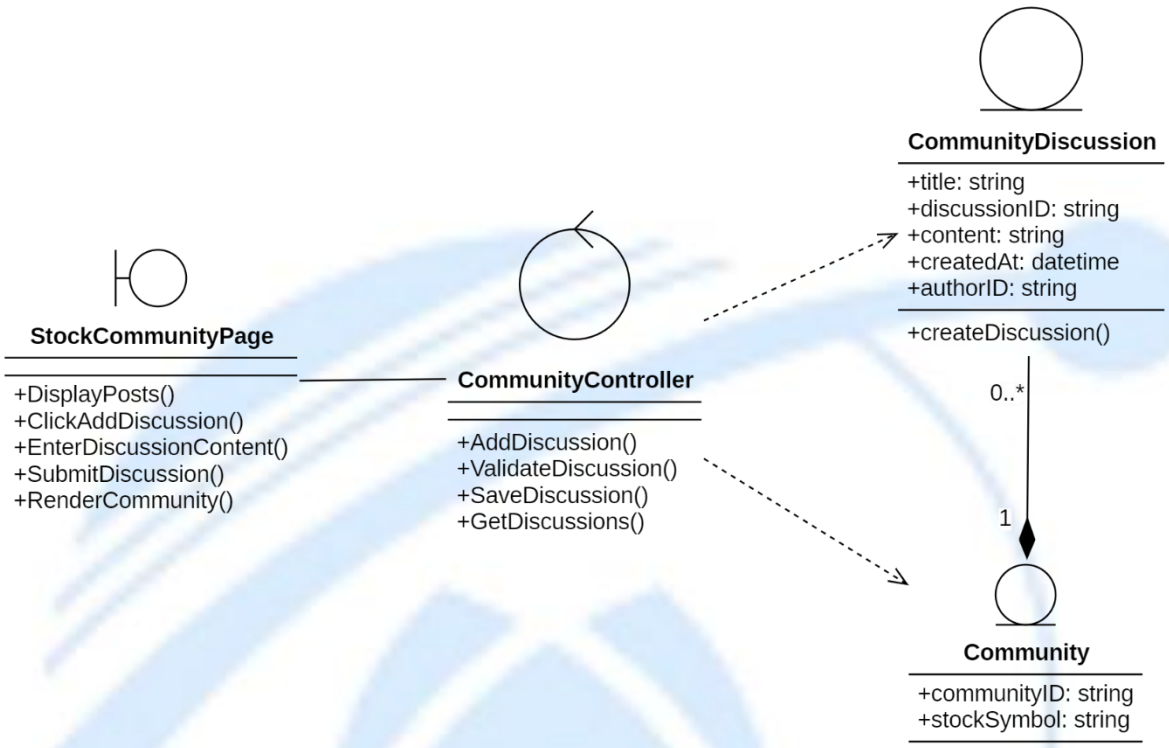
Class Diagram 12: Like Discussion

3.4.5.3.2 Add New Comment



Class Diagram 13: Add New Comment

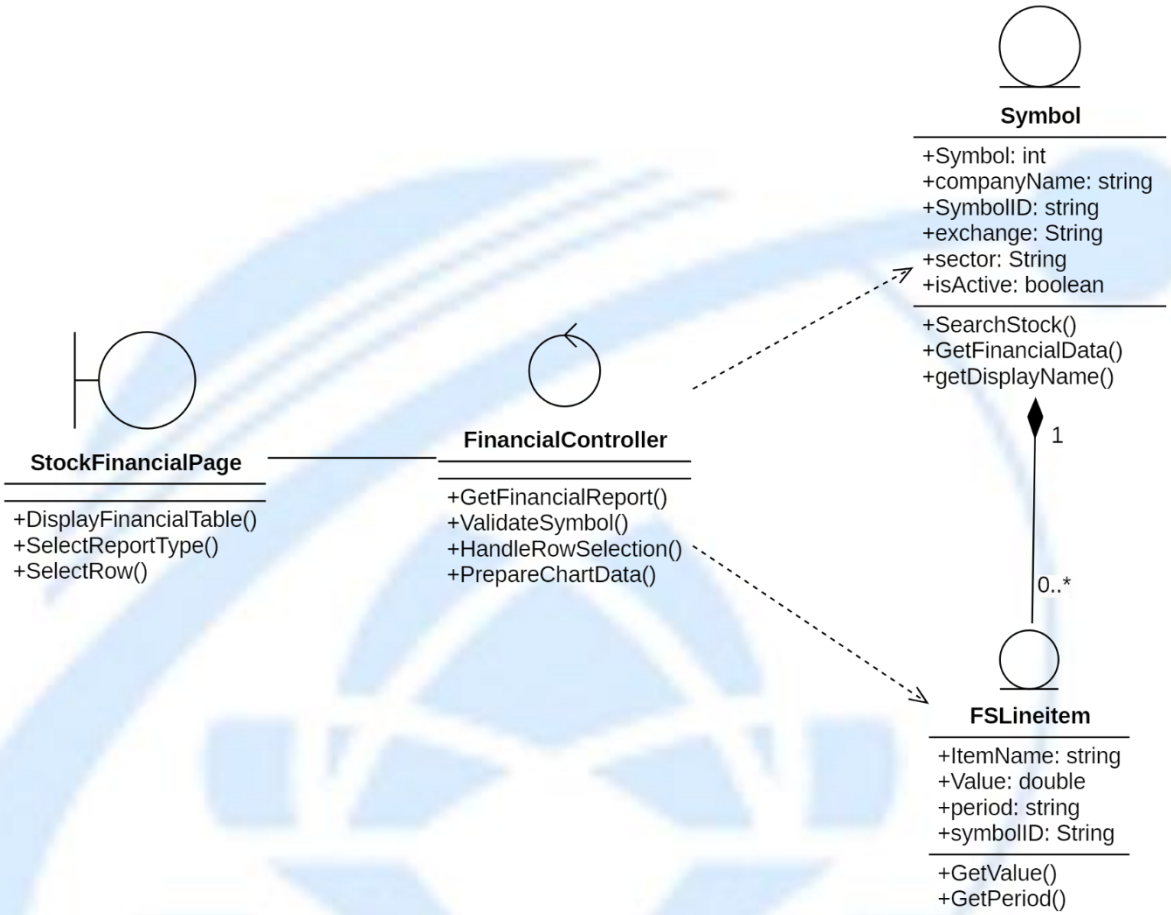
3.4.5.3.3 Create a New Discussion



Class Diagram 14: Create a New Discussion

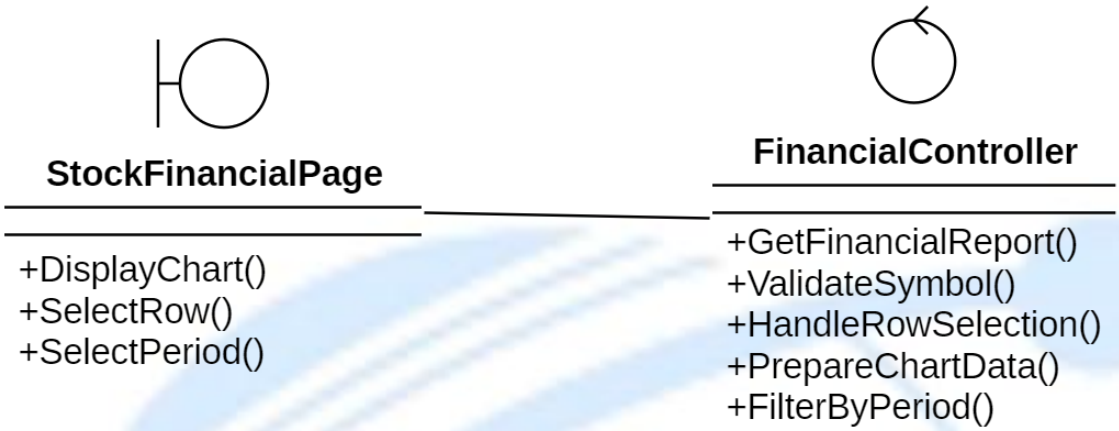
3.4.5.4. View Financials Statements

3.4.5.4.1 Change Report Type



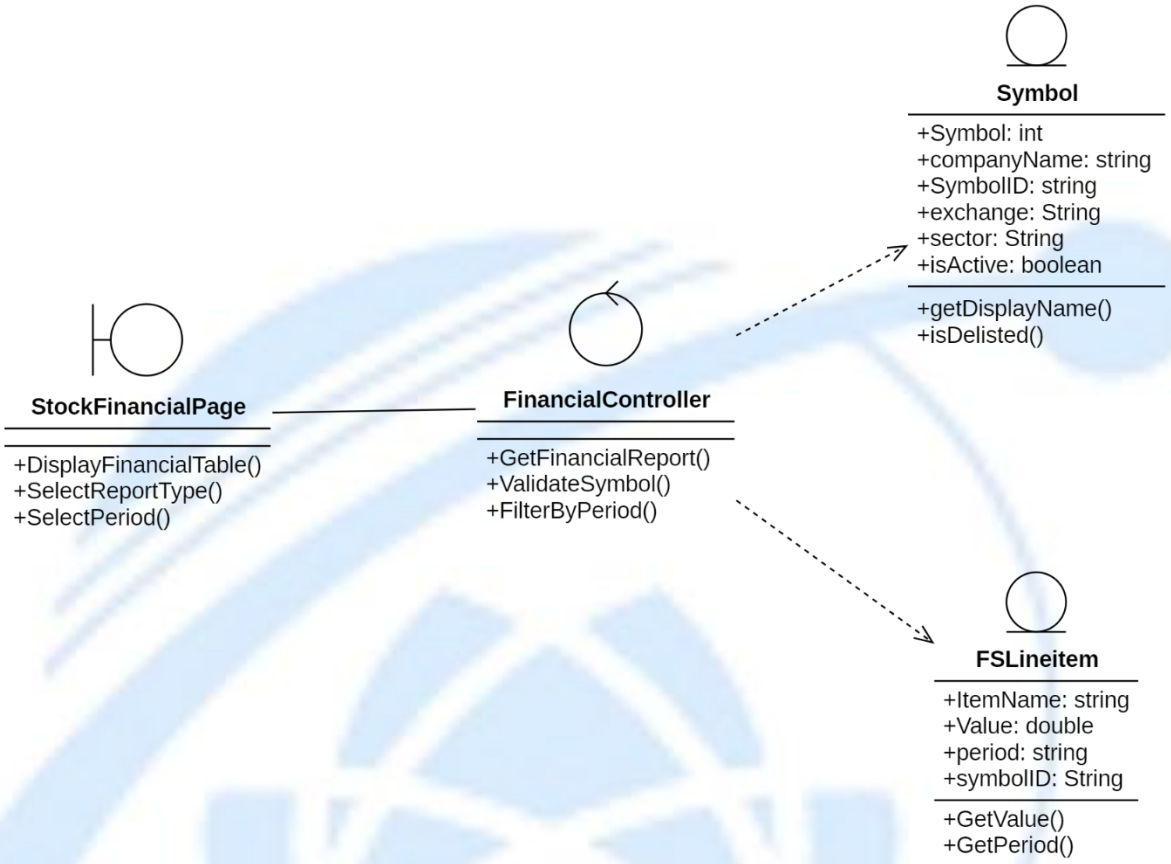
Class Diagram 15: Change Report Type

3.4.5.4.2 Visualize Data



Class Diagram 16: Visualize Data

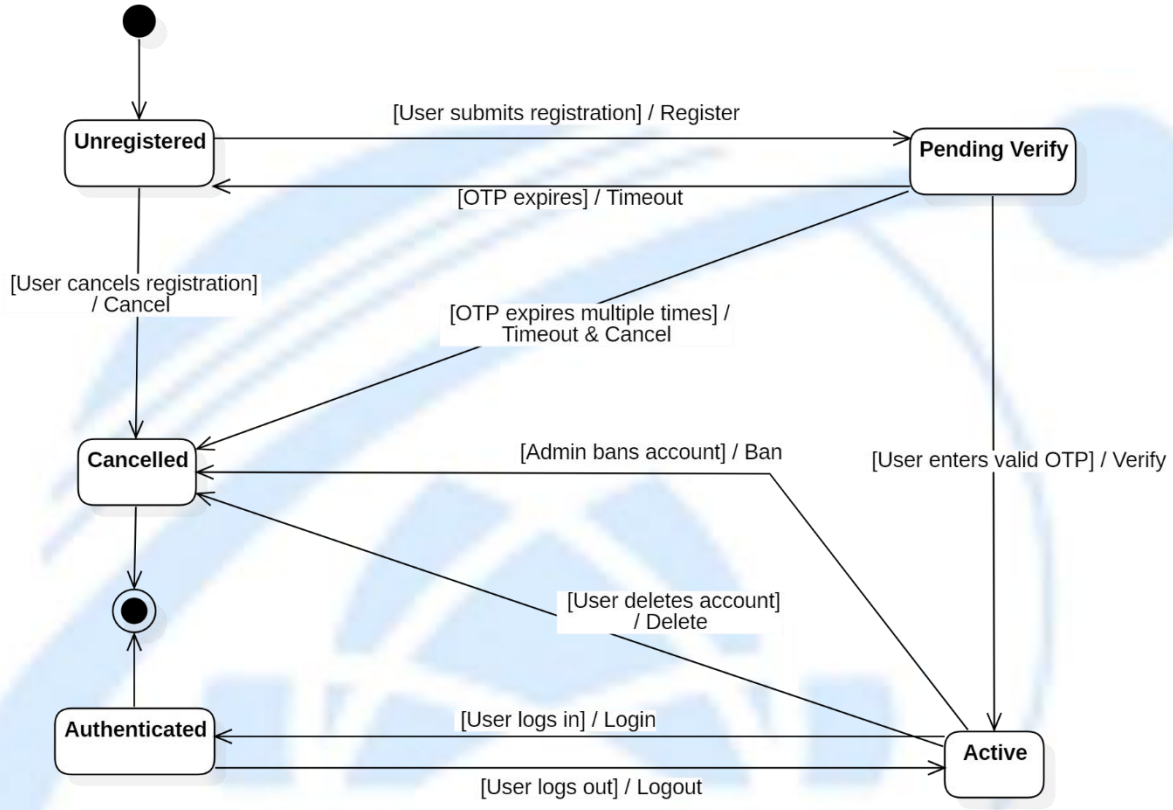
3.4.5.4.3 Select Period



Class Diagram 17: Select Period

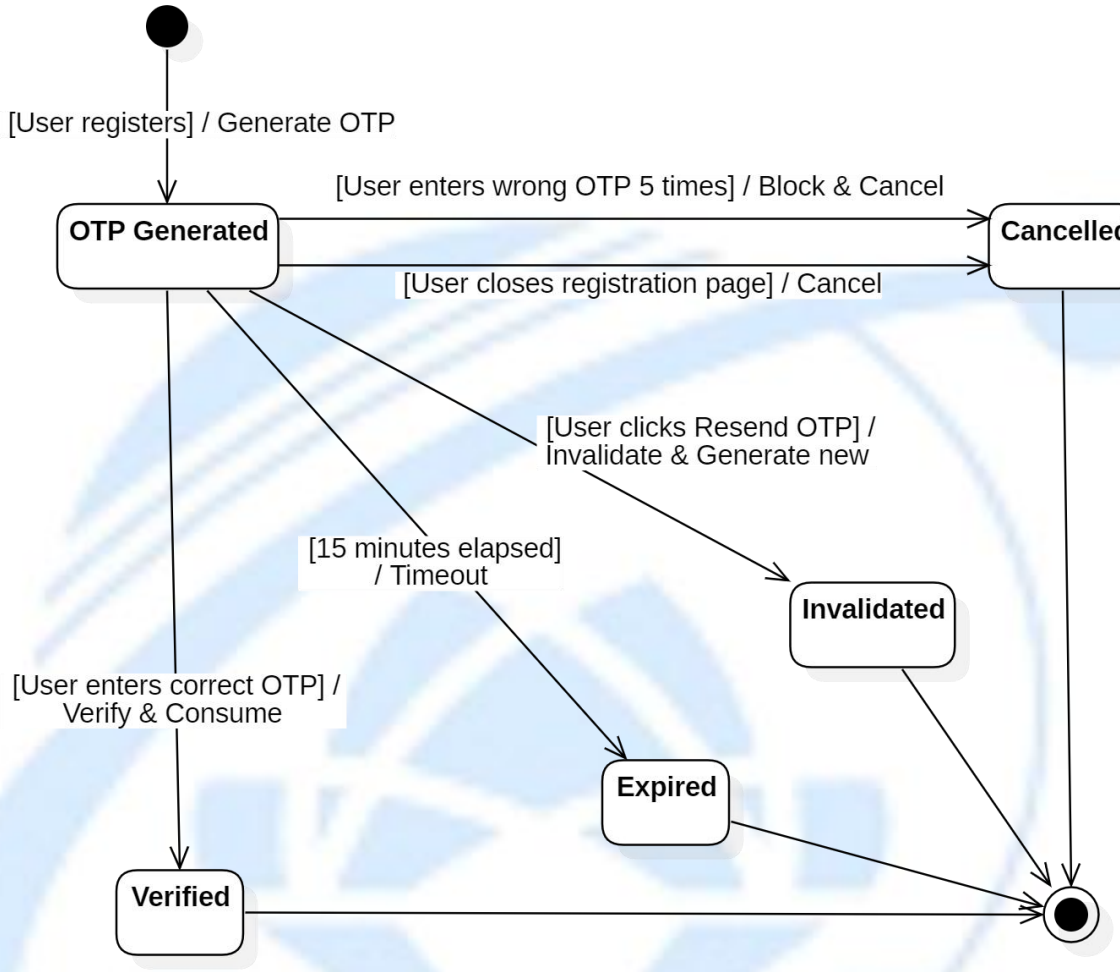
3.5. State Diagram

3.5.1. User Account



State Diagram 1: User Account

3.5.2. OTP Token



State Diagram 2: OTP Token

CHAPTER 4. Implementation, Testing, and Scalability

4.1. Development and Deployment Environment

The development and deployment environment of the Stock Portfolio Management Platform is designed to support efficient development workflows, reliable testing, and scalable system deployment. Modern development tools and containerization technologies are employed to ensure consistency across different environments and to minimize configuration-related issues throughout the project lifecycle.

During the development phase, the system is implemented using a modular microservices-based codebase, allowing individual services to be developed, tested, and maintained independently. Developers utilize integrated development environments and version control systems to manage source code, track changes, and support collaborative development. Environment configuration is managed through standardized configuration files and environment variables, ensuring that development, testing, and deployment environments remain consistent.

For deployment, the platform adopts container-based virtualization using Docker to package each microservice along with its required dependencies. This approach enables the system to be deployed consistently across different machines and environments without compatibility issues. Containers also simplify service isolation, making it easier to manage dependencies and perform independent updates without impacting other components of the system.

The platform supports local deployment for development and testing purposes, allowing developers to simulate real-world system behavior in a controlled environment. In production scenarios, container orchestration tools are used to manage service deployment, scaling, and fault recovery. These tools ensure that services can be scaled dynamically based on workload demand and that system availability is maintained even in the event of individual service failures.

Overall, the development and deployment environment is designed to provide a stable, flexible, and scalable foundation for the Stock Portfolio Management Platform. By combining standardized development practices with containerized deployment strategies, the environment supports rapid development, reliable testing, and efficient system operation, while also enabling future scalability and continuous system improvement.

4.2. User Interface

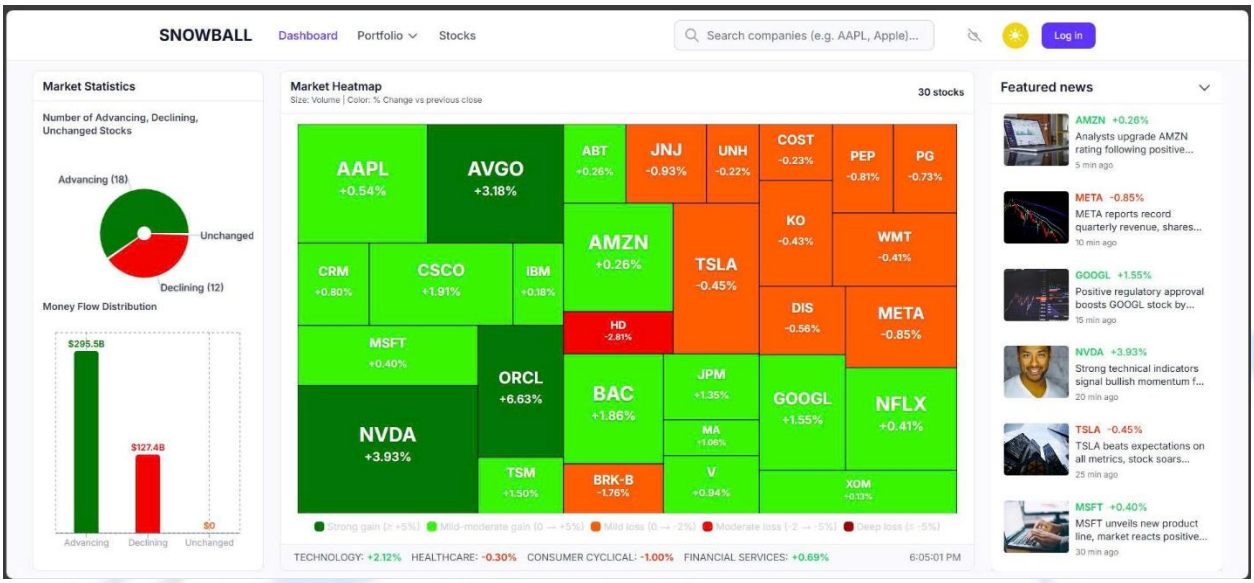


Figure 1: Home Page

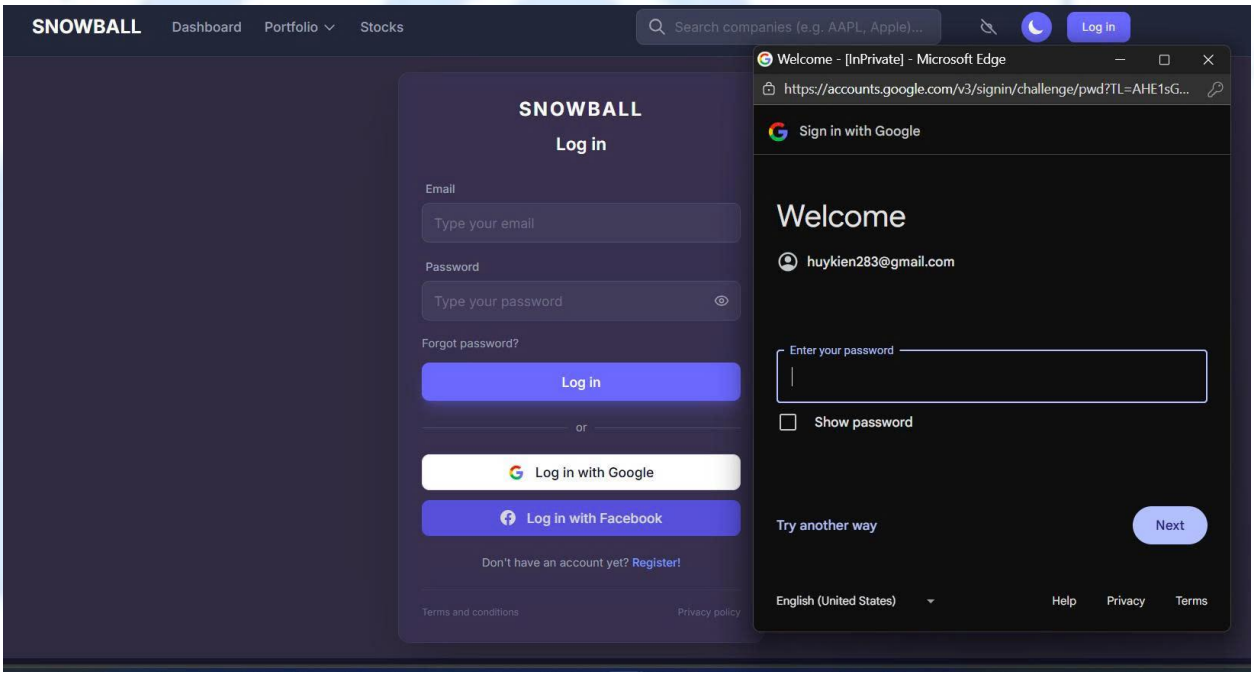


Figure 2: Login with Google

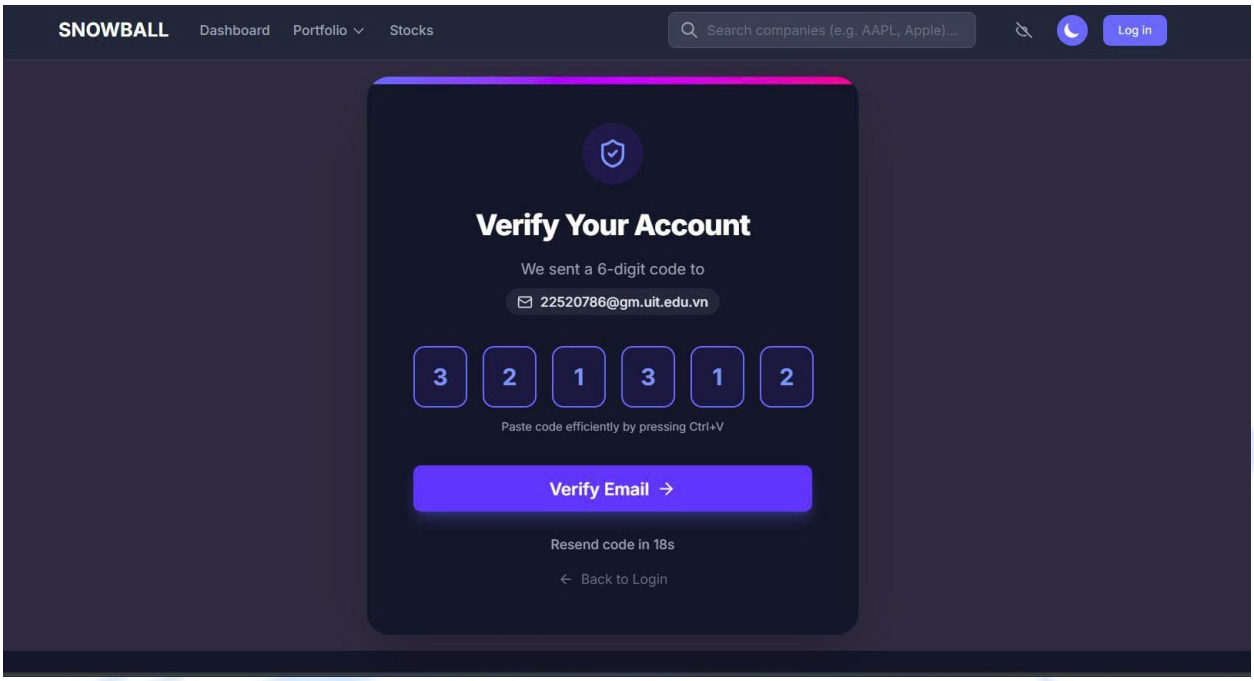


Figure 3: Verify Page

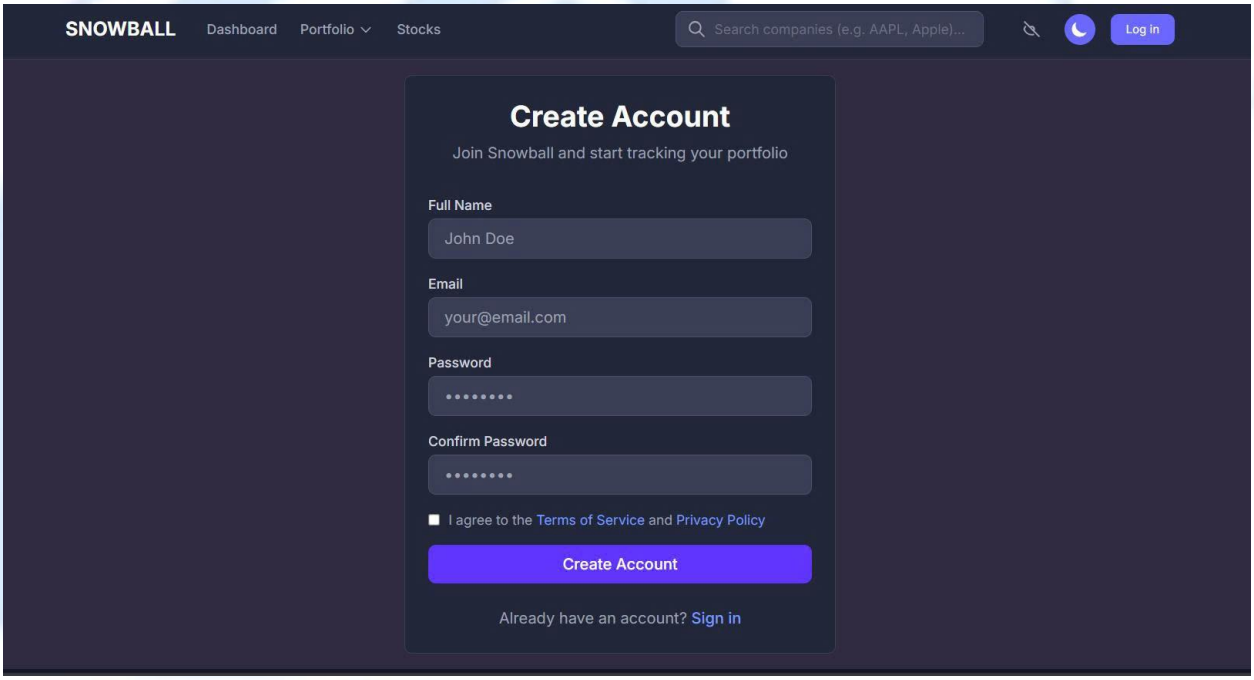


Figure 4: Register Page

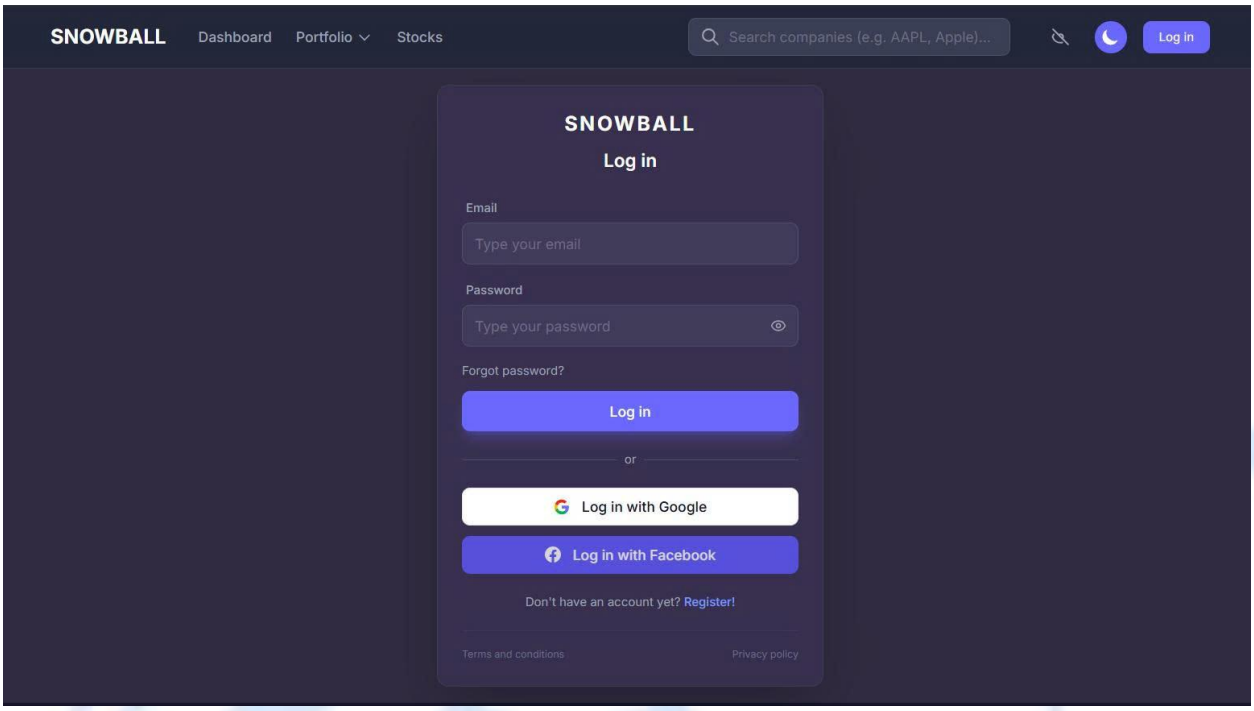


Figure 5: Login Page

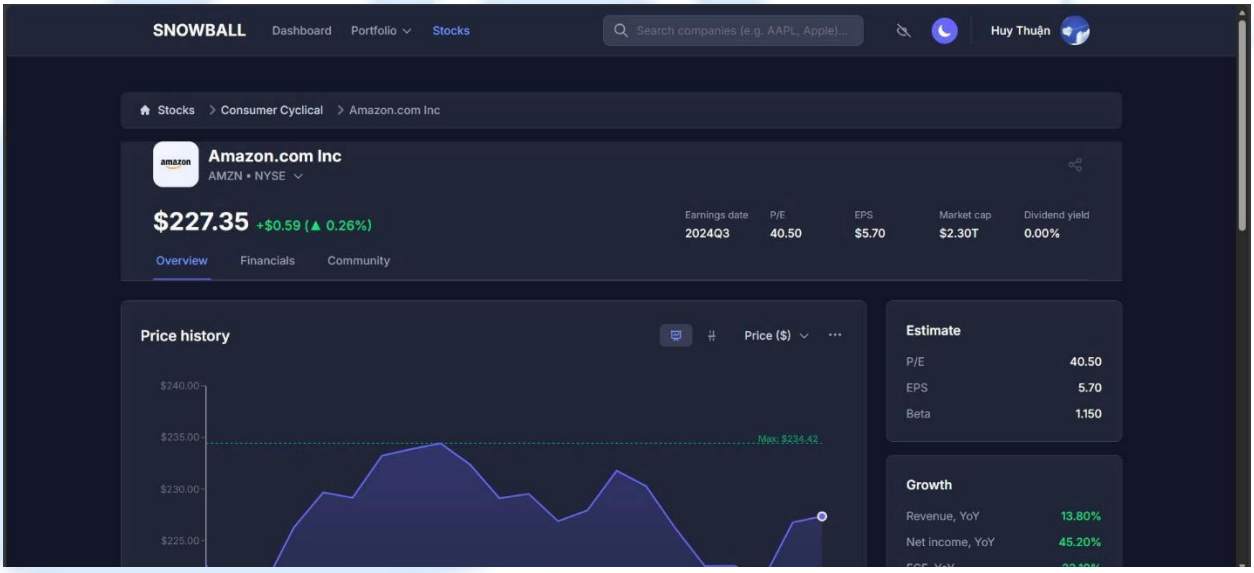


Figure 6: Stock Price Page

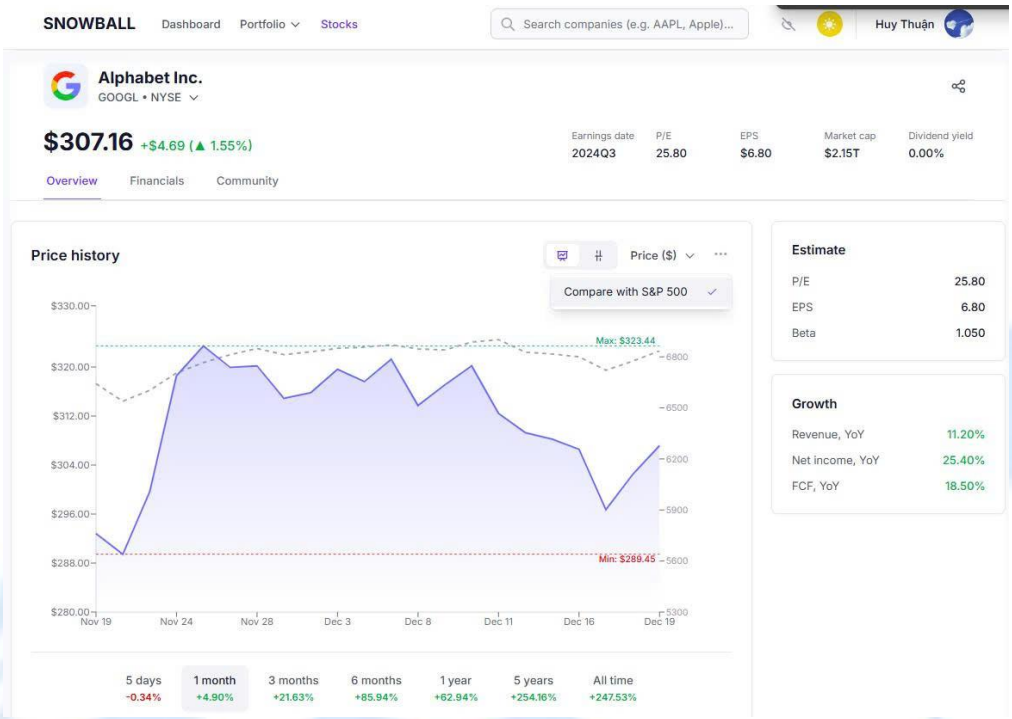


Figure 7: Stock Price Page with Light mode

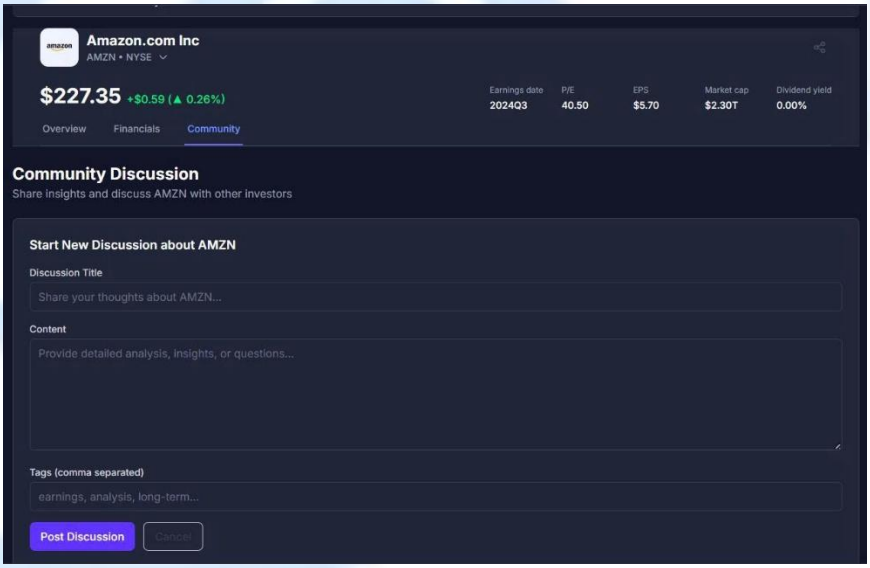


Figure 8: Create New Community Discussion Form

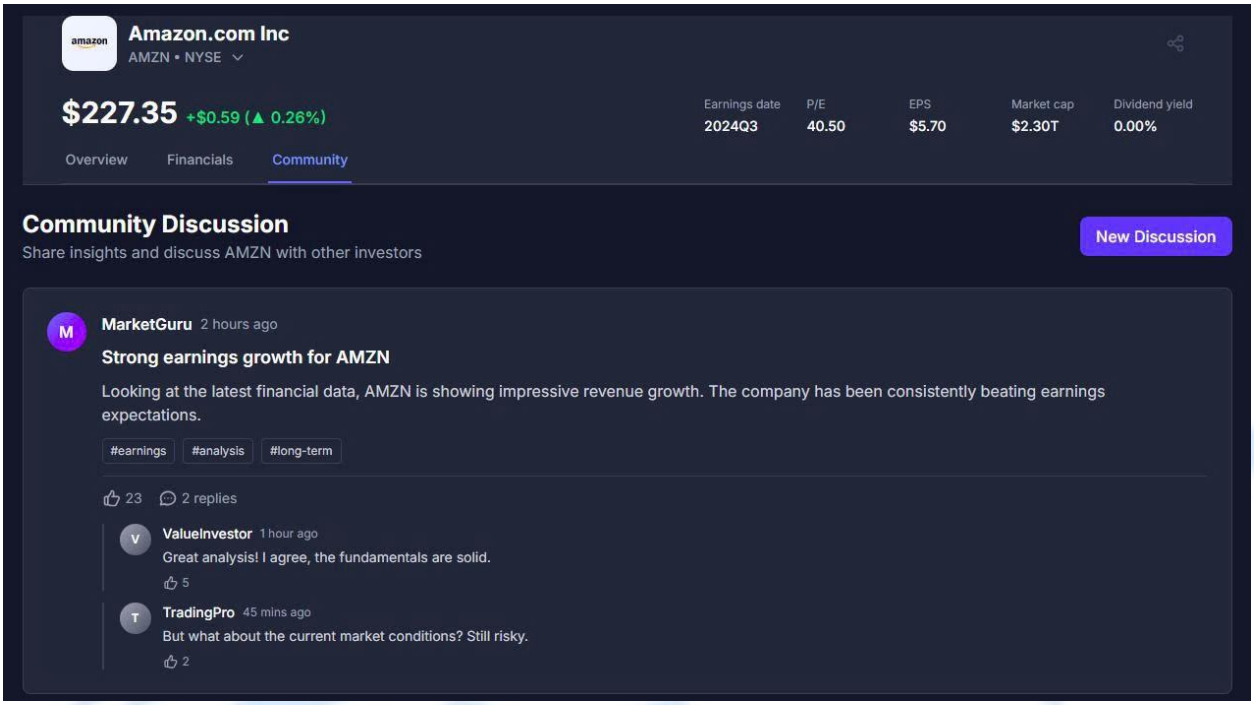


Figure 9: Stock Community Page

Figure 10: Stock Financial Page

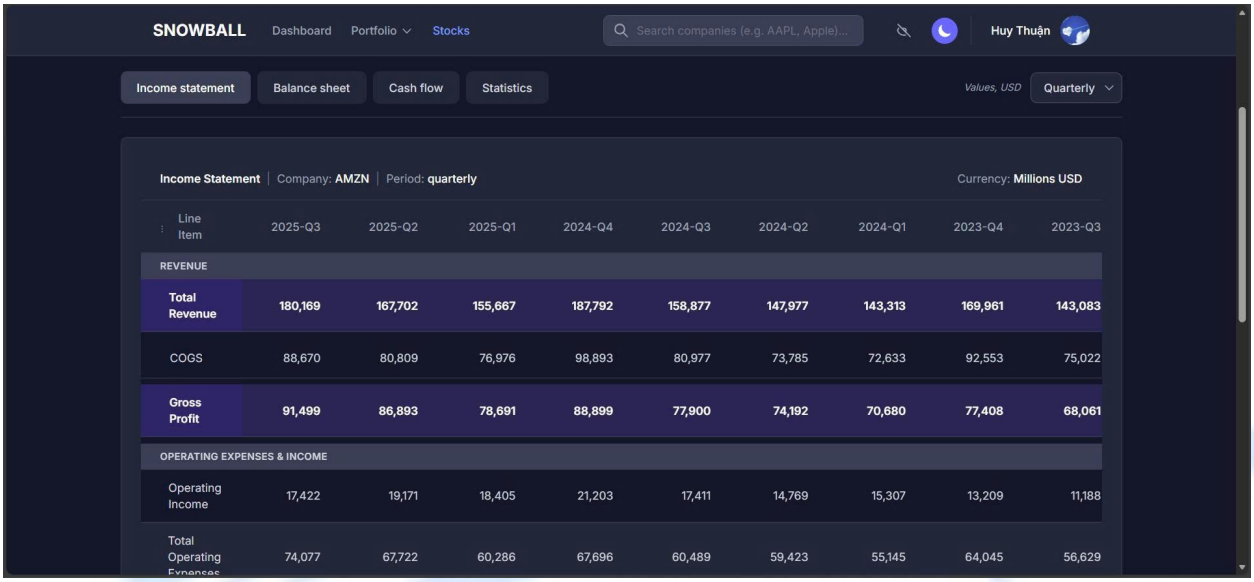


Figure 11: Stock Financial Page

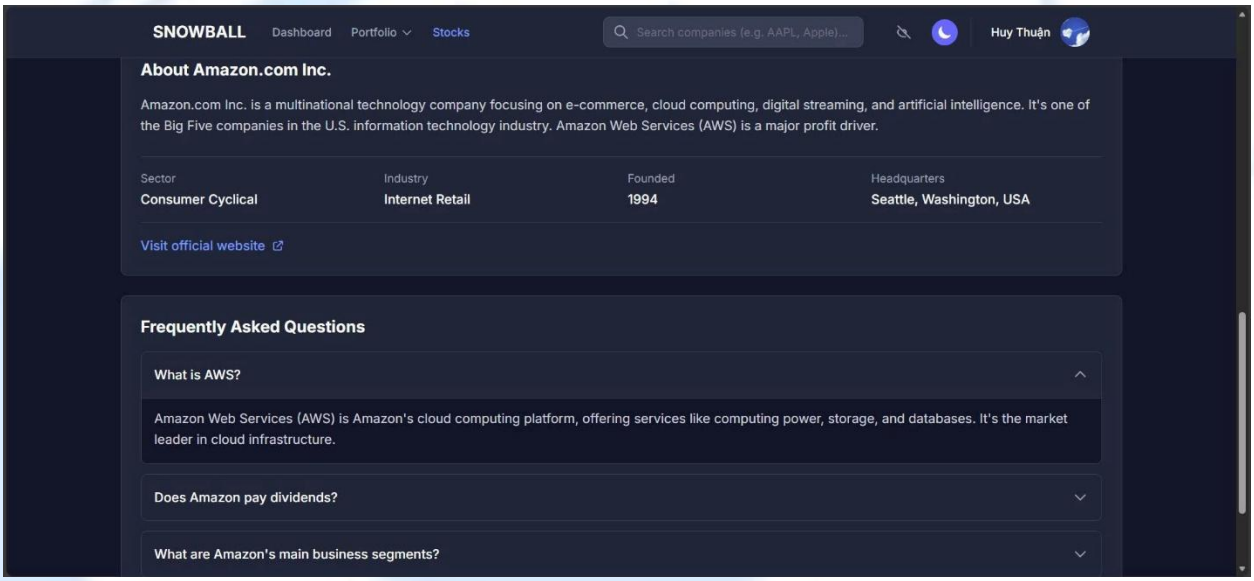


Figure 12: Stock Financial Information

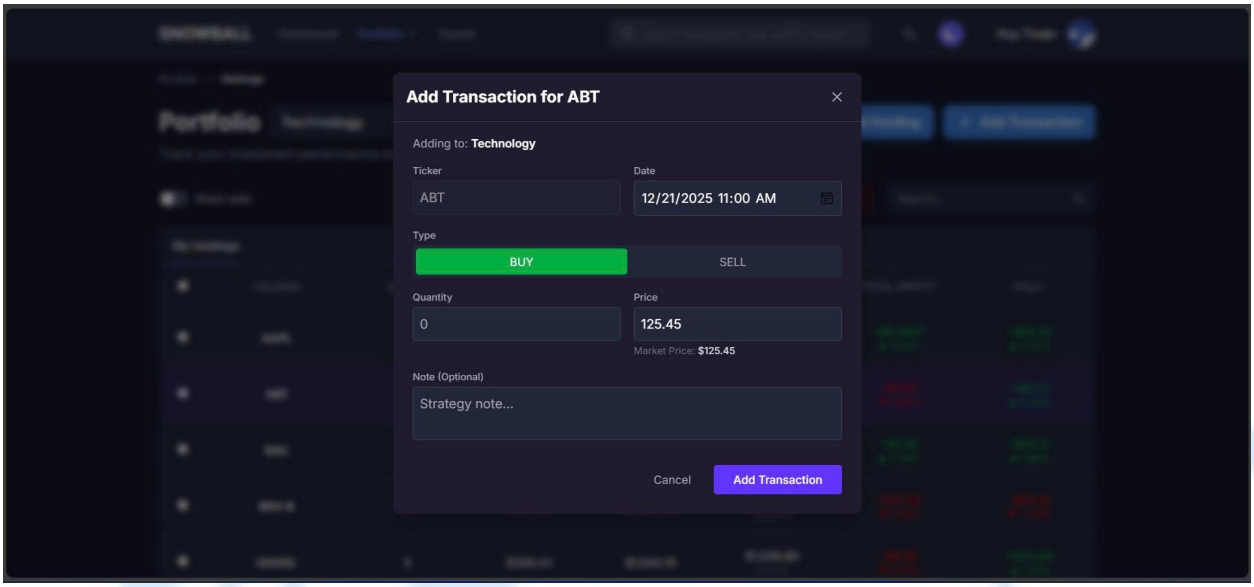


Figure 13: Create New Transaction Form

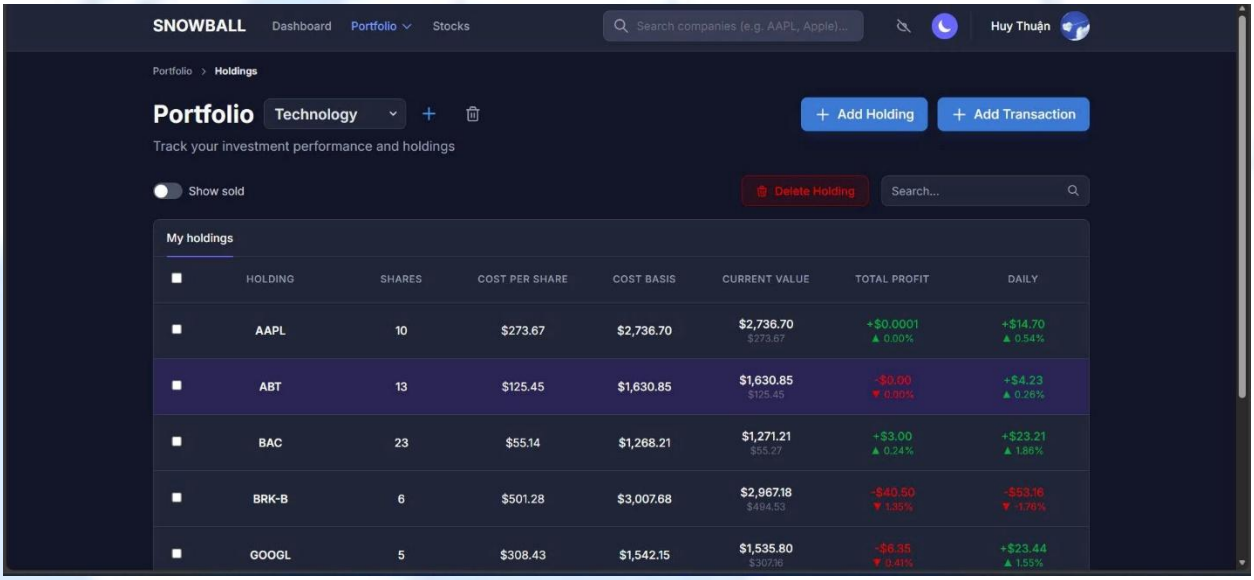


Figure 14: Portfolio Transaction Page

Figure 15: Select Portfolio

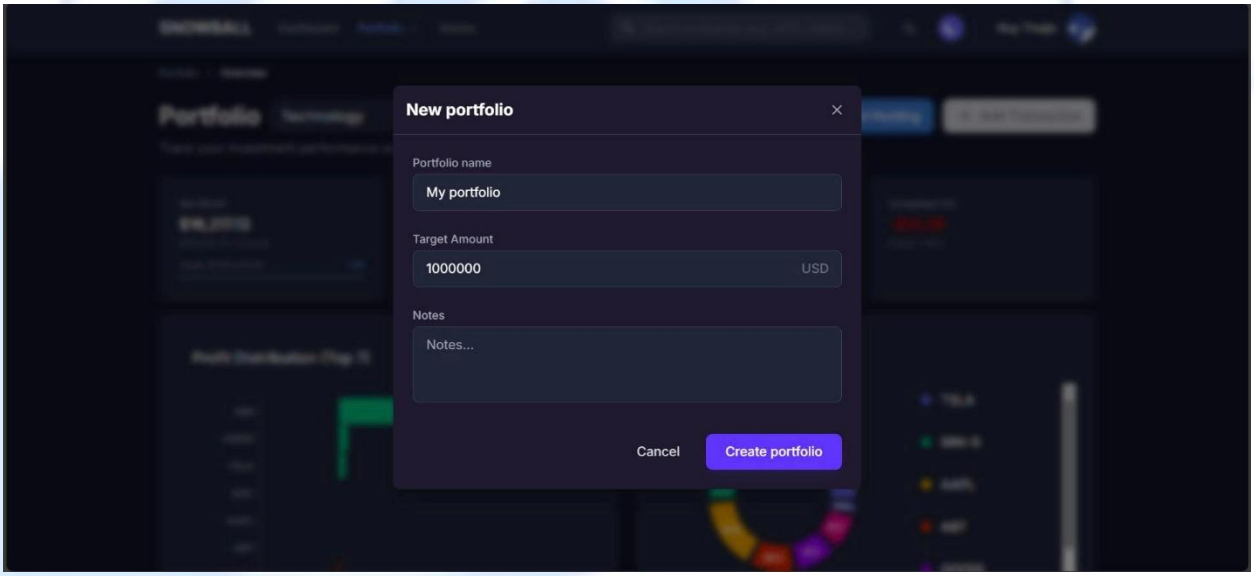


Figure 16: Create New Portfolio Form

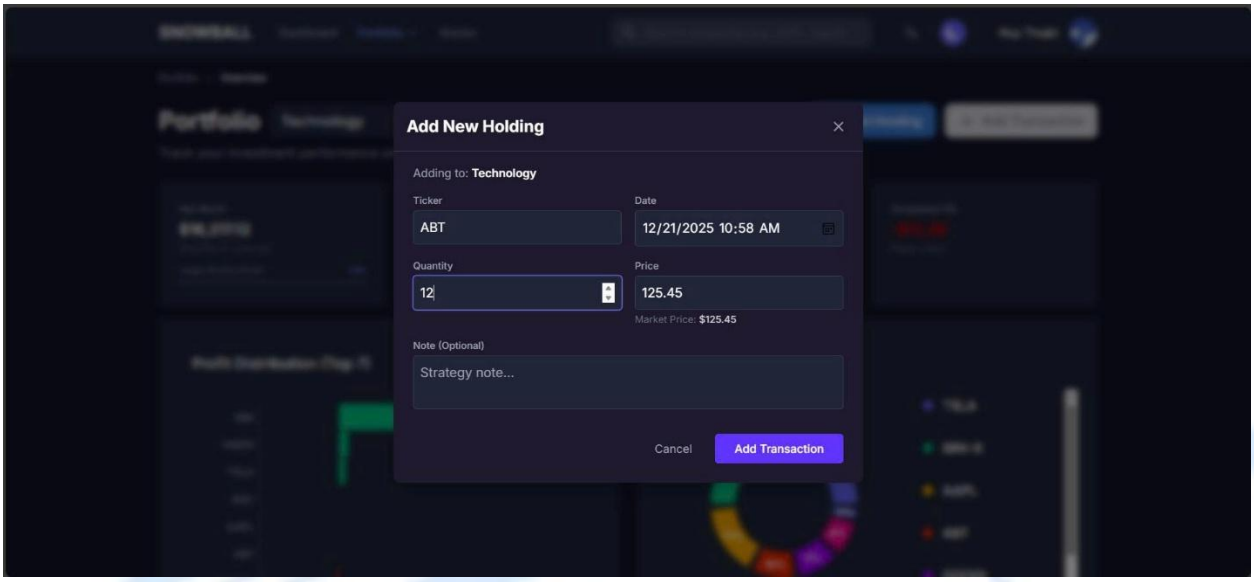


Figure 17: Create New Holding Form

Figure 18: Portfolio Overview Page

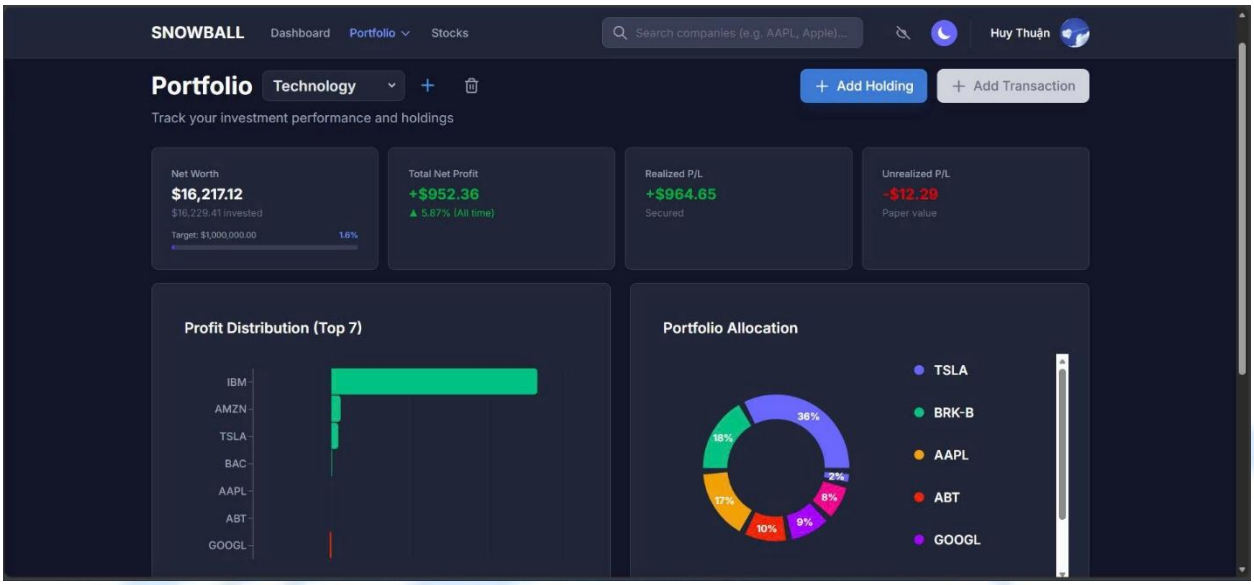


Figure 19: Portfolio Overview Page

CHAPTER 5. Conclusion

5.1. Project Evaluation

The **Stock Portfolio Management Platform** was designed and implemented with the primary objective of addressing the limitations of traditional Monolithic architectures in processing real-time financial data. Following the analysis, design, and implementation phases, the team presents the following overall evaluation:

Achievements:

- **Architectural Transformation:** The system successfully transitioned to a **Microservices architecture**, effectively decoupling core services such as *Data Ingestion*, *Portfolio Management*, and *User Authentication*. The utilization of **Docker** and **Kubernetes** has proven effective in deploying and independently scaling services based on actual workload demands.
- **Functional Fulfillment:** The system meets the key business requirements outlined in Chapter 2, specifically:
    - Processing and visualizing real-time market data with low latency via WebSocket and Redis Cache.
    - Providing intuitive portfolio management tools that allow users to track asset performance and transaction history.

  - Supporting stock analysis through historical price charts and detailed financial statements.
  - Facilitating community interaction through discussion features.

- **User Experience (UX):** The user interface is designed to be modern and user-friendly, supporting the visualization of complex data through interactive charts and heatmaps, thereby aiding investors in decision-making.

Limitations:

- **Data Source Integration:** While the system handles real-time data efficiently, integration with complex external data sources beyond Alpaca (such as Bloomberg or Reuters) has not been implemented due to time and resource constraints.
- **Trading Execution:** The functionality for direct real-time trading execution has not yet been deployed; the system currently functions primarily as a tracking and simulation tool.
- **Advanced Security:** Advanced security features, such as Two-Factor Authentication (2FA), remain incomplete.

5.2. Challenges and Lessons Learned

Throughout the project's development, the research team encountered several technical challenges and derived significant lessons:

Challenges:

- **Data Consistency in Distributed Systems:** Ensuring data consistency across microservices when using asynchronous mechanisms like **Kafka** and **Redis Streams** proved to be a complex task, requiring rigorous configuration and monitoring to prevent data loss.
- **Infrastructure Management:** Setting up and managing the **Kubernetes** environment for container orchestration required in-depth DevOps knowledge, posing difficulties during the initial stages of the project.
- **Performance Optimization:** Processing large volumes of continuous stock data streams necessitated the optimization of logic within the Backend layers (Node.js/FastAPI) to prevent bottlenecks.

Lessons Learned:

- **The Importance of System Modeling:** Detailed construction of UML diagrams (Use Case, Activity, Sequence, Class Diagrams) during the early analysis phase (Chapter 3) helped the team clearly visualize data flows and service responsibilities, significantly reducing the need for code refactoring later.

- **Microservices Mindset:** The team gained a deeper understanding of decomposing large problems into independent services, which enhanced flexibility in maintenance and upgrades compared to Monolithic architectures.
- **Collaborative Efficiency:** Effective coordination in modular development, utilizing version control tools (Git) and containerization to synchronize development environments, was crucial for progress.

5.3. Future Work

To evolve the **Stock Portfolio Management Platform** into a comprehensive and competitive solution, the team proposes the following directions for future development:

1. **Real-time Trading Execution:** Develop features allowing users to place buy/sell orders directly through API connections with stock exchanges, transforming the platform from a tracking tool into a full-fledged trading interface.
2. **AI and Machine Learning Integration:** Incorporate Advanced Analytics modules to forecast price trends, assess portfolio risks, and provide personalized investment strategies.
3. **Mobile Application Development:** Build native applications for iOS and Android to enhance accessibility, allowing investors to monitor the market from anywhere.
4. **Data Source Expansion:** Integrate additional reputable financial data providers to diversify market information, including financial news and macroeconomic data.
5. **Enhanced Security:** Implement high-level security mechanisms such as 2FA, end-to-end data encryption, and Role-Based Access Control (RBAC) to ensure maximum protection for user assets and data.

General Conclusion: This project has demonstrated the feasibility and effectiveness of applying Microservices architecture and modern system modeling techniques to the problem of stock portfolio management. It serves as a solid foundation for the continued

development of advanced features, meeting the increasing demands of the digital financial market.